Todo list

- Write a short intro to this section . . . 1
- Double check the frame conventions here. . . . 28
- Write a brief introduction summarizing the purpose of this chapter: to define the hardware, software, evaluation metrics, and exact physical and algorithmic parameters required to reproduce the global trajectory optimization benchmarks. . . . 51
- Define Computational Effort: Wall-clock CPU time, including notes on parallelization/multithreading for MC-GBO and IPSO. . . . 52
- Define Solution Quality: The final objective cost J achieved (effort for spacecraft, effort + acceleration for the manipulator). . . . 52
- show geodesic slew as proof of optimizer correctness. . . . 54
- List what workstations were used . . . 56
- State the number of independent runs, random seeds, test cases, and the workstation used for each result reported in this chapter. . . . 67
- State how failed runs, timeouts, infeasible solutions, and missing measurements are handled in the tables and plots. . . . 67
- Confirm the statistical summaries reported throughout this chapter: success rate, computation time, objective cost, constraint violation, and solution diversity. . . . 67
- Summarize the main spacecraft findings and connect them to the algorithm mechanisms discussed in § 1.6. . . . 67
- Identify which methods are most reliable, fastest, and most effective at finding low-cost or diverse solutions. . . . 67
- Make the text on this success plots larger, or make only 2 plots per row . . . 68
- probably remove the limit dashed lines on all except the normalized plot. cant see. . . . 76
- Summarize the main manipulator findings and relate them to the additional dynamic and terminal constraints. . . . 76
- Identify which methods best balance reliability, computation time, objective quality, and catch-trajectory diversity. . . . 76
- Provide a compact comparison of algorithm rankings across reliability, solution quality, computation time, and solution diversity for both models. . . . 80

Explain which conclusions generalize across models and which are specific to spacecraft attitude or manipulator ball catching. 80

Discuss limitations caused by different workstation configurations, sample counts, time limits, or test-case difficulty. 80

Summarize the most important quantitative findings without introducing new analysis. Use this section to transition to the conclusions in **??**. 80

Chair of Sensor Based Robotic Systems and Intelligent Assistance Systems

Technical University of Munich

Ryan Lacey

Comparative Performance Analysis of RRT*-GBO and Global Solvers for Constrained Nonlinear Trajectory Optimization in Robotics and Aerospace

Thesis Submitted in Partial Fulfillment of the Requirements for the Degree of Master of Science in Robotics, Mechatronics, and Biomechanical Engineering at the TUM School of Engineering and Design

Supervisor:	Prof. Dr. Alin Albu-Schäffer Chair of Sensor Based Robotic Systems and Intelligent Assistance Systems
Advisor:	Dr. Roberto Lampariello; Arne Sachtler, M.Sc.
Submitted by:	Ryan Lacey
Submission date:	2 November 2026 in Garching bei München

Task Description

Compare the Rapidly-exploring Random Tree Star with Gradient-Based Optimization (RRT*-GBO) algorithm with global solvers for constrained nonlinear trajectory optimization:

- Perform a literature review of kinodynamic motion planning algorithms with applications in robotics and aerospace.
- Utilize, verify, and adapt the DLR designed RRT*-GBO algorithm.
- Implement several other State-of-the-Art (SOTA) global solvers from the literature.
- Apply the implemented algorithms to three systems to evaluate their performance: 1. an special orthogonal group in three dimensions (SO(3)) spacecraft slew maneuver, 2. a 7 degree-of-freedom (degree of freedom (DOF)) robot manipulator, 3. an under-actuated aircraft (e.g. ducted fan).
- Implement dynamic (force and torque), kinematic (position, velocity, and acceleration), and spatial constraints (obstacle avoidance) for the evaluated systems.
- Evaluate and compare the performance of the RRT*-GBO algorithm and the implemented global solvers based on criteria such as computational efficiency, solution quality, and robustness.
- Document the findings and provide recommendations for the use of RRT*-GBO and global solvers in constrained nonlinear trajectory optimization for robotics and aerospace applications for future comparisons against machine learning-based approaches.

Abstract

Please provide an abstract in English here: `./auxfiles/abstractEN.tex`

Acknowledgments

I would like to thank the following for their support during my thesis: my advisor and supervisor, Dr. Roberto Lampariello, for the opportunity to pursue my thesis under his oversight at the German Aerospace Center (DLR) and his guidance, insight, and debates throughout the course of my research; to Dr. Caroline Specht for her assistance with the legacy RRT*-GBO code; to my colleagues and fellow thesis students at the DLR for their comradery throughout the past year both in and away from the office; and finally to my dog Timber for keeping me company on days at home and forcing me to take breaks to enjoy the sun outside.

Contents

Task Description **v**

Abstract **vii**

1 Methodology **1**

1.1 Mathematical Foundations & Representations 1

1.1.1 Attitude Representations 1

1.1.2 Manipulator Kinematics 9

1.2 B-Splines 11

1.3 Physical Models, Kinematics, Dynamics, & Constraints 14

1.3.1 Spacecraft Model 14

1.3.2 7-DOF Robotic Manipulator Model 22

1.4 Optimal Control Problem (OCP) & Nonlinear Programming (NLP) Formulation 28

1.5 Optimal Control Problem 29

1.5.1 Problem 1: Spacecraft Attitude Slew Maneuver 29

1.5.2 Problem 2: 7-DOF Manipulator Ball-Catching Maneuver 29

1.5.3 NLP Decision Variable Structure 30

1.5.4 NLP Optimization Framework 31

1.6 Algorithm Frameworks 31

1.6.1 Monte Carlo Gradient-Based Optimization (MC-GBO) 32

1.6.2 Rapidly-exploring Random Tree Star with Gradient-Based Optimization (RRT*-GBO) 34

1.6.3 Probabilistic Homotopy Optimization (P-HO) 39

1.6.4 Inverse Dynamics Particle Swarm Optimization (IPSO) 43

1.6.5 Adaptive Constraint Refinement 48

1.7 Analytical Verification 49

2 Experimental Setup **51**

2.1 Computational Environment & Software Stack . . . 51
2.2 Evaluation Metrics . . . 52
2.3 Problem 1: Spacecraft Attitude Slew . . . 53
2.3.1 Physical Parameters . . . 53
2.3.2 B-Spline Discretization . . . 53
2.3.3 Test Scenarios . . . 54
2.3.4 Algorithm-Specific Hyperparameters . . . 56
2.4 Problem 2: 7-DOF Robotic Manipulator Ball Catching . . . 59
2.4.1 Physical Parameters . . . 59
2.4.2 Target (Ball) Parameters . . . 61
2.4.3 B-Spline Discretization . . . 61
2.4.4 Test Scenarios . . . 62
2.4.5 Algorithm-Specific Hyperparameters . . . 62

3 Results **67**
3.1 Evaluation Protocol . . . 67
3.2 Spacecraft Attitude Slew Results . . . 67
3.2.1 Spacecraft Results Discussion . . . 67
3.3 Robotic Manipulator Ball-Catching Results . . . 68
3.3.1 1000lookup table (LUT)-Runs Evaluation Set . . . 68
3.3.2 100-Lut Runs Global Landscape Evaluation Set . . . 76
3.3.3 Robotic Manipulator Results Discussion . . . 76
3.4 Cross-Model Algorithm Comparison . . . 80
3.5 Chapter Summary . . . 80

A Notation **83**
A.1 Special operators and symbols . . . 83
A.2 Abbreviations and acronyms . . . 86

B 100-Run Ball Velocity Triplets **89**

C 1000-Run Ball Velocity Triplets **91**

List of Algorithms **99**

List of Figures **101**

List of Tables **103**

References **106**

Chapter 1

Methodology

Write a short intro to this section

1.1 Mathematical Foundations & Representations

1.1.1 Attitude Representations

The attitude of a rigid body in three dimensional space (3-D) space can be represented using a myriad of parameterizations, each with its own advantages and limitations. Common representations include rotation matrices, quaternions, Modified Rodrigues Parameters (MRP)s, Euler angles, and axis—angle representations. Rotation matrices are a classic choice for representing attitudes offering a singularity-free, intuitive geometric interpretation, with a linear mapping of vectors. However, they are over-parameterized with nine elements versus the minimal three required to describe rotations. This increases the computational load. Quaternions offer a more compact, four-element singularity-free representation of rotations compared to rotation matrices, but suffer from double cover and an unintuitive physical meaning of the parameters. MRPs are a three-element representation—making them a minimal and efficient description of rotations—that avoid the double-cover issue of quaternions but contain singularities at 360 degrees. Euler angles are also a minimal representation with very intuitive geometric interpretation, but they suffer from gimbal lock. Finally, axis—angle representations describe rotations using a rotation axis and an angle about that axis. They provide a clear geometric interpretation and are minimal, but they become singular at zero rotation whereby the axis-of-rotation becomes ill-defined.

Coordinate Systems

To describe the attitude of a rigid body, it is first essential to describe the coordinate systems involved. Throughout this description, we consider two coordinate systems:

- **Inertial frame:** A reference frame fixed in space, notated by the subscript **i**. The inertial frame serves as the global reference frame upon which all other reference frames, positions, and orientations are defined in relation to.

- **Body frame:** A coordinate system fixed to the rigid body, denoted by the subscript **b**, used to describe the orientation of the body relative to itself.

Vectors are expressed with respect to a specific coordinate system, with the preceding superscript indicating the frame the vector is expressed in, e.g., ${}^{\mathbf{i}}\mathbf{x}$ denotes a vector $\mathbf{x}$ expressed in the inertial frame.

While in principle, the inertial frame is fixed in space, in practice a convenient approximation is to consider merely a very slowly moving frame, such as one attached to the Earth or center of the Solar system.

Rotation Matrices

A rotation matrix—also known as a direction cosine matrix (DCM) [12]—is a matrix which, when multiplied with a vector expressed in one coordinate frame, transforms it into another coordinate frame. Alternatively, it can be seen as an operator that rotates a vector while preserving its length [5]. The former description describes a *passive* rotation, where the coordinate frame is rotated relative to a fixed vector, while the latter describes an *active* rotation, where the vector itself is rotated within a fixed coordinate frame. The passive rotation conventions will be adopted throughout this work. A second important convention is also important to clarify: This work defines these matrices as mapping the body frame to the inertial frame—as opposed to the other way around. An example of the relative orientation of the inertial and body frames is shown in Fig. 1.1.

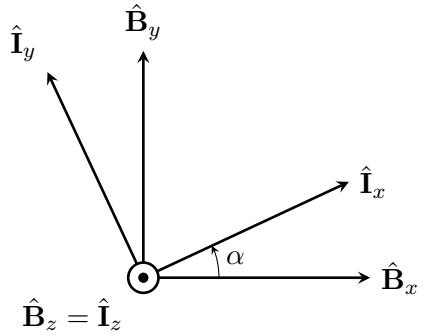


Figure 1.1: Rotation of the body frame $\{\hat{\mathbf{B}}_x, \hat{\mathbf{B}}_y, \hat{\mathbf{B}}_z\}$ by angle α about the z-axis to the inertial frame $\{\hat{\mathbf{I}}_x, \hat{\mathbf{I}}_y, \hat{\mathbf{I}}_z\}$.

Rotation matrices are the core mathematical representation used in this work to perform coordinate transformations of vectors, points, and coordinate frames. As such, a brief overview of their properties and usage is provided below.

The set of all proper orthogonal rotation matrices forms the *special orthogonal group* SO(3):

$$\mathrm{SO}(3) = \left\{ \mathbf{R} \in \mathbb{R}^{3\times 3} \mid \mathbf{R}^\top \mathbf{R} = \mathbf{I}, \det(\mathbf{R}) = 1 \right\}. \tag{1.1}$$

$$\mathbf{R}^\top \mathbf{R} = \mathbf{I} \quad \Longleftrightarrow \quad \mathbf{R}^\top = \mathbf{R}^{-1}. \tag{1.2}$$

The elements of rotation matrices are referenced by their row and column indices:

$$\mathbf{R} = \begin{bmatrix} r_{11} & r_{12} & r_{13} \\ r_{21} & r_{22} & r_{23} \\ r_{31} & r_{32} & r_{33} \end{bmatrix}. \tag{1.3}$$

Vector coordinate transformations are performed by premultiplying the *vector* $\mathbf{v} \in \mathbb{R}^3$ by a rotation matrix:

$$^{\mathrm{i}}\mathbf{v} = \mathbf{R}_{\mathrm{ib}}\,{}^{\mathrm{b}}\mathbf{v} \tag{1.4}$$

Here the vector ${}^{\mathrm{b}}\mathbf{v}$ originally expressed in the body frame is transformed into the inertial frame ${}^{\mathrm{i}}\mathbf{v}$ using the rotation matrix $\mathbf{R}_{\mathrm{ib}}$. Equation (1.4) highlights that rotation matrices provide a direct linear mapping of vectors via simple matrix multiplication.

Unlike vector transformations, transforming a *point* $\mathbf{x} \in \mathbb{R}^3$ requires both rotation and translation, as points have a fixed position in space rather than just a direction and magnitude. To demonstrate, consider a point ${}^{\mathrm{b}}\mathbf{x}$ expressed in the body frame, the origin of the body frame being ${}^{\mathrm{b}}\mathbf{x}_{\mathrm{b}}$, and the origin of the inertial frame being ${}^{\mathrm{i}}\mathbf{x}_{\mathrm{i}}$. Its coordinates in the inertial frame ${}^{\mathrm{i}}\mathbf{x}$ are obtained by applying both the rotation and translation:

$$^{\mathrm{i}}\mathbf{x} = \mathbf{R}_{\mathrm{ib}}\left({}^{\mathrm{b}}\mathbf{x} - {}^{\mathrm{b}}\mathbf{x}_{\mathrm{b}}\right) + {}^{\mathrm{i}}\mathbf{x}_{\mathrm{i}} = \mathbf{R}_{\mathrm{ib}}\,{}^{\mathrm{b}}\mathbf{x} - {}^{\mathrm{i}}\mathbf{x}_{\mathrm{b}} + {}^{\mathrm{i}}\mathbf{x}_{\mathrm{i}} \tag{1.5}$$

Naturally, the origins of the body and inertial frames expressed in their respective frames are zero vectors $\mathbf{0}$ simplifying (1.5):

$$^{\mathrm{i}}\mathbf{x} = \mathbf{R}_{\mathrm{ib}}\left({}^{\mathrm{b}}\mathbf{x} - {}^{\mathrm{b}}\mathbf{x}_{\mathrm{b}}\right) = \mathbf{R}_{\mathrm{ib}}\,{}^{\mathrm{b}}\mathbf{x} - {}^{\mathrm{i}}\mathbf{x}_{\mathrm{b}} \tag{1.6}$$

Multiple rotation matrices may be composed to represent successive rotations, multiplying from right to left. If $\mathbf{R}_{\mathrm{ib}}$ maps vectors from the body frame to the inertial frame, and $\mathbf{R}_{\mathrm{bs}}$ maps vectors from the sensor frame to the body frame, then the composite rotation matrix $\mathbf{R}_{\mathrm{is}}$ mapping vectors from the sensor frame to the inertial frame is given by:

$$\mathbf{R}_{\mathrm{is}} = \mathbf{R}_{\mathrm{ib}}\mathbf{R}_{\mathrm{bs}} \tag{1.7}$$

Rotation matrices are also known as DCMs because each element represents the cosine of the angle between the corresponding axes of the two coordinate frames.

$$\mathbf{R}_{\mathrm{ib}} = \begin{bmatrix} \hat{\mathbf{I}}_x \cdot \hat{\mathbf{B}}_x & \hat{\mathbf{I}}_x \cdot \hat{\mathbf{B}}_y & \hat{\mathbf{I}}_x \cdot \hat{\mathbf{B}}_z \\ \hat{\mathbf{I}}_y \cdot \hat{\mathbf{B}}_x & \hat{\mathbf{I}}_y \cdot \hat{\mathbf{B}}_y & \hat{\mathbf{I}}_y \cdot \hat{\mathbf{B}}_z \\ \hat{\mathbf{I}}_z \cdot \hat{\mathbf{B}}_x & \hat{\mathbf{I}}_z \cdot \hat{\mathbf{B}}_y & \hat{\mathbf{I}}_z \cdot \hat{\mathbf{B}}_z \end{bmatrix} \tag{1.8}$$

Because the dot product of two unit vectors $\hat{\mathbf{I}}$ and $\hat{\mathbf{B}}$ is:

$$\hat{\mathbf{I}}_i \cdot \hat{\mathbf{B}}_j = \|\hat{\mathbf{I}}_i\|\|\hat{\mathbf{B}}_j\| \cos\alpha_{ij} = \cos\alpha_{ij}, \quad i,j \in \{x,y,z\} \tag{1.9}$$

(1.8) may therefore be expressed in terms of the cosines of the angles between the corresponding axes. Let α_{ij} denote the angle between the inertial frame axis $\hat{\mathbf{I}}_i$ and the

body frame axis $\hat{\mathbf{B}}_j$, corresponding respectively to row i and column j of the rotation matrix (where $i, j \in \{x, y, z\}$).

$$\mathbf{R}_{\text{ib}} = \begin{bmatrix} \cos\alpha_{xx} & \cos\alpha_{xy} & \cos\alpha_{xz} \\ \cos\alpha_{yx} & \cos\alpha_{yy} & \cos\alpha_{yz} \\ \cos\alpha_{zx} & \cos\alpha_{zy} & \cos\alpha_{zz} \end{bmatrix} \tag{1.10}$$

example: The equation of the rotation shown in Fig. 1.1

$$\mathbf{R}_{\text{ib}} = \begin{bmatrix} \cos\alpha & \cos(\pi/2+\alpha) & \cos(\pi/2) \\ \cos(\pi/2-\alpha) & \cos\alpha & \cos(\pi/2) \\ \cos(\pi/2) & \cos(\pi/2) & \cos(0) \end{bmatrix} = \begin{bmatrix} \cos\alpha & -\sin\alpha & 0 \\ \sin\alpha & \cos\alpha & 0 \\ 0 & 0 & 1 \end{bmatrix} \tag{1.11}$$

We conclude with a brief discussion of the drawbacks and benefits of using rotation matrices for attitude representation.

Pros

- Direct Mapping: Rotation matrices provide a direct linear mapping of vectors via matrix multiplication.
- Singularity-Free: They are free from geometric singularities.
- Intuitive Kinematics: They provide a clear geometric interpretation of rotational kinematics whereby elements directly correspond to the relationship between frame unit vectors.
- Unique: Each unique 3-D rotation corresponds to exactly one matrix in SO(3).

Cons

- Redundant: They use nine elements to represent only three degrees of freedom.
- Orthogonality and Normalization: Maintaining the orthogonality and normalization of rotation matrices can be challenging due to numerical errors, often requiring re-canonization.
- Interpolation: Interpolating between rotation matrices is not straightforward and often requires conversion to other representations like quaternions for smooth interpolation.

Homogeneous Transformation Matrices

A common extension of the rotation matrix representation is the homogeneous transformation matrix [5], often used in robotics, which incorporates both rotation and translation components as described in (1.6). A homogeneous transformation matrix $\mathbf{T}_{\text{ib}}$ can be expressed as:

$$\mathbf{T}_{\text{ib}} = \begin{bmatrix} \mathbf{R}_{\text{ib}} & {}^{\text{i}}\mathbf{x}_{\text{b}} \\ \mathbf{0}_{3\times1}^{\mathsf{T}} & 1 \end{bmatrix} \in \text{SE}(3), \tag{1.12}$$

where $\mathbf{R}_{\text{ib}} \in \text{SO}(3)$ and ${}^{\text{i}}\mathbf{x}_{\text{b}} \in \mathbb{R}^3$ is the position of the body frame origin relative to the inertial frame's origin, expressed in the inertial frame. The homogeneous transformation matrix $\mathbf{T}_{\text{ib}}$ thus defines the *special Euclidean group* SE(3), which encompasses all possible rigid body transformations in three-dimensional space. Like rotation matrices, homogeneous transformation matrices may be multiplied from right to left to compose multiple transformations, allowing the transformation from one frame to another through intermediate frames.

The inverse of a homogeneous transformation matrix can be computed as:

$$\mathbf{T}_{\text{ib}}^{-1} = \mathbf{T}_{\text{bi}} = \begin{bmatrix} \mathbf{R}_{\text{ib}}^{\mathsf{T}} & -\mathbf{R}_{\text{ib}}^{\mathsf{T}}\,{}^{\text{i}}\mathbf{x}_{\text{b}} \\ \mathbf{0}_{3\times 1}^{\mathsf{T}} & 1 \end{bmatrix}. \tag{1.13}$$

The homogeneous transformation, along with the Denavit-Hartenberg (DH) parameters, provides a systematic way to describe the kinematics of robotic manipulators and the various transformations between different coordinate frames.

Quaternions

Quaternions, specifically *unit* quaternions are an efficient representation of rotations very commonly used in aerospace and robotics. They form the backbone of this work as it applies to optimization problems on SO(3). Trajectories and the corresponding attitude kinematics and dynamics for the spacecraft slew maneuvers are parameterized using unit quaternions. However, while the trajectories are parameterized using unit quaternions, rotation matrices are still used for most computations and transformations; as such, this section covers the mathematical properties and relations with respect to their application to this work. A more complete description of quaternions, their algebra, and their relationship to rotation matrices can be found in [5].

A quaternion, $\mathbf{q} \in \mathbb{H}$, consists of a scalar part q_s and a vector part $\mathbf{q_v} = [q_x, q_y, q_z]^{\mathsf{T}} \in \mathbb{R}^3$ which together form a four dimensional space (4-D) vector [5, 12] as depicted in Fig. 1.2. Two conventions exist: a scalar first convention and a scalar last convention; this work utilizes the scalar last convention:

$$\mathbf{q} = \begin{bmatrix} q_x \\ q_y \\ q_z \\ q_w \end{bmatrix} = \begin{bmatrix} \mathbf{q_v} \\ q_s \end{bmatrix} = \begin{bmatrix} sin(\theta/2)\hat{\mathbf{u}} \\ cos(\theta/2) \end{bmatrix} \tag{1.14}$$

Generally, quaternions both rotate *and scale* vectors. However, a subset of quaternions, known as *unit quaternions*, only rotate vectors without scaling them. Unit quaternions satisfy:

$$\|\mathbf{q}\|_2 = \sqrt{q_x^2 + q_y^2 + q_z^2 + q_w^2} = 1. \tag{1.15}$$

The unit norm constraint is a critical property of our optimizations ensuring that the quaternions represent valid elements of SO(3).

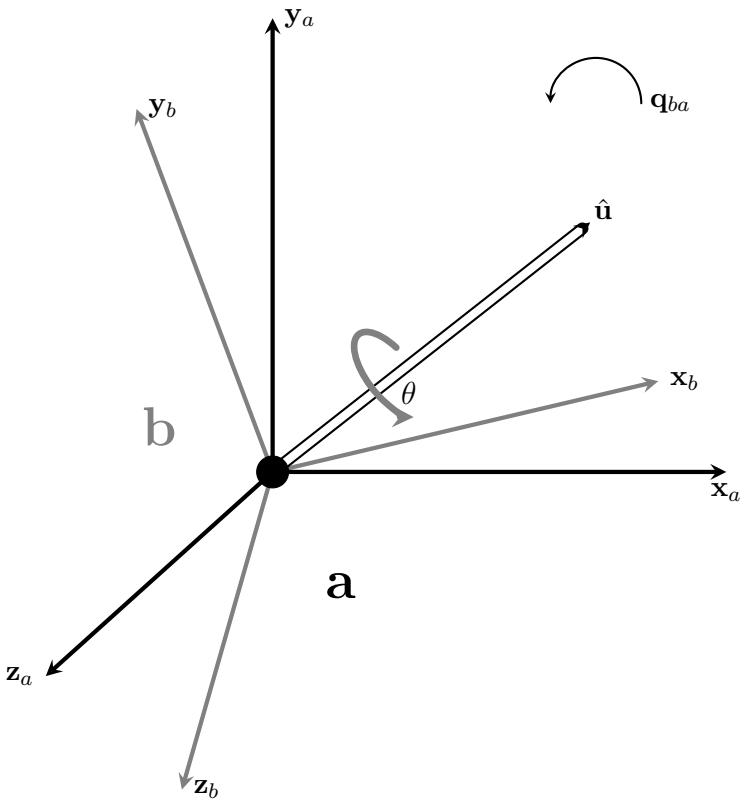


Figure 1.2: Initial frame a, rotated frame b, unit rotation axis $\hat{\mathbf{u}}$, rotation angle θ, and unit quaternion rotation $\mathbf{q}_{ba}$.

Quaternion multiplication is used to represent relative rotations between orientations, where $\mathbf{q}_{\text{relative}}$ transforms the initial frame into the identity orientation, represented by the identity quaternion $\mathbf{q}_{\text{identity}} = [\mathbf{0}_{3\times1}^{\mathsf{T}}, 1]^{\mathsf{T}}$.

$$\overline{\mathbf{q}} = \begin{bmatrix} -\mathbf{q_v} \\ q_s \end{bmatrix}, \tag{1.16}$$

$$\mathbf{q}^{-1} = \frac{\overline{\mathbf{q}}}{\|\mathbf{q}\|_2^2}. \tag{1.17}$$

For unit quaternions the inverse equals the conjugate, i.e., $\mathbf{q}^{-1} = \overline{\mathbf{q}}$. The relative rotation quaternion is then:

$$\mathbf{q}_{\text{relative}} = \mathbf{q}_{\text{initial}}^{-1} \otimes \mathbf{q}_{\text{final}}, \tag{1.18}$$

$$\mathbf{q}_1 \otimes \mathbf{q}_2 = \begin{bmatrix} q_{s,1}\mathbf{q}_{v,2} + q_{s,2}\mathbf{q}_{v,1} + \mathbf{q}_{v,1} \times \mathbf{q}_{v,2} \\ q_{s,1}q_{s,2} - \mathbf{q}_{v,1} \cdot \mathbf{q}_{v,2} \end{bmatrix} \tag{1.19}$$

A relative quaternion, $\mathbf{q}_{\text{relative}}$, represents the quaternion that rotates from the initial orientation to the final orientation and is useful for the mapping of MRPs to SO(3).

Quaternions have a property known as the *double-cover*:

$$\mathbf{R}(\mathbf{q}) = \mathbf{R}(-\mathbf{q}) \in \text{SO}(3), \tag{1.20}$$

meaning that both represent the same rotation. To avoid ambiguity in representing rotations, it is common practice to enforce a convention, such as requiring the scalar part of the quaternion to be non-negative, i.e., $q_s \geq 0$.

$$\mathbf{q} \to \begin{cases} \mathbf{q}, & q_s \geq 0 \\ -\mathbf{q}, & q_s < 0 \end{cases} \tag{1.21}$$

Enforcing this convention ensures a unique representation for each rotation and obviates an optimized slew taking the longer path due to quaternion sign ambiguity. Along with the unit norm constraints and the scalar last convention, quaternion negation functions as the final condition of canonical quaternion representation for this work. Enforcing these conditions is known as quaternion canonization.

A final utility function is the conversion from quaternions to rotation matrices:

$$R_{\text{bi}}(q) = \begin{bmatrix} q_w^2 + q_x^2 - q_y^2 - q_z^2 & 2(q_xq_y + q_wq_z) & 2(q_xq_z - q_wq_y) \\ 2(q_xq_y - q_wq_z) & q_w^2 - q_x^2 + q_y^2 - q_z^2 & 2(q_yq_z + q_wq_x) \\ 2(q_xq_z + q_wq_y) & 2(q_yq_z - q_wq_x) & q_w^2 - q_x^2 - q_y^2 + q_z^2 \end{bmatrix}. \tag{1.22}$$

Like rotation matrices, quaternions have their own set of pros and cons:

Pros

- Singularity-free: Avoids the inherent singularities present in all smooth minimal attitude representations, allowing for continuous and well-defined attitude representation on the entirety of SO(3).

- Computationally Efficient: Contain only four elements—far fewer parameters than rotation matrices—and rely purely upon algebraic operations for frame transformations, avoiding transcendental trigonometric functions.
- Smooth Interpolation: Allow for smooth interpolation between rotations using Spherical Linear Interpolation (SLERP).

Cons

- Unit Norm Constraint: Only unit quaternions represent valid rotations, necessitating normalization after numerical operations to maintain the unit norm. The unit norm constraint also poses challenges in optimization frameworks as the feasible set exists on a 3-D manifold in 4-D space.
- Double Cover: q and $-q$ map to the same physical rotation, introducing ambiguity that must be managed via consistent canonization, especially in optimization algorithms to avoid taking longer paths around the sphere.
- Redundant: They use four elements versus the three element minimal representation.
- Geometrically Confusing: Being four-dimensional hyperspheres, quaternions are not easily conceptualized in comparison to other attitude representations.

MRPs

MRPs represent the final attitude representation discussed in this work, offering a three-parameter minimal parameterization of rotations derived as stereographic projections of the unit quaternion hypersphere onto a 3-D hyperplane [12].

MRPs were introduced in this work to implement the Inverse-dynamics Particle Swarm Optimization (IPSO) algorithm—which due to characteristics of how Particle Swarm Optimization (PSO) algorithms explore search spaces, made quaternions, which require the unit norm constraint, impractical for this algorithm class.

This representation is defined in terms of the rotation axis $\hat{\mathbf{u}}$ and the rotation angle θ [12] as shown in Fig. 1.2:

$$\boldsymbol{\sigma}(\theta, \hat{\mathbf{u}}) = \tan\left(\frac{\theta}{4}\right)\hat{\mathbf{u}} = \frac{\mathbf{q}_v}{1 + q_w}, \tag{1.23}$$

As a minimal representation, MRPs avoid redundancy inherent to quaternions and rotation matrices, but succumb unavoidably to either singularities or discontinuities. From (1.23), it is evident that MRPs become singular when the rotation angle θ approaches 2π, as $\tan\left(\frac{\theta}{4}\right)$ approaches infinity. If the angle of rotation is bounded less than 2π, MRPs provide a continuous, singularity-free representation of rotations; however, spacecraft maneuvers are not constrained as such and singularities may appear [12]. One common approach to mitigate this issue is to switch to the shadow set of MRPs when the norm of the current set exceeds unity—occurring for rotations greater than π—effectively avoiding the singularity at 2π, but this introduces instead a discontinuity that optimizers struggle to handle.

Conversion between MRPs and quaternions or rotation matrices occur via:

$$\mathbf{q}(\boldsymbol{\sigma}) = \frac{1}{1+\sigma^2}\begin{bmatrix} 2\boldsymbol{\sigma} \\ 1-p^2 \end{bmatrix} \tag{1.24}$$

$$\mathbf{R}_{bi}(\boldsymbol{\sigma}) = I_{3\times 3} - \frac{4(1-\sigma^2)}{(1+\sigma^2)^2}[\boldsymbol{\sigma}]_\times + \frac{8}{(1+\sigma^2)^2}[\boldsymbol{\sigma}]_\times^2 \tag{1.25}$$

Naturally, MRPs come with their own set of tradeoffs:

Pros

- Minimal Representation: Uses exactly three parameters to describe three DOF rotations, eliminating the unit-norm constraint required by quaternions.
- Extended Singularity: As opposed to classic Rodrigues parameters, MRPs exhibit singularities only at rotations of 2π rather than π, providing a larger singularity-free region for practical rotations.
- Transcendental-free Kinematics: Unlike Euler angles, the kinematic differential equations for MRPs do not involve trigonometric functions.

Cons

- Singular or Discontinuous: Avoiding singularities involves switching to the shadow set for rotations greater than π, which can introduce discontinuities; without the shadow set, the representation is smooth but contains singularities.
- Spatial Warping: Due to MRPs being a *stereographic projection* of a 4-D hypersphere onto a 3-D hyperplane, straight paths in the MRP space *generally* do not correspond geodesics (great circles) on SO(3); a straight line trajectory only yields a true geodesic if the trajectory passes through the origin ($\boldsymbol{\sigma} = \mathbf{0}$) and typically requires the use of relative rotations between coordinate frames to help align maneuvers with the origin. The distortion induces a change in the loss landscape, which affects optimization and control strategies.

1.1.2 Manipulator Kinematics

To describe the kinematics of the 7-DOF robotic arm, the Modified Denavit-Hartenberg (MDH) parameter convention is employed to relate consecutive link coordinate frames and systematically generate homogeneous transformation matrices in (1.12). The MDH convention was introduced by John J. Craig [4], and is a slight reformation of the standard DH convention, where the coordinate frames are attached differently to the links, primarily in how the parameters are defined relative to the link origins. MDH uses four parameters to describe the 6-DOF pose between link frames.

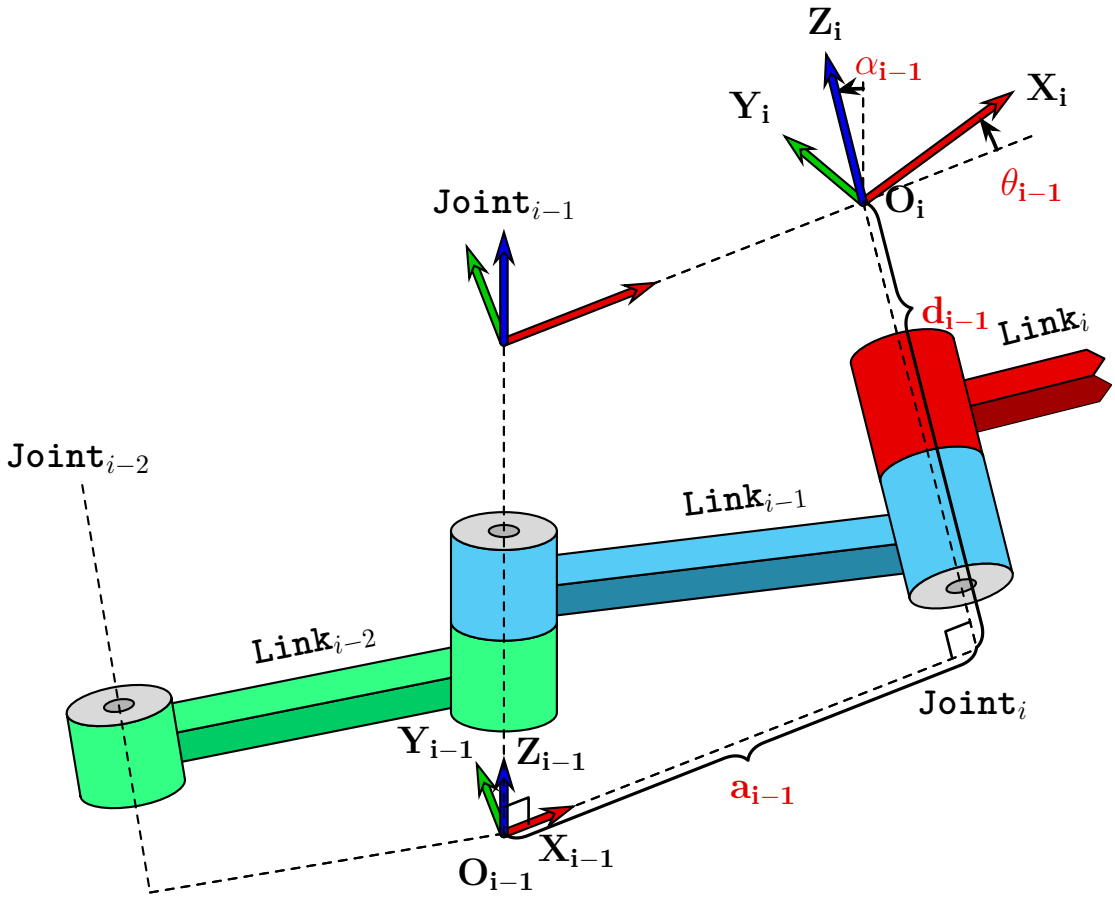


Figure 1.3: Modified Denavit–Hartenberg (MDH) parameters for consecutive link frames.

Modified Denavit–Hartenberg Parameters

Unlike the standard DH convention which attaches frame i to the end-effector side—where link i connects to link $i+1$—the modified convention attaches frame i to the base side—where link i connects to link $i-1$—as shown in Fig. 1.3. The mapping of frame i to frame $i-1$ is characterized by four geometric parameters [4]:

- α_{i-1}: **Link twist**, the angle from $\hat{\mathbf{Z}}_{i-1}$ to $\hat{\mathbf{Z}}_i$ about the $\hat{\mathbf{X}}_{i-1}$ axis.
- a_{i-1}: **Link length**, the distance from $\hat{\mathbf{Z}}_{i-1}$ to $\hat{\mathbf{Z}}_i$ along the $\hat{\mathbf{X}}_{i-1}$ axis.
- θ_{i-1}: **Joint angle**, the angle from $\hat{\mathbf{X}}_{i-1}$ to $\hat{\mathbf{X}}_i$ about the $\hat{\mathbf{Z}}_i$ axis.
- d_{i-1}: **Link offset**, the distance from $\hat{\mathbf{X}}_{i-1}$ to $\hat{\mathbf{X}}_i$ along the $\hat{\mathbf{Z}}_i$ axis.

For a revolute joint, θ_{i-1} is the variable DOF, while a_{i-1}, α_{i-1}, and d_{i-1} are constants unique to the link geometry. The homogeneous transformation matrix $\mathbf{T}_{i,i-1} \in \mathrm{SE}(3)$ connecting to frame i from frame $i-1$ is constructed from the transformation sequence: α_{i-1}, a_{i-1}, θ_{i-1}, and d_{i-1}. Multiplying these sequential transformations yields the homogeneous transformation matrix:

$$\mathbf{T}_{i,i-1}(\theta_{i-1}) = \begin{bmatrix} cos\theta_{i-1} & -sin\theta_{i-1} & 0 & a_{i-1} \\ sin\theta_{i-1}cos\alpha_{i-1} & cos\theta_{i-1}cos\alpha_{i-1} & -sin\alpha_{i-1} & -sin\alpha_{i-1}d_{i-1} \\ sin\theta_{i-1}sin\alpha_{i-1} & cos\theta_{i-1}sin\alpha_{i-1} & cos\alpha_{i-1} & cos\alpha_{i-1}d_{i-1} \\ 0 & 0 & 0 & 1 \end{bmatrix} \tag{1.26}$$

By defining these four parameters for each of the seven joints of the manipulator, the forward kinematics may be computed. As described in § 1.1.1, homogeneous transformation matrices can be multiplied to represent successive coordinate changes. In this manner, the pose of the end-effector (frame 7) relative to the manipulator base (frame 0) is obtained by chaining successive transformations:

$$\mathbf{T}_{0,7} = \prod_{i=1}^{7} \mathbf{T}_{i-1,i} = \mathbf{T}_{0,1}\mathbf{T}_{1,2}\cdots\mathbf{T}_{6,7} \tag{1.27}$$

This transformation matrix $\mathbf{T}_{0,7} \in \mathrm{SE}(3)$ yields the complete 6-D pose of the end-effector. Within the scope of this work, generating these transformation matrices forms the foundation of the manipulator's kinematic model.

1.2 B-Splines

To transcribe the continuous-time optimal control problem (OCP) into a discretized time, finite-dimensional nonlinear program (NLP) solvable by algorithms such as Interior Point OPTimizer (IPOPT), the state and control trajectories are parameterized using B-splines. B-splines (Basis splines) are piecewise polynomial functions defined over a sequence of values known as a knot vector—with each element greater than or equal to

the preceding one. This parameterization defines the trajectory's shape as a discrete set of control vertices, reducing the infinite-dimensional optimization to a finite set of decision variables while preserving continuous kinematics [3]. A B-spline curve $\mathbf{x}(s)$ of degree p is constructed from n control vertices $\mathbf{c}_i \in \mathbb{R}^d (i = 0, \ldots, n-1)$ and corresponding scalar basis functions $N_{i,p}(s)$:

$$\mathbf{x}(s) = \sum_{i=0}^{n-1} \mathbf{c}_i N_{i,p}(s), \tag{1.28}$$

where $s \in [0, 1]$ is the normalized time parameter along the trajectory. The basis functions $N_{i,p}(s)$ are computed recursively over a knot vector $\boldsymbol{\kappa} = \{t_0, t_1, \ldots, t_{n+p+1}\}$ via the Cox–de Boor algorithm [3]:

$$N_{i,0}(s) = \begin{cases} 1 & \text{if } t_i \leq s < t_{i+1} \\ 0 & \text{otherwise} \end{cases}, \tag{1.29}$$

$$N_{i,p}(s) = \frac{s - t_i}{t_{i+p} - t_i} N_{i,p-1}(s) + \frac{t_{i+p+1} - s}{t_{i+p+1} - t_{i+1}} N_{i+1,p-1}(s). \tag{1.30}$$

In this formulation, $0/0 \equiv 0$ explicitly to handle identical sequential knots.

Knot Vector and Clamping

This work employs *clamped, uniform* knot vectors. A clamped knot vector duplicates the first and last knots $p + 1$ times which forces the curve to exactly start and end at the first and last control vertices: $\mathbf{x}(0) = \mathbf{c}_0$ and $\mathbf{x}(1) = \mathbf{c}_{n-1}$. The clamped structure ensures the trajectory deterministically obeys the boundary conditions of the NLP—such as the initial and final orientation of a spacecraft slew, or the terminal intercept position of a robotic manipulator—through the corresponding boundary control vertices. The uniform property means the internal knots are uniformly distributed—with equal knot spans between consecutive knots—simplifying the evaluation time grid.

Properties Critical for Trajectory Optimization

The B-spline parameterization provides three mathematical properties essential for formulating kinodynamic optimization problems:

- **Convex Hull Property:** A B-spline curve is strictly contained within the convex hull of the control vertices defining any given segment—i.e. the values of the trajectory are bounded by the values of the influencing control vertices. Consequently, if the control vertices are bounded by a maximum magnitude, the resulting continuous trajectory is strictly bound by the same magnitude—no overshoot occurs, unlike many other trajectory via spline interpolation methods. This enables the NLP solver to enforce continuous-time state bounds (e.g., joint limits) by applying simple bound constraints directly to the discrete control vertices. The convex hull property also applies to the derivative B-spline trajectories.

- **Local Support:** The basis function $N_{i,p}(s)$ is non-zero only within the interval $[t_i, t_{i+p+1})$. Modifying a single control vertex $\mathbf{c}_i$ alters the trajectory only within

this local span. In the context of the NLP, local support yields sparse and banded Jacobian and Hessian matrices, which interior-point solvers exploit for efficient, large-scale matrix calculation. Furthermore, this property prevents individual control vertex updates from inducing global trajectory oscillations, which is a common drawback of global polynomial parameterizations.

- **Analytic Differentiation:** The time derivative of a degree p B-spline is analytically defined as another B-spline of degree $p-1$. The derivative trajectory is parameterized by a new set of control vertices $\mathbf{v}_i$, derived from the finite differences of the position vertices:

$$\dot{\mathbf{x}}(s) = \sum_{i=0}^{n-2} \mathbf{v}_i N_{i+1,p-1}(s), \tag{1.31}$$

where the velocity control vertices are defined as:

$$\mathbf{v}_i = \frac{\mathrm{d}}{\mathrm{d}s} N_{i+1,p-1}(s) = p \frac{\mathbf{c}_{i+1} - \mathbf{c}_i}{t_{i+p+1} - t_{i+1}}. \tag{1.32}$$

All higher-order derivatives (acceleration, jerk, etc.) follow the same pattern, eliminating the need for finite-difference approximations or numerical integration when evaluating the forward and inverse dynamics of the spacecraft and robotic manipulator models.

B-Spline Trajectory Time Scaling

The normalized spline parameter $s \in [0,1]$ maps to physical time $t \in [t_0, t_f]$ via $s(t) = \frac{t-t_0}{T}$, where $T = t_f - t_0$ represents the total trajectory duration. The physical trajectory $\mathbf{x}(t)$ is evaluated from the normalized parameterization as:

$$\mathbf{x}(t) = \mathbf{x}(s(t)) = \sum_{i=0}^{n-1} \mathbf{c}_i N_{i,p}\left(\frac{t-t_0}{T}\right). \tag{1.33}$$

Applying the chain rule yields the physical time derivatives required for evaluating system dynamics and kinematic limits:

$$\frac{\mathrm{d}^k \mathbf{x}(t)}{\mathrm{d}t^k} = \frac{1}{T^k} \frac{\mathrm{d}^k \mathbf{x}(s)}{\mathrm{d}s^k}, \quad \text{for } k \geq 0. \tag{1.34}$$

Consequently, the physical velocity $\dot{\mathbf{x}}(t)$ and acceleration $\ddot{\mathbf{x}}(t)$ are given by scaling the parameter derivatives by $\frac{1}{T^k}$. If T is optimized as a decision variable in a free-final-time OCP, this scaling directly influences the path constraints and required control torques.

1.3 Physical Models, Kinematics, Dynamics, & Constraints

This thesis establishes two physical models:

- Spacecraft attitude dynamic model for pure rotational motion.
- 7-DOF robotic manipulator dynamic model.

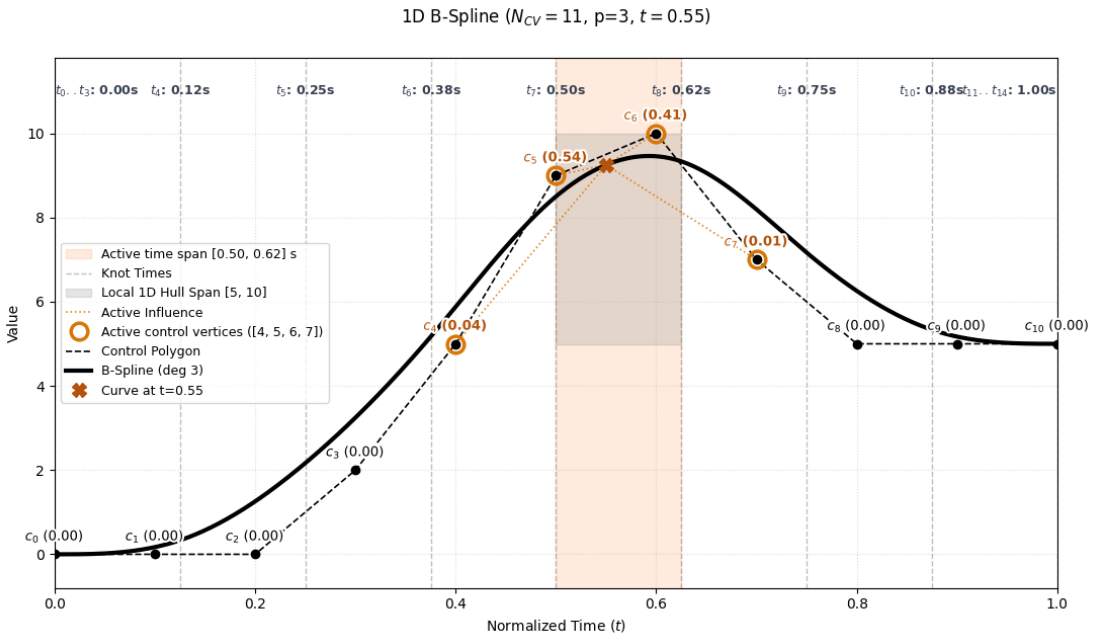


Figure 1.4: 1-D cubic B-spline trajectory demonstrating the local support property and control vertex influence at an example point $t = 0.55\,\text{s}$. The active knot span $[t_7, t_8] = [0.50, 0.62]\,\text{s}$ dictates that the curve value is determined by control vertices $\mathbf{c}_4$ through $\mathbf{c}_7$, with basis weights $N_{i,3}$ corresponding to active basis functions. Basis weights vary depending on the value of the basis functions for the evaluated point t, respective only to its location within the active knot span. Clamping the knot vector ($\{t_0, \ldots, t_3\}$ and $\{t_{11}, \ldots, t_{14}\}$) anchors the curve endpoints to $\mathbf{c}_0$ and $\mathbf{c}_{10}$, while repeating the initial and final three control vertex values enforces zero derivatives up to second order ($\dot{\mathbf{q}} = \mathbf{0}, \ddot{\mathbf{q}} = \mathbf{0}$) at the boundaries. The grey shaded region indicates local convex hull bounds $[5, 10]$.

The function of these models is to serve as the foundational physical plants for the trajectory optimization algorithms evaluated in this thesis. Both physical models are formulated under ideal rigid-body assumptions. Consequently, higher-order dynamic effects—including structural flexibility, fluid slosh, actuator and joint non-idealities, and environmental disturbances such as gravity gradient torques, solar radiation pressure, and drag forces are omitted from the system equations.

Our implementation uses a direct B-spline inverse-dynamic trajectory parameterization for both models, whereby the continuous-time state trajectories are evaluated at discrete sampling times for enforcing control and kinodynamic constraints, eliminating the need for numerical integration of the equations of motion (EOM).

This section introduces both kindodynamic plant models and their associated constraints.

1.3.1 Spacecraft Model

The spacecraft model describes pure rotational motion—existing on SO(3)—ignoring any translational motion—which would be encompassed by special Euclidean group in three dimensions (SE(3)). The optimization parameters are purely the B-spline control vertices for the quaternion trajectory, $\mathbf{q}$,with all other kinematic and dynamic quantities derived from the quaternions and their derivatives.

Quaternion Trajectories and Their Derivatives

If the quaternion control vertex matrix is $\mathbf{P} \in \mathbb{R}^{4\times n_{ctrl}}$ and the B-spline basis matrix is $\boldsymbol{\varphi} \in \mathbb{R}^{n_{ctrl}\times 1}$:

$$\mathbf{q} = \mathbf{P}\boldsymbol{\varphi} \in \mathbb{R}^{4\times 1}, \tag{1.35}$$

or for the entire trajectory over all sampling times, we can write:

$$\mathbf{Q} = \mathbf{P}\boldsymbol{\Phi} \in \mathbb{R}^{4\times n_{via}}, \tag{1.36}$$

where $\mathbf{Q}$ is the matrix of quaternion values at all sampling times and $\boldsymbol{\Phi} \in \mathbb{R}^{n_{ctrl}\times n_{via}}$ is the B-spline basis matrix evaluated at all sampling times. n_{ctrl} is the number of control vertices—equivalently the number of elements in the optimization vector—parameterizing the B-spline and n_{via} represents the number of via points or sampling times for the trajectory.

$$\dot{\mathbf{q}} = \mathbf{P}\dot{\boldsymbol{\varphi}} \in \mathbb{R}^{4\times 1} \tag{1.37}$$

$$\dot{\mathbf{Q}} = \mathbf{P}\dot{\boldsymbol{\Phi}} \in \mathbb{R}^{4\times n_{\text{via}}} \tag{1.38}$$

$$\ddot{\mathbf{q}} = \mathbf{P}\ddot{\boldsymbol{\varphi}} \in \mathbb{R}^{4\times 1} \tag{1.39}$$

$$\ddot{\mathbf{Q}} = \mathbf{P}\ddot{\boldsymbol{\Phi}} \in \mathbb{R}^{4\times n_{\text{via}}} \tag{1.40}$$

$$\dddot{\mathbf{q}} = \mathbf{P}\dddot{\boldsymbol{\varphi}} \in \mathbb{R}^{4\times 1} \tag{1.41}$$

$$\dddot{\mathbf{Q}} = \mathbf{P}\dddot{\boldsymbol{\Phi}} \in \mathbb{R}^{4\times n_{\text{via}}} \tag{1.42}$$

Equations of Motion (EOM)

The state vector of the spacecraft is defined as $\mathbf{x} = [\mathbf{q}^\top, \boldsymbol{\omega}^\top]^\top$, where $\mathbf{q} \in S^3 \subset \mathbb{H}$ represents the attitude quaternion and $\boldsymbol{\omega} \in \mathbb{R}^3$ is the angular velocity vector. The rotational dynamics of the spacecraft are governed by Euler's equation:

$$\boldsymbol{\tau} = \mathbf{J}_{\text{sc}}\dot{\boldsymbol{\omega}} + \boldsymbol{\omega} \times (\mathbf{J}_{\text{sc}}\boldsymbol{\omega}), \tag{1.43}$$

where $\mathbf{J}_{\text{sc}}$ is the spacecraft's inertia matrix, $\boldsymbol{\omega}$ is the angular velocity vector, and τ represents the control torque *input* applied by the actuators.

The relation between the quaternion rate and the angular velocity is:

$$\dot{\mathbf{q}} = \frac{1}{2}\mathbf{q} \otimes \boldsymbol{\omega} = \frac{1}{2}\mathbf{C}_q(\mathbf{q})^\top \boldsymbol{\omega}, \tag{1.44}$$

$$2\mathbf{C}_q(\mathbf{q})\dot{\mathbf{q}} = \boldsymbol{\omega}, \tag{1.45}$$

$$\mathbf{C}_q(\mathbf{q}) = \begin{bmatrix} q_w\mathbf{I}_{3\times 3} + [\mathbf{q_v}]_\times & -\mathbf{q_v} \end{bmatrix} = \begin{bmatrix} q_w & q_z & -q_y & -q_x \\ -q_z & q_w & q_x & -q_y \\ q_y & -q_x & q_w & -q_z \end{bmatrix} \tag{1.46}$$

$$\mathbf{C}_q(\mathbf{q})\mathbf{C}_q(\mathbf{q})^\top = \mathbf{I}_{3\times 3}, \tag{1.47}$$

where $\mathbf{C}_q(\mathbf{q})$ is the quaternion differential matrix.

Applying the chain rule, the time derivative of the angular velocity $\dot{\boldsymbol{\omega}}$—i.e. angular acceleration—and the second time derivative $\ddot{\boldsymbol{\omega}}$—i.e. jerk—can be expressed in terms of the quaternion and its derivatives:

$$\dot{\boldsymbol{\omega}} = 2\left[\mathbf{C}_q(\dot{\mathbf{q}})\dot{\mathbf{q}} + \mathbf{C}_q(\mathbf{q})\ddot{\mathbf{q}}\right]. \tag{1.48}$$

$$\ddot{\boldsymbol{\omega}} = 2\left[\mathbf{C}_q(\ddot{\mathbf{q}})\dot{\mathbf{q}} + 2\mathbf{C}_q(\dot{\mathbf{q}})\ddot{\mathbf{q}} + \mathbf{C}_q(\mathbf{q})\,\dddot{\mathbf{q}}\right]. \tag{1.49}$$

Alternatively, if the angular velocity, angular acceleration, or jerk are known—as is the case for enforcing boundary constraints on the OCP—one can solve for the derivatives of the quaternion as:

$$\dot{\mathbf{q}} = \frac{1}{2}\mathbf{C}_q(\mathbf{q})^\top \boldsymbol{\omega}, \tag{1.50}$$

$$\ddot{\mathbf{q}} = \frac{1}{2}\left[\mathbf{C}_q(\mathbf{q})^\top \dot{\boldsymbol{\omega}} + \mathbf{C}_q(\dot{\mathbf{q}})^\top \boldsymbol{\omega}\right], \tag{1.51}$$

$$\dddot{\mathbf{q}} = \frac{1}{2}\left[\mathbf{C}_q(\mathbf{q})^\top \ddot{\boldsymbol{\omega}} + 2\mathbf{C}_q(\dot{\mathbf{q}})^\top \dot{\boldsymbol{\omega}} + \mathbf{C}_q(\ddot{\mathbf{q}})^\top \boldsymbol{\omega}\right]. \tag{1.52}$$

For the interior of the OCP, where the optimizer has freedom to optimize interior control vertices, $\mathbf{c}_{int}$, and therefore the trajectory, the order of operations follows: $\boxed{\mathbf{c}_{int} \longrightarrow \mathbf{q}, \dot{\mathbf{q}}, \ddot{\mathbf{q}} \longrightarrow \boldsymbol{\omega}, \dot{\boldsymbol{\omega}} \longrightarrow \boldsymbol{\tau}}$.

While at the initial and final boundaries, where the quaternion and its derivatives are constrained by equality constraints, the exterior control vertices $\mathbf{c}_{ext}$ are determined by: $\boxed{\mathbf{q}, \boldsymbol{\omega}, \dot{\boldsymbol{\omega}}, \ddot{\boldsymbol{\omega}} \longrightarrow \dot{\mathbf{q}}, \ddot{\mathbf{q}}, \dddot{\mathbf{q}} \longrightarrow \mathbf{c}_{ext}}$.

Exterior Control Vertex Calculation

As stated in the previous section, the exterior control vertices are determined by the boundary conditions on the quaternion trajectory and its derivatives. This section details the procedure for analytically and deterministically calculating the outermost control vertices of the B-spline trajectory. This reduces both the number of optimization variables and the computational complexity of the OCP as boundary condition kinematics are directly solved rather than passed to the optimizer to be enforced via equality constraints.

The quaternion B-spline control-vertex matrix is partitioned into the boundary and interior control vertices:

$$\mathbf{P} = \begin{bmatrix} \mathbf{P}_0 & \mathbf{P}_{\text{int}} & \mathbf{P}_f \end{bmatrix} \in \mathbb{R}^{4\times n_{ctrl}}, \tag{1.53}$$

where $\mathbf{P}_0$ contains the first $n_{\text{BC,0}}$ control vertices, $\mathbf{P}_f$ contains the last $n_{\text{BC,f}}$ control vertices, and $\mathbf{P}_{\text{int}}$ contains the interior free control vertices.

Consequently,

$$n_{ctrl} = n_{\text{BC,0}} + n_{\text{FCV}} + n_{\text{BC,f}}, \tag{1.54}$$

where n_{FCV} is the number of free control vertices. After the exterior vertices are determined from the boundary conditions, only $\mathbf{P}_{\text{int}}$ is treated as an optimization variable.

Let $\boldsymbol{\varphi}^{(r)}(t)$ denote the rth derivative of the B-spline basis vector at time t. The boundary basis matrices are formed from the active control-vertex entries at the initial and final times:

$$\mathbf{B}_0 = \begin{bmatrix} \boldsymbol{\varphi}_0^{(0)} & \boldsymbol{\varphi}_0^{(1)} & \boldsymbol{\varphi}_0^{(2)} \end{bmatrix} \in \mathbb{R}^{n_{\text{BC,0}}\times n_{\text{BC,0}}}, \tag{1.55}$$

$$\mathbf{B}_f = \begin{bmatrix} \boldsymbol{\varphi}_f^{(0)} & \boldsymbol{\varphi}_f^{(1)} & \boldsymbol{\varphi}_f^{(2)} \end{bmatrix} \in \mathbb{R}^{n_{\text{BC,f}}\times n_{\text{BC,f}}}, \tag{1.56}$$

where

$$\boldsymbol{\varphi}_0^{(r)} = \boldsymbol{\varphi}^{(r)}(t_0) \tag{1.57}$$

contains the first $n_{\text{BC,0}}$ entries, and

$$\boldsymbol{\varphi}_f^{(r)} = \boldsymbol{\varphi}^{(r)}(t_f) \tag{1.58}$$

contains the last $n_{\text{BC,f}}$ entries.

For the default case, $n_{\text{BC,0}} = n_{\text{BC,f}} = 3$, the prescribed boundary conditions are quaternion position, angular velocity, and angular acceleration, with the latter two converted to quaternion first and second derivatives for enforcing the analytical boundary conditions:

$$\mathbf{Y}_0 = \begin{bmatrix} \mathbf{q}_0 & \dot{\mathbf{q}}_0 & \ddot{\mathbf{q}}_0 \end{bmatrix} \in \mathbb{R}^{4\times 3}, \tag{1.59}$$

$$\mathbf{Y}_f = \begin{bmatrix} \mathbf{q}_f & \dot{\mathbf{q}}_f & \ddot{\mathbf{q}}_f \end{bmatrix} \in \mathbb{R}^{4\times 3}. \tag{1.60}$$

Using

$$\mathbf{q}^{(r)}(t) = \mathbf{P}\boldsymbol{\varphi}^{(r)}(t), \tag{1.61}$$

the boundary conditions can be written as

$$\mathbf{P}_0\mathbf{B}_0 = \mathbf{Y}_0, \qquad \mathbf{P}_f\mathbf{B}_f = \mathbf{Y}_f. \tag{1.62}$$

The exterior control vertices are therefore obtained analytically as

$$\mathbf{P}_0 = \mathbf{Y}_0\mathbf{B}_0^{-1}, \qquad \mathbf{P}_f = \mathbf{Y}_f\mathbf{B}_f^{-1}. \tag{1.63}$$

Invertibility note. For a clamped B-spline, the zeroth-order basis function at $t = 0$ depends only on the first control vertex. The first derivative introduces the second control vertex, the second derivative introduces the third control vertex, and so on. Therefore, for $n_{\mathrm{BC},0} = 3$,

$$\mathbf{B}_0 = \begin{bmatrix} \boldsymbol{\varphi}_0^{(0)} & \boldsymbol{\varphi}_0^{(1)} & \boldsymbol{\varphi}_0^{(2)} \end{bmatrix} \tag{1.64}$$

has an upper-triangular structure

$$\mathbf{B}_0 \sim \begin{bmatrix} \star & \star & \star \\ 0 & \star & \star \\ 0 & 0 & \star \end{bmatrix}. \tag{1.65}$$

The final endpoint matrix has an analogous lower-triangular structure:

$$\mathbf{B}_f \sim \begin{bmatrix} \star & 0 & 0 \\ \star & \star & 0 \\ \star & \star & \star \end{bmatrix}. \tag{1.66}$$

The diagonal entries are nonzero if

$$\text{degree} \geq n_{\mathrm{BC},0} - 1, \tag{1.67}$$

Hence the triangular structure ensures that the matrices are nonsingular,

$$\det(\mathbf{B}_0) \neq 0, \qquad \det(\mathbf{B}_f) \neq 0, \tag{1.68}$$

guaranteeing that $\mathbf{B}_0$ and $\mathbf{B}_f$ are invertible and (1.63) exists and is unique.

The quaternion derivatives appearing in $\mathbf{Y}_0$ and $\mathbf{Y}_f$ are obtained from the angular quantities using (1.50)–(1.52).

If $n_{\mathrm{BC},0} = n_{\mathrm{BC,f}} = 4$, jerk is also imposed at both boundaries. In that case,

$$\mathbf{Y}_0 = \begin{bmatrix} \mathbf{q}_0 & \dot{\mathbf{q}}_0 & \ddot{\mathbf{q}}_0 & \dddot{\mathbf{q}}_0 \end{bmatrix}, \tag{1.69}$$

$$\mathbf{Y}_f = \begin{bmatrix} \mathbf{q}_f & \dot{\mathbf{q}}_f & \ddot{\mathbf{q}}_f & \dddot{\mathbf{q}}_f \end{bmatrix}, \tag{1.70}$$

and the boundary matrices become

$$\mathbf{B}_0 = \begin{bmatrix} \boldsymbol{\varphi}_0^{(0)} & \boldsymbol{\varphi}_0^{(1)} & \boldsymbol{\varphi}_0^{(2)} & \boldsymbol{\varphi}_0^{(3)} \end{bmatrix}, \tag{1.71}$$

$$\mathbf{B}_f = \begin{bmatrix} \boldsymbol{\varphi}_f^{(0)} & \boldsymbol{\varphi}_f^{(1)} & \boldsymbol{\varphi}_f^{(2)} & \boldsymbol{\varphi}_f^{(3)} \end{bmatrix}. \tag{1.72}$$

Therefore, the exterior control vertices are fixed deterministically from the boundary conditions, while the optimizer searches only over $\mathbf{P}_{\mathrm{int}}$.

State and Control Vertex Bounds

A helpful property of parameterizing SO(3) via unit quaternions $\mathbf{q} \in S^3 \subset \mathbb{R}^4$ is that S^3 is a compact manifold, meaning its search space in the Euclidean $\mathbb{R}^4$ is both bounded and closed.

Definition 1.1 (Bounded)**.** A set $\mathcal{S} \subset \mathbb{R}^n$ of a Euclidean space is said to be *bounded* if there exists a finite upper bound $M > 0$ on the magnitude of all its vectors: $|\mathbf{x}|_2 \leq M, \forall \mathbf{x} \in \mathcal{S}$.
Note: for unit quaternions, the maximum value of an element is one, making it bounded with $M = 1$. ▲

Definition 1.2 (Closed)**.** A set $\mathcal{S}$ of a metric space $(\mathcal{M}, d)$ is said to be *closed* if it contains all its limit points. That is, if a sequence $\{x_n\} \subset \mathcal{S}$ converges to a point $x \in \mathcal{M}$ (e.g., $d(x_n, x) \to 0$ as $n \to \infty$), then x must also belong to $\mathcal{S}$ ($x \in \mathcal{S}$). ▲

Definition 1.3 (Compact)**.** In Euclidean space $\mathbb{R}^n$, a subset $\mathcal{S}$ is *compact* if and only if it is both *bounded* and *closed* (Heine-Borel theorem). ▲

On S^3, all quaternion elements q_i belong to the compact set $[-1, 1]/\|\mathbf{q}\|_2 = 1$, preventing the state space from extending to infinity. While all points on the B-spline trajectory obey the convex hull of the control vertex polygon, the individual control vertices themselves may lie outside the unit hypercube $[-1, 1]^4$. To rigorously bound the optimizer's search space, we must derive the control vertex maximum bounds.

Control Vertex Bounds for S^3. Let the quaternion trajectory $\mathbf{q}(t)$ be parameterized by a clamped, uniform cubic B-spline ($p = 3$). The greatest weight a basis function can have at any point along the interior—i.e. away from the clamped ends—of the B-spline curve occurs at the knots. At any interior knot t_i, the active, non-zero basis functions evaluate to $\frac{1}{6}$, $\frac{2}{3}$, and $\frac{1}{6}$. The curve's value for a single quaternion element $k \in \{x, y, z, w\}$ at knot t_i is exactly:

$$q_k(t_i) = \frac{1}{6}c_{i-3,k} + \frac{2}{3}c_{i-2,k} + \frac{1}{6}c_{i-1,k}. \tag{1.73}$$

The trajectory is bounded by the unit quaternion definition such that $q_k(t) \in [-1, 1]$. To find the maximum possible theoretical value for the central control vertex $B_{\max} = c_{i-2,k}$ while satisfying $q_k(t_i) \leq 1$, the adjacent vertices $c_{i-3,k}$ and $c_{i-1,k}$ must take their minimum possible values.

However, the adjacent vertices must not cause the trajectory to violate the lower bound (-1) at their respective knots t_{i-1} and t_{i+1}. Pushing this to the extreme yields an alternating control sequence $[\ldots, -B_{\max}, B_{\max}, -B_{\max}, B_{\max}, \ldots]$. Substituting this alternating sequence into the knot evaluation yields:

$$q_k(t_i) = \frac{1}{6}(-B_{\max}) + \frac{2}{3}(B_{\max}) + \frac{1}{6}(-B_{\max}) = \frac{2}{6}B_{\max} = \frac{1}{3}B_{\max}. \tag{1.74}$$

Constraining the peak elementwise trajectory value to its upper bound ($q_k(t_i) = 1$) allows us to solve for $B_{\max}$:

$$1 = \frac{1}{3}B_{\max} \implies B_{\max} = 3. \tag{1.75}$$

By symmetry, applying the same argument to $-q_k(t)$ gives the minimum possible central control-vertex value:

$$B_{\min} = -B_{\max} = -3. \tag{1.76}$$

For the clamped ends of the B-spline, the maximum weight a basis function can have is 1, which occurs at the very first and last knots. Therefore, the control vertices at the clamped ends are directly bounded by the unit quaternion constraints, i.e., $c_{0,k}, c_{n-1,k} \in [-1, 1]$. Near-exterior control vertices, $c_{1,k}, \ldots, c_{p-2,k}, \ldots, c_{n-p+1,k}, \ldots, c_{n-2,k}$, are influenced by both the clamped ends and the interior vertices, forcing their maximum possible values less than the extreme interior bound of 3.

Therefore, the absolute theoretical bounds for any control vertex parameterizing a unit quaternion trajectory via a clamped, uniform cubic B-spline are strictly $[-3, 3]$. ■

The only remaining concern involves the initialization of the problem, as we have not defined a concrete bound for the control vertices. Two clear strategies were considered:

1. Initialize the control vertices within the unit hypercube $[-1, 1]^4$, ensuring that they are close to the unit quaternion manifold—which results in a set of quaternions near but not on S^3—and allow the optimizer to refine them during the optimization process.

2. Generate an initial set of quaternions on S^3 and use them to calculate the initial control vertices, ensuring that the trajectory starts exactly on the unit quaternion manifold.

To the author's surprise, the first strategy resulted in a faster wall-clock computation time. The performance gain is likely due to the elimination of the preliminary matrix inversion required by the latter strategy during high-level Python orchestration, instead allowing the C++-based optimizer core to pull the trajectory onto the manifold smoothly during early iterations. The properties of S^3 alongside either initialization strategy allow us to easily condition the search space of the optimizer.

Sampling on SO(3)

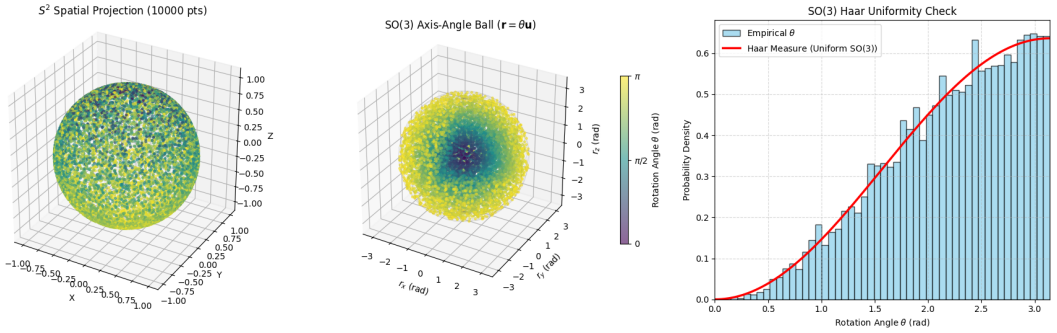
When searching the Special Orthogonal group $SO(3)$, sampling orientations requires careful consideration to ensure that the sample distribution corresponds to the Haar measure—the unique bi-invariant volume measure on the rotation group [7, 9]. A naïve approach to quaternion sampling consists of independently drawing four scalar components $w, x, y, z \sim \mathcal{U}[-1, 1] \in \mathbb{R}^4$ and normalizing the resulting vector to unit length. This approach fails to produce a uniform distribution on S^3; the geometry of the bounding 4D hypercube $[-1, 1]^4$ concentrates volume along its diagonals, causing normalized samples to heavily cluster toward the 4D hypercube corners—corresponding to 16 clusters when projected onto SO(3) [9].

To ensure an unbiased uniform distribution over $SO(3)$, we employ the explicit uniform quaternion generation algorithm derived by Shoemake [13] and evaluated by Kuffner [7].

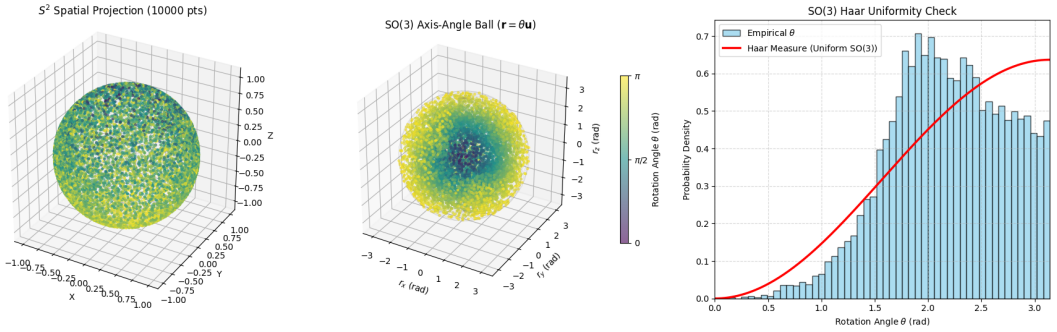
Given three independent uniform random variables $u_1, u_2, u_3 \sim \mathcal{U}[0,1]$, a unit quaternion $\mathbf{q} = [w, x, y, z]^\top \in S^3$ is constructed directly as:

$$\mathbf{q} = \begin{bmatrix} \sqrt{1-u_1}\sin(2\pi u_2) \\ \sqrt{1-u_1}\cos(2\pi u_2) \\ \sqrt{u_1}\sin(2\pi u_3) \\ \sqrt{u_1}\cos(2\pi u_3) \end{bmatrix} \tag{1.77}$$

The resulting spatial and angular distributions are shown in Figs. 1.5a and 1.5b. The naive sampling method produces a highly non-uniform ditributions with clustering correlated to the corners of the 4D hypercube, reduced density of small angle samples, and major deviations when compared to the Haar measure curve.



(a) Uniform quaternion sampling.



(b) Naïve quaternion sampling.

Figure 1.5: Comparison of uniform and naïve quaternion sampling. The spatial projection, axis–angle distribution, and empirical rotation-angle distribution illustrate the effect of sampling method on the resulting distribution over SO(3).

This parametrization maps the 3D unit cube $[0,1]^3$ directly onto the 3-sphere S^3 with uniform surface density, satisfying $\|\mathbf{q}\|_2 = 1$ identically without requiring normalization [7, 9]. We account for the double-cover property of quaternions ($\mathbf{q} \sim -\mathbf{q}$) by mapping any sample located in the lower hemisphere ($w < 0$) to its antipodal equivalent $-\mathbf{q}$ without distorting the uniform density [9].

Kinodynamic Constraints

Constraints imposed on the OCP are divided into two categories: (initial or final) boundary conditions, and path constraints.

The collection of boundary condition equality constraints is straightforward:

- Orientation: $\mathbf{q}(t_0) = \mathbf{q}_0$, $\mathbf{q}(t_f) = \mathbf{q}_f$
- Angular velocity: $\boldsymbol{\omega}(t_0) = \mathbf{0}$, $\boldsymbol{\omega}(t_f) = \mathbf{0}$
- Angular acceleration: $\boldsymbol{\alpha}(t_0) = \mathbf{0}$, $\boldsymbol{\alpha}(t_f) = \mathbf{0}$
- Angular jerk: $\dot{\boldsymbol{\alpha}}(t_0) = \mathbf{0}$, $\dot{\boldsymbol{\alpha}}(t_f) = \mathbf{0}$ (for select cases)

To enforce the boundary conditions, the corresponding B-spline control vertices are solved deterministically.

Kinematic and dynamic path constraints consist of:

- Angular velocity limits: $\|\boldsymbol{\omega}(t)\|_\infty \leq \omega_{\max}$
- Torque limits: $\|\boldsymbol{\tau}(t)\|_\infty \leq \tau_{\max}$
- Orientation exclusion zone (EZ): $(R_{ib}{}^b\hat{\mathbf{s}})^\top {}^i\hat{\mathbf{e}} \leq \cos(\theta_{\text{exclusion}})$
- Quaternion unit norm: $0.998 \leq \|\mathbf{q}\|_2 \leq 1.002$,

where ${}^b\hat{\mathbf{s}}$ is the sensor direction vector, ${}^i\hat{\mathbf{e}}$ is the exclusion direction vector in the inertial coordinate frame, and $\theta_{\text{exclusion}}$ is the angular radius of the EZ. The quaternion unit norm equality constraint was relaxed to a narrow inequality constraint to improve the optimizer's ability to explore the search space where instead of a 3-D manifold, a thin 4-D shell is permitted.

1.3.2 7-DOF Robotic Manipulator Model

The robotic manipulator model describes the joint-space kinematics and dynamics of a redundant 7-DOF arm conisisting of revolute joints existing on a bounded joint space $\mathcal{Q} \subset \mathbb{R}^7$. Task space end-effector poses belong to SE(3). The optimization parameters are purely the B-spline control vertices for the joint-position trajectories, $\mathbf{q}(t)$, with all other kinematic (joint velocities, accelerations, jerks, end-effector poses, and Jacobians) and dynamic (joint torques) quantities derived directly from the joint trajectories and their derivatives.

Joint Trajectories and Their Derivatives

If the joint control vertex matrix is $\mathbf{P} \in \mathbb{R}^{7\times n_{\text{ctrl}}}$ and the B-spline basis vector evaluated over normalized time $s = \frac{t-t_0}{T} \in [0,1]$ is $\boldsymbol{\varphi}(s) \in \mathbb{R}^{n_{\text{ctrl}}\times 1}$, the joint position trajectory is given by:

$$\mathbf{q}(s) = \mathbf{P}\boldsymbol{\varphi}(s) \in \mathbb{R}^{7\times 1}, \tag{1.78}$$

or, evaluated across all discrete sampling times, in matrix form:

$$\mathbf{Q} = \mathbf{P}\boldsymbol{\Phi} \in \mathbb{R}^{7\times n_{\text{via}}}, \tag{1.79}$$

where $\mathbf{Q}$ is the matrix of joint positions at all via points, $\mathbf{\Phi} \in \mathbb{R}^{n_{\text{ctrl}} \times n_{\text{via}}}$ is the B-spline basis matrix evaluated across all via points, n_{ctrl} is the number of control vertices per joint, and n_{via} represents the number of evaluation points along the trajectory.

Because time-scaling to the total trajectory duration $T = t_{\text{catch}} - t_0$ introduces the differential factor $\frac{ds}{dt} = \frac{1}{T}$, higher-order time derivatives explicitly scale by powers of $\frac{1}{T}$. The joint velocity, acceleration, and jerk trajectories are evaluated directly via the derivative basis functions:

$$\dot{\mathbf{q}}(t) = \frac{1}{T}\mathbf{P}\dot{\boldsymbol{\varphi}}(s) \in \mathbb{R}^{7\times 1}, \tag{1.80}$$

$$\dot{\mathbf{Q}} = \frac{1}{T}\mathbf{P}\dot{\mathbf{\Phi}} \in \mathbb{R}^{7\times n_{\text{via}}}, \tag{1.81}$$

$$\ddot{\mathbf{q}}(t) = \frac{1}{T^2}\mathbf{P}\ddot{\boldsymbol{\varphi}}(s) \in \mathbb{R}^{7\times 1}, \tag{1.82}$$

$$\ddot{\mathbf{Q}} = \frac{1}{T^2}\mathbf{P}\ddot{\mathbf{\Phi}} \in \mathbb{R}^{7\times n_{\text{via}}}, \tag{1.83}$$

$$\dddot{\mathbf{q}}(t) = \frac{1}{T^3}\mathbf{P}\dddot{\boldsymbol{\varphi}}(s) \in \mathbb{R}^{7\times 1}, \tag{1.84}$$

$$\dddot{\mathbf{Q}} = \frac{1}{T^3}\mathbf{P}\dddot{\mathbf{\Phi}} \in \mathbb{R}^{7\times n_{\text{via}}}. \tag{1.85}$$

Equations of Motion (EOM)

The state vector of the robot is defined as $\mathbf{x} = [\mathbf{q}^\top, \dot{\mathbf{q}}^\top]^\top$, where $\mathbf{q} \in \mathbb{R}^7$ is the vector of joint positions and $\dot{\mathbf{q}} \in \mathbb{R}^7$ is the vector of joint velocities.

The joint-space dynamics of the manipulator are governed by the EOM:

$$\boldsymbol{\tau} = \mathbf{M}(\mathbf{q})\ddot{\mathbf{q}} + \mathbf{h}(\mathbf{q}, \dot{\mathbf{q}}) - \mathbf{g}(\mathbf{q}), \tag{1.86}$$

where $\mathbf{M}(\mathbf{q}) \in \mathbb{R}^{7\times 7}$ is the mass matrix, $\mathbf{h}(\mathbf{q}, \dot{\mathbf{q}}) \in \mathbb{R}^7$ is the centripetal-Coriolis vector, $\mathbf{g}(\mathbf{q}) \in \mathbb{R}^7$ is the gravity vector, and $\boldsymbol{\tau} \in \mathbb{R}^7$ represents the control torque *input* applied by the actuators.

Rearranging (1.86) for the joint acceleration yields:

$$\ddot{\mathbf{q}} = \mathbf{M}(\mathbf{q})^{-1}\left[\boldsymbol{\tau} - \mathbf{h}(\mathbf{q}, \dot{\mathbf{q}}) + \mathbf{g}(\mathbf{q})\right]. \tag{1.87}$$

The end-effector pose is obtained from the forward-kinematics:

$$\mathbf{T}_{0,8} = \left(\prod_{i=1}^{7} \mathbf{T}_{i-1,i}(q_i)\right)\mathbf{T}_{7,8} = \mathbf{T}_{0,7}(\mathbf{q})\mathbf{T}_{7,8} \tag{1.88}$$

$$\mathbf{T}_{i,ee}(\mathbf{q}) = \mathbf{T}_{i,0}\mathbf{T}_{0,7}(\mathbf{q})\mathbf{T}_{7,8}\mathbf{T}_{8,ee}, \tag{1.89}$$

where indices i, ee, and $n \in \{0, \ldots, 8\}$ refer to the inertial frame, the end-effector frame, and the individual joint frames as described by the MDH table, respectively. The final joint frame, denoted by $n = 8$, is fixed relative to joint frame $n = 7$ and does not depend on the joint configuration. The end-effector position and orientation are extracted as:

$$ {}^{i}\mathbf{p}_e(\mathbf{q}) = \mathbf{T}_{i,ee}(\mathbf{q})(1{:}3, 4), \qquad \mathbf{R}_{i,ee}(\mathbf{q}) = \mathbf{T}_{i,ee}(\mathbf{q})(1{:}3, 1{:}3). \tag{1.90} $$

The corresponding end-effector twist, ${}^{i}\mathbf{v}_e(\mathbf{q}, \dot{\mathbf{q}})$ is related to the joint velocity by the geometric Jacobian $\mathbf{J}_g(\mathbf{q})$:

$$ {}^{i}\mathbf{v}_e(\mathbf{q}, \dot{\mathbf{q}}) = {}^{i}\mathbf{J}_{g,e}(\mathbf{q})\dot{\mathbf{q}}, \tag{1.91} $$

and its acceleration, ${}^{i}\mathbf{a}_e(\mathbf{q}, \dot{\mathbf{q}}, \ddot{\mathbf{q}})$, is:

$$ {}^{i}\mathbf{a}_e(\mathbf{q}, \dot{\mathbf{q}}, \ddot{\mathbf{q}}) = {}^{i}\mathbf{J}_{g,e}(\mathbf{q})\ddot{\mathbf{q}} + {}^{i}\dot{\mathbf{J}}_{g,e}(\mathbf{q}, \dot{\mathbf{q}})\dot{\mathbf{q}}. \tag{1.92} $$

For the interior of the OCP, where the trajectory is parameterized directly in joint coordinates using B-splines, the optimizer has the freedom to optimize the interior control vertices, $\mathbf{c}_{int}$. At each interior via point, the order of operations follows:

$$ \boxed{\mathbf{c}_{int} \longrightarrow \mathbf{q}, \dot{\mathbf{q}}, \ddot{\mathbf{q}} \longrightarrow \mathbf{M}(\mathbf{q}), \mathbf{h}(\mathbf{q}, \dot{\mathbf{q}}), \mathbf{g}(\mathbf{q}) \longrightarrow \boldsymbol{\tau}}. $$

At the trajectory boundaries, the B-spline parameterization deterministically enforces the kinematic boundary conditions via assignment of the B-spline control vertices. The trajectory is defined by n control vertices ($\mathbf{c}_0$ through $\mathbf{c}_{n-1}$). The initial conditions impose a fixed joint position, zero velocity, and zero acceleration. The final conditions impose zero velocity and zero acceleration, but leave the final joint position unconstrained so that it may be optimized to intercept the target.

As a result, five dependent control vertices ($\mathbf{c}_0, \mathbf{c}_1, \mathbf{c}_2, \mathbf{c}_{n-3}, \mathbf{c}_{n-2}$) are deterministic. They are calculated as exact functions of the initial state constraints, the final velocity and acceleration constraints, and the optimized final position, which is directly represented by the final vertex ($\mathbf{c}_{n-1}$). This analytical reduction leaves the remaining $n-5$ control vertices as the free decision variables provided to the optimizer:

$$ \begin{aligned} &\text{Kinematics:} && \boxed{\mathbf{q}_0, \dot{\mathbf{q}}_0, \ddot{\mathbf{q}}_0, \dot{\mathbf{q}}_f, \ddot{\mathbf{q}}_f, \mathbf{c}_{n-1} \longrightarrow \mathbf{c}_0, \mathbf{c}_1, \mathbf{c}_2, \mathbf{c}_{n-3}, \mathbf{c}_{n-2} \longrightarrow \mathbf{q}, \dot{\mathbf{q}}, \ddot{\mathbf{q}}} \\ &\text{Dynamics:} && \boxed{\mathbf{q}, \dot{\mathbf{q}}, \ddot{\mathbf{q}} \longrightarrow \mathbf{M}(\mathbf{q}), \mathbf{h}(\mathbf{q}, \dot{\mathbf{q}}), \mathbf{g}(\mathbf{q}) \longrightarrow \boldsymbol{\tau}} \end{aligned} $$

Exterior Control Vertex Calculation

For the robotic manipulator model, each joint trajectory is represented by a cubic B-spline:

$$ \mathbf{q}(t) = \mathbf{P}\boldsymbol{\varphi}(t), \qquad \mathbf{P} = \begin{bmatrix} \mathbf{c}_0 & \mathbf{c}_1 & \cdots & \mathbf{c}_{n_{\text{ctrl}}-1} \end{bmatrix} \in \mathbb{R}^{7 \times n_{\text{ctrl}}}, \tag{1.93} $$

where $\mathbf{q}(t) \in \mathbb{R}^7$ is the joint-position vector and each $\mathbf{c}_i \in \mathbb{R}^7$ contains one control vertex for each joint.

For a clamped cubic B-spline, the number of knot spans is

$$ n_{\text{seg}} = n_{\text{ctrl}} - p, \qquad p = 3. \tag{1.94} $$

For a uniform knot vector, the knot-span duration is therefore

$$ h = \frac{t_f}{n_{\text{seg}}}, \tag{1.95} $$

where t_f is the duration of the joint trajectory.

The initial boundary conditions are

$$\mathbf{q}(0) = \mathbf{q}_0, \qquad \dot{\mathbf{q}}(0) = \dot{\mathbf{q}}_0, \qquad \ddot{\mathbf{q}}(0) = \ddot{\mathbf{q}}_0. \tag{1.96}$$

For a clamped cubic B-spline, these conditions determine the first three control vertices:

$$\mathbf{c}_0 = \mathbf{q}_0, \tag{1.97}$$

$$\mathbf{c}_1 = \mathbf{c}_0 + \frac{h}{3}\dot{\mathbf{q}}_0, \tag{1.98}$$

$$\mathbf{c}_2 = 3\mathbf{c}_1 - 2\mathbf{c}_0 + \frac{h^2}{3}\ddot{\mathbf{q}}_0. \tag{1.99}$$

At the terminal boundary, the final velocity and acceleration are:

$$\dot{\mathbf{q}}(t_f) = \dot{\mathbf{q}}_f, \qquad \ddot{\mathbf{q}}(t_f) = \ddot{\mathbf{q}}_f. \tag{1.100}$$

The final control vertex is the final joint position when the ball is caught. The remaining final exterior vertices are determined from this vertex and the terminal derivative conditions:

$$\mathbf{c}_{n_{\text{ctrl}}-2} = \mathbf{c}_{n_{\text{ctrl}}-1} - \frac{h}{3}\dot{\mathbf{q}}_f, \tag{1.101}$$

$$\mathbf{c}_{n_{\text{ctrl}}-3} = \mathbf{c}_{n_{\text{ctrl}}-1} - h\dot{\mathbf{q}}_f + \frac{h^2}{3}\ddot{\mathbf{q}}_f. \tag{1.102}$$

Thus, the control vertices matrix is

$$\mathbf{P} = \begin{bmatrix} \mathbf{c}_0 & \mathbf{c}_1 & \mathbf{c}_2 & \mathbf{c}_3 & \cdots & \mathbf{c}_{n_{\text{ctrl}}-4} & \mathbf{c}_{n_{\text{ctrl}}-3} & \mathbf{c}_{n_{\text{ctrl}}-2} & \mathbf{c}_{n_{\text{ctrl}}-1} \end{bmatrix}. \tag{1.103}$$

For the ball-catching problem, the interior control vertices

$$\mathbf{c}_3, \ldots, \mathbf{c}_{n_{\text{ctrl}}-4}$$

and the terminal-position vertex

$$\mathbf{c}_{n_{\text{ctrl}}-1}$$

are the optimization variables. The remaining exterior vertices are reconstructed analytically from the boundary conditions.

Because the terminal joint position is not set directly in the ball-catching problem, $\mathbf{c}_{n_{\text{ctrl}}-1}$ is determined indirectly by the terminal end-effector and approach-axis constraints.

For a problem with a fixed joint-space endpoint, the terminal joint position is prescribed:

$$\mathbf{c}_{n_{\text{ctrl}}-1} = \mathbf{q}_f. \tag{1.104}$$

In that case, all six exterior control vertices would be determined by the boundary conditions, as shown for the spacecraft model, where only the interior control vertices would remain as optimization variables.

State and Control Vertex Bounds

The state space for the robotic manipulator is simply bounded by the joint position limits $[q_{i,\min}, q_{i,\max}], i \in \{1, \ldots, 7\}$. However, unlike the spacecraft model, the robotic manipulator does not have an easily defined space for the control vertex bounds. This is because, while the B-spline convex hull property applies, there is no guaranteed lower bound on the ratio of trajectory position to control vertex value; the control vertex values may be substantially greater than the individual joint limits while still producing feasible trajectories. As the aim of this work is to perform *global* trajectory optimization, it is critical to derive a set of control vertex bounds that do not arbitrarily exclude feasible local minima. Therefore, it is necessary to derive the bounds using B-spline principles and model specific details.

Control Vertex Bounds. Let the joint position trajectory, $q(t)$, be a clamped, uniform B-spline of degree p and N control vertices. There are $N_T = N + p + 1$ knots and $N - p$ internal knot spans. To ensure the NLP's search space covers all dynamically valid trajectories up to the maximum solve time, $t_{f,\max}$, the bounds must be calculated using the maximum possible knot span:

$$\Delta t_{\max} = \frac{t_{f,\max}}{N - p}. \tag{1.105}$$

The joint velocity, $\dot{q}(t)$, is a quadratic B-spline defined by velocity control vertices, v_i. From the convex hull property of B-splines, $|\dot{q}(t)|$ is bounded by the maximum velocity vertex $|v_i|$. Therefore, the following relation must be satisfied:

$$|v_i| \leq \dot{q}_{\max}. \tag{1.106}$$

Velocity Control Vertex Formula: For a B-spline with degree p and knot vector $\boldsymbol{\kappa} = \{t_1, t_2, \ldots, t_{N+p+1}\}$, the velocity control vertices are given by:

$$v_i = \frac{p}{t_{i+p+1} - t_{i+1}}(c_{i+1} - c_i), \tag{1.107}$$

where c_i are the joint position control vertices of the B-spline.

Endpoint Vertices (c_0, c_{N-1}): For clamped B-splines, $c_0 = q_0$ and $c_{N-1} = q_f$. The bounds on endpoint vertices are simply the position bounds:

$$B_{\max,i} = q_{\max}, \quad B_{\min,i} = q_{\min}, \quad i \in \{0, N-1\}. \tag{1.108}$$

Interior Vertices ($c_1, \ldots, c_{N-2}$): Evaluating the velocity control vertex formula for degree p:

$$v_i = \frac{p}{t_{i+p+1} - t_{i+1}}(c_{i+1} - c_i) = \frac{p}{(p+1)\Delta t_{\max} - \Delta t_{\max}}(c_{i+1} - c_i) = \frac{1}{\Delta t_{\max}}(c_{i+1} - c_i). \tag{1.109}$$

Constraining via $|v_i| \leq \dot{q}_{\max}$ yields:

$$|c_{i+1} - c_i| \leq \dot{q}_{\max}\Delta t_{\max}, \tag{1.110}$$

which provides the control vertex bounds:

$$B_{\max,i} = q_{\max} + \dot{q}_{\max}\Delta t_{\max}, \qquad B_{\min,i} = q_{\min} - \dot{q}_{\max}\Delta t_{\max} \quad i \in \{1, \ldots, N-2\}. \tag{1.111}$$

While $i \in \{1, \ldots, p-2, N-p+1, \ldots, N-2\}$ can be given tighter bounds due to knot multiplicity at the clamped endpoints, the optimizer does not optimize the control vertices corresponding to these indices. As a simplified and conservative approach, the same looser bounds of the interior vertices are applied to these exterior control vertices as well.

Additionally, the search space could be further constrained by enforcing independent lower and upper bounds for each joint position, rather than the joint-specific symmetric bounds used here. ■

Sampling on $\mathbb{R}^7$

Unlike for the sampling of quaternions, where special care must be taken to ensure uniform coverage of the unit sphere, the sampling of joint positions can be performed independently for each joint within its respective bounds via

$$q_i \sim \mathcal{U}(B_{\min,i}, B_{\max,i}), \quad i \in \{1, \ldots, 7\}, \tag{1.112}$$

where $\mathcal{U}(B_{\min,i}, B_{\max,i})$ denotes the uniform distribution over the interval $[B_{\min,i}, B_{\max,i}]$ for joint i.

Kinodynamic Constraints

As in the spacecraft model, constraints imposed on the robot model consist of boundary and path constraint.

The collection of boundary condition constraints is:

- Initial joint position: $\mathbf{q}(t_0) = \mathbf{q}_{\text{initial}}$
- Joint velocity: $\dot{\mathbf{q}}(t_0) = \mathbf{0}$, $\dot{\mathbf{q}}(t_f) = \mathbf{0}$
- Joint acceleration: $\ddot{\mathbf{q}}(t_0) = \mathbf{0}$, $\ddot{\mathbf{q}}(t_f) = \mathbf{0}$
- Terminal end-effector position: ${}^{i}\mathbf{p}_{\text{ee}}(\mathbf{q}(t_f)) - {}^{i}\mathbf{p}_b(t_f) = \mathbf{0}$
- Terminal end-effector orientation: $\arcsin({}^{i}a_{\text{ee},z}) - \arcsin({}^{i}a_{b,z}) = 0$ and $\operatorname{atan2}({}^{i}a_{\text{ee},y}, {}^{i}a_{\text{ee},x}) - \operatorname{atan2}({}^{i}a_{b,y}, {}^{i}a_{b,x}) = 0$
- Catch time: $0.06 \leq t_f \leq 1.8$, where the spline duration is $T = t_f - t_0 > 0$,

where ${}^{i}\mathbf{p}_{\text{ee}}(\mathbf{q}(t_f))$ and ${}^{i}\mathbf{p}_b(t_f)$ are the positions of the end-effector and the ball at the catch time, respectively, ${}^{i}\mathbf{a}_{\text{ee}}(\mathbf{q}(t_f))$ is the unit vector of the end-effector approach axis—the axis pointing out of the palm of the robotic hand—at the terminal joint configuration, and ${}^{i}\mathbf{a}_b(t_f)$ is the unit vector of the ball approach axis—equivalent to the direction of the ball's velocity—at the catch time.

${}^{i}\mathbf{p}_{\text{ee}}(\mathbf{q})$ and $\mathbf{R}_{\text{i,ee}}(\mathbf{q})$ are components retrieved from the homogenous transformation matrix $\mathbf{T}_{\text{i,ee}}(\mathbf{q})$ obtained via forward kinematics:

$$\mathbf{T}_{\text{i,ee}}(\mathbf{q}) = \begin{bmatrix} \mathbf{R}_{\text{i,ee}}(\mathbf{q}) & {}^{i}\mathbf{p}_{\text{ee}}(\mathbf{q}) \\ \mathbf{0}^{\top} & 1 \end{bmatrix}. \tag{1.113}$$

$\mathbf{T}_{\text{i,ee}}\mathbf{q}$ is obtained via the chain of homogeneous transformation matrices from the base frame to the end-effector frame: $\mathbf{T}_{\text{i,ee}}(\mathbf{q}) = \mathbf{T}_{i,B}\mathbf{T}_{B,F}(\mathbf{q})\mathbf{T}_{\text{F,ee}}$, where i denotes the inertial frame, B denotes the robot base frame, F denotes the joint dependent MDH frame, and ee denotes the end-effector frame.

Double check the frame conventions here.

Unlike the directly specified terminal orientations in the spacecraft model, the robot's terminal joint position $\mathbf{q}(t_f)$ is not fixed. Instead, it is implicitly determined by the end-effector pose constraints required to intercept the target ball. The ball position at the catch time is defined analytically using the ballistic equation ${}^i\mathbf{p}_b(t_f) = {}^i\mathbf{p}_{b,0} + t_f{}^i\mathbf{v}_{b,0} + \frac{1}{2}t_f^2{}^i\mathbf{g}$, yielding three scalar equality constraints for position.

To ensure a successful catch, the end-effector approach axis ${}^i\mathbf{a}_{\text{ee}}(\mathbf{q}(t_f))$—extracted from the third column of the forward-kinematics rotation matrix $\mathbf{R}_{\text{i,ee}}(\mathbf{q}(t_f))$—must align (specifically, be anti-parallel) with the ball approach axis::

$$ {}^i\mathbf{a}_b(t_f) = -\frac{{}^i\mathbf{v}_{b,0} + t_f{}^i\mathbf{g}}{\|{}^i\mathbf{v}_{b,0} + t_f{}^i\mathbf{g}\|} \tag{1.114} $$

This alignment is enforced via the two orientation constraints defined in the list above, utilizing the x, y, and z components of the respective approach vectors. Together, the terminal end-effector pose contributes a total of five scalar equality constraints (3 position, 2 orientation) to the boundary conditions. For the 7-DOF robot, the five terminal end-effector pose constraints leave two additional degrees of freedom in the joint space. The redundancy results in infinite possible joint configurations satisfying the terminal end-effector pose constraints and allows for the global search of optimal trajectories. No additional control architecture for the remaining two-dimensional null-space is explicitly considered in this formulation, including avoidance of kinematic singularities resulting from certain joint configurations. Instead, the NLP formulation naturally avoids this risk because near-singular poses require near-infinite joint velocities and torques, which would yield high objective function costs and violate the path constraints. The optimizer inherently steers the solution away from singular and near-singular configurations.

Kinematic and dynamic path constraints consist of:

- Joint position limits: $\mathbf{q}_{\min} \leq \mathbf{q}(t) \leq \mathbf{q}_{\max}$
- Joint velocity limits: $\dot{\mathbf{q}}_{\min} \leq \dot{\mathbf{q}}(t) \leq \dot{\mathbf{q}}_{\max}$
- Torque limits: $\boldsymbol{\tau}_{\min} \leq \boldsymbol{\tau}(t) \leq \boldsymbol{\tau}_{\max}$.

There is a conspicuous absence of obstacle avoidance constraints in the current formulation, particularly concerning self-collisions. However, previous work by [8] showed self-collisions never occurred in practice; given this model is a replication of that work, it is reasonable to assume self-collisions will similarly not occur here.

1.4 Optimal Control Problem (OCP) & Nonlinear Programming (NLP) Formulation

The objective of the algorithms posed in this work is to generate optimal trajectories for the spacecraft and robotic manipulator models. In order to obtain an optimal result, the

objective must first be formalized into an OCP followed by a transcription to an NLP. The transcription involves discretizing the continuous-time problem into a finite set of state, control, and time nodes through the use of B-splines. A myriad of transcription methods exist including direct collocation, indirect and direct shooting, and pseudospectral methods. For a robust review of the commonly used transcription methods, refer to [2, 11]. The method employed in this work is direct collocation via B-spline parameterization which provides a very sparse structure for the resulting nonlinear program, facilitating efficient optimization. Following discretization, a solver must be employed, over which several types exist; here we use IPOPT as the underlying *interior-point* NLP solver.

1.5 Optimal Control Problem

To evaluate the B-spline trajectory optimization framework across different dynamical systems, two physical models are studied as benchmarks: a 3-DOF rigid-body spacecraft attitude slew and a 7-DOF robotic manipulator ball-catching maneuver.

1.5.1 Problem 1: Spacecraft Attitude Slew Maneuver

The first problem considers a two-point boundary value problem (BVP) for a spacecraft attitude slew maneuver over a fixed time horizon. The specifications on the state vector, constraints, and EOM are detailed in § 1.3.1.

Continuous-Time OCP Statement

The continuous-time spacecraft OCP minimizes control effort:

$$\min_{\mathbf{q}(t)} \quad J_{\text{spacecraft}} = \frac{1}{w_{\text{scale}}} \int_{t_0}^{t_f} \|\boldsymbol{\tau}(t)\|_2^2 \, \mathrm{d}t, \tag{1.115}$$

subject to:

$$\text{Inverse Kinodynamics:} \quad \boldsymbol{\omega}(t) = 2\mathbf{C}_q(\mathbf{q}(t))\dot{\mathbf{q}}(t), \tag{1.116}$$

$$\boldsymbol{\tau}(t) = \mathbf{I}\dot{\boldsymbol{\omega}}(t) + \boldsymbol{\omega}(t) \times (\mathbf{I}\boldsymbol{\omega}(t)), \tag{1.117}$$

$$\text{Quaternion Unit Norm:} \quad \|\mathbf{q}(t)\|_2 - 1 = 0, \tag{1.118}$$

$$\text{Actuator Torque Limits:} \quad \boldsymbol{\tau}_{\min} \leq \boldsymbol{\tau}(t) \leq \boldsymbol{\tau}_{\max}, \tag{1.119}$$

$$\text{Angular Velocity Limits:} \quad \boldsymbol{\omega}_{\min} \leq \boldsymbol{\omega}(t) \leq \boldsymbol{\omega}_{\max}, \tag{1.120}$$

$$\text{Initial Rest Boundary:} \quad \mathbf{q}(t_0) = \mathbf{q}_0, \quad \boldsymbol{\omega}(t_0) = \mathbf{0}, \quad \dot{\boldsymbol{\omega}}(t_0) = \mathbf{0}, \quad \ddot{\boldsymbol{\omega}}(t_0) = \mathbf{0}^{\dagger}, \tag{1.121}$$

$$\text{Final Rest Boundary:} \quad \mathbf{q}(t_f) = \mathbf{q}_f, \quad \boldsymbol{\omega}(t_f) = \mathbf{0}, \quad \dot{\boldsymbol{\omega}}(t_f) = \mathbf{0}, \quad \ddot{\boldsymbol{\omega}}(t_f) = \mathbf{0}^{\dagger}. \tag{1.122}$$

† enforced for select test cases.

1.5.2 Problem 2: 7-DOF Manipulator Ball-Catching Maneuver

The second problem considers an 7-DOF robotic manipulator catching a ball over a free time horizon $t \in [0.06, t_f]$s, with the terminal catch time t_f and end-effector pose are

free decision variables. The specifications on the state vector, constraints, and EOM are detailed in § 1.3.2.

Continuous-Time OCP Statement

The continuous-time optimal control problem for the 7-DOF manipulator ball catch minimizes joint control effort and acceleration over the free catch time $t_f \in [0.06, 1.8]\,\text{s}$:

$$\min_{\mathbf{q}(t), t_f} \quad J_{\text{robot}} = \int_{t_0}^{t_f} \left(\sum_{i=1}^{7} \left(\tau_i(t)\dot{q}_i(t)\right)^2 + \|\mathbf{J}_{\text{m}}\ddot{\mathbf{q}}(t)\|_2^2 \right) \mathrm{d}t, \tag{1.123}$$

where $\mathbf{J}_{\text{m}}$ is the diagonal matrix of joint motor inertias,
subject to:

$$\begin{aligned}
\text{Inverse Kinodynamics:} \quad & \mathbf{M}(\mathbf{q})\ddot{\mathbf{q}} + \mathbf{h}(\mathbf{q}, \dot{\mathbf{q}}) - \mathbf{g}(\mathbf{q}) = \boldsymbol{\tau}(t), && (1.124)\\
& \mathbf{p}_{ee}(\mathbf{q}) = \mathbf{T}_{i,ee}(\mathbf{q})(1{:}3, 4), && (1.125)\\
& \mathbf{R}_{i,ee}(\mathbf{q}) = \mathbf{T}_{i,ee}(\mathbf{q})(1{:}3, 1{:}3), && (1.126)\\
\text{Joint Position Bounds:} \quad & \mathbf{q}_{\min} \leq \mathbf{q}(t) \leq \mathbf{q}_{\max}, && (1.127)\\
\text{Joint Velocity Bounds:} \quad & \dot{\mathbf{q}}_{\min} \leq \dot{\mathbf{q}}(t) \leq \dot{\mathbf{q}}_{\max}, && (1.128)\\
\text{Actuator Torque Bounds:} \quad & \boldsymbol{\tau}_{\min} \leq \boldsymbol{\tau}(t) \leq \boldsymbol{\tau}_{\max}, && (1.129)\\
\text{Initial Rest Boundary:} \quad & \mathbf{q}(t_0) = \mathbf{q}_0, \quad \dot{\mathbf{q}}(t_0) = \mathbf{0}, \quad \ddot{\mathbf{q}}(t_0) = \mathbf{0}, && (1.130)\\
\text{Final Rest Boundary:} \quad & \dot{\mathbf{q}}(t_f) = \mathbf{0}, \quad \ddot{\mathbf{q}}(t_f) = \mathbf{0}, && (1.131)\\
\text{Final Catch Position:} \quad & \mathbf{p}_{\text{ee}}(\mathbf{q}(t_f)) = \mathbf{p}_{ball}(t_f), && (1.132)\\
\text{Final Approach Alignment:} \quad & \text{approach-axis}(\mathbf{q}(t_f)) - \text{ball-axis}(t_f) = 0, && (1.133)\\
\text{Catch Time Horizon Bounds:} \quad & 0.06 \leq t_f \leq 1.8\text{s}, && (1.134)
\end{aligned}$$

with (1.133) defined in § 1.3.2 and all points and vectors expressed in the inertial frame unless otherwise noted.

1.5.3 NLP Decision Variable Structure

When transcribed using B-splines, both continuous problems map the boundary state constraints that are known a priori directly onto the corresponding exterior control vertices ($\mathbf{P}_{\text{ext}}$)—which are evaluated
analytically. For the spacecraft, only the interior control vertices $\mathbf{P}_{\text{int}}$ remain as free decision variables. For the robotic manipulator, the interior control vertices, the final control vertex $\mathbf{P}_f$, and the final time t_f are free decision variables in the optimization vector $\mathbf{w}$:

- **Spacecraft Slew Decision Vector:** Parameterized by 4 quaternion elements (n_{ctrl} control vertices per element). With boundary conditions embedded, the number of free variables is reduced by the number of boundary constraints, $n_{\text{free}} = n_{\text{ctrl}} - n_{\text{bc}}$:

$$\mathbf{w}_{\text{spacecraft}} = \mathbf{P}_{\text{int,sc}} \in \mathbb{R}^{4n_{\text{free}}}. \tag{1.135}$$

  $n_{\text{bc}} = n_{\text{bc},0} + n_{\text{bc,f}}$ may equal either six or eight, depending on whether angular jerk is included in the boundary conditions.

- **7-DOF Robotic Arm Decision Vector:** Parameterized by seven B-spline joint curves (n_{ctrl} control vertices per curve) with $n_{\text{free}} = n_{\text{ctrl}} - n_{\text{bc}}$ free interior control vertices per joint:

$$\mathbf{w}_{\text{robot}} = \begin{bmatrix} \mathbf{P}_{\text{int}} & \mathbf{P}_f & t_f \end{bmatrix} \in \mathbb{R}^{7n_{\text{free}}+1}. \tag{1.136}$$

  For the purpose of defining the number of free variables, $n_{\text{bc}} = n_{\text{bc,0}} + n_{\text{bc,f}} = 5$; the boundary conditions on catch position, approach angle, and time do not reduce the number of decision variables because the final control vertex is not constrained to a single configuration.

To prevent inter-sample constraint violation between discretization nodes, the continuous B-spline trajectories should be over-sampled—i.e. they should contain more via points than control points $n_{\text{via}} > n_{\text{ctrl}}$. The control points represent the degree of freedom of the B-spline trajectory; if the number of via points is less than or equal to the number of control points, the problem is under-constrained and the optimizer can perfectly match the constraints at the via points while likely violating them between the via points. For the spacecraft model, an oversampling ratio of approximately $4:1$ ($n_{\text{via}} \approx 4n_{\text{ctrl}}$) is used to enforce adherence to the constraints, specifically the exclusion zones. For the robotic manipulator, a ratio of approximately $2:1$ ($n_{\text{via}} \approx 2n_{\text{ctrl}}$) is applied to balance path constraint fidelity with computational efficiency for this more difficult optimization problem.

1.5.4 NLP Optimization Framework

The transcribed finite-dimensional NLPs are solved using the primal-dual interior-point solver IPOPT developed by Wächter and Biegler at Carnegie Mellon University for the solving of nonlinear, nonconvex optimization problems [17]. IPOPT is released as open-source software [17]. More can be found in the original publication by Wächter and Biegler [17]. The solver minimizes the objective function subject to path and boundary constraints by solving a sequence of barrier subproblems using a filter line-search method, with local optima satisfying the Karush-Kuhn-Tucker (KKT) conditions. Exact first- and second-order derivatives—specifically the constraint Jacobian $\nabla\mathbf{g}(\mathbf{w})$ and the Hessian of the Lagrangian $\nabla^2\mathcal{L}(\mathbf{w}, \boldsymbol{\lambda})$—are evaluated analytically via automatic differentiation using Computer Algebra System for Automatic Differentiation and Optimization (CasADi) [1]. Because B-splines possess local support, each decision variable influences only a small portion of the trajectory, i.e., a limited subset of collocation nodes, resulting in highly sparse, banded matrices that IPOPT exploits for computationally efficient linear solves.

1.6 Algorithm Frameworks

The core of this work is the implementation, evaluation, and comparison of global trajectory optimization approaches for realistic physical models and scenarios in aerospace and robotic applications. The spacecraft and robotic manipulator models and tasks prove highly nonlinear and are constrained by complex kinodynamic and environmental limitations. After a review of the SOTA of *kinodynamically-consistent* trajectory optimization methods, four algorithms are selected and introduced:

- Monte Carlo Gradient-Based Optimization (MC-GBO),
- Rapidly-exploring Random Tree Star with Gradient-Based Optimization (RRT*-GBO) [16],
- Probabilistic Homotopy Optimization (P-HO) [10],
- Inverse Dynamics Particle Swarm Optimization (IPSO) [14, 15].

This selection encompasses a diverse set of strategies—spanning naive multistart, sampling, homotopy/continuation, and metaheuristic optimization approaches, as well as a combination of these paradigms. Each algorithm is evaluated in terms of its ability to generate optimal trajectories while satisfying kinodynamic and environmental constraints. Primary evaluation criteria include solution quality, convergence speed, and robustness under varying test case scenarios. Each algorithm is further detailed in the following subsections.

1.6.1 Monte Carlo Gradient-Based Optimization (MC-GBO)

Monte Carlo with Gradient-Based Optimization (MC-GBO) serves as the naive-baseline multistart approach to global trajectory optimization, where multiple wholly independent initial trajectory guesses are generated and optimized to explore the search space, as visualized in Fig. 1.6. The advantages of MC-GBO include its simplicity and ease of parallelization. However, its main drawbacks are its lack of guidance between each independent optimization run—which results in numerous redundant solves—its direct, single-stage approach to solving the fully constrained, tight tolerance, highly nonlinear problem, and its difficulty in handling increasingly complex and constrained scenarios. Algorithm 1.1 details the procedure for this straightforward multistart approach: Define the base random seed s_0, the maximum number of Monte Carlo iterations N_{MC}, the desired number of solutions $N_{sol,des}$, and the maximum allowed time T_{max} before executing the optimization loop. For each Monte Carlo iteration, a new random initial trajectory guess $\mathbf{x}_0^{(i)}$ is generated using the current seed s_i via random_initial_guess()–sampling of the B-spline control points is detailed in § 1.3.1 and 1.3.2—and the optimization is performed to potentially update the best solution found so far $\mathbf{x}_{best}$. gbo_solve()($\mathbf{x}_0^{(i)}$) is called to perform the optimization of the NLP with the current initial trajectory guess using the chosen gradient-based optimizer–in this work, IPOPT. If the optimization is successful and yields a better solution than the current best, update $\mathbf{x}_{best}$ and c_{best}. Once any of the stopping criteria are met (maximum iterations N_{MC}, desired number of solutions $N_{sol,des}$, or maximum time T_{max}), the optimization loop terminates, and the best solution found is returned.

Critically, MC-GBO can parallelize the entire optimization process, as each Monte Carlo iteration is independent. Modern libraries and frameworks for parallel computing can be leveraged to significantly reduce the overall computation time by distributing the optimization tasks across multiple processors. This parallelizability is the saving grace of MC-GBO, allowing it to efficiently explore a large number of initial guesses without being bottlenecked by sequential optimization runs, thus keeping it relatively competitive to more sophisticated global optimization approaches.

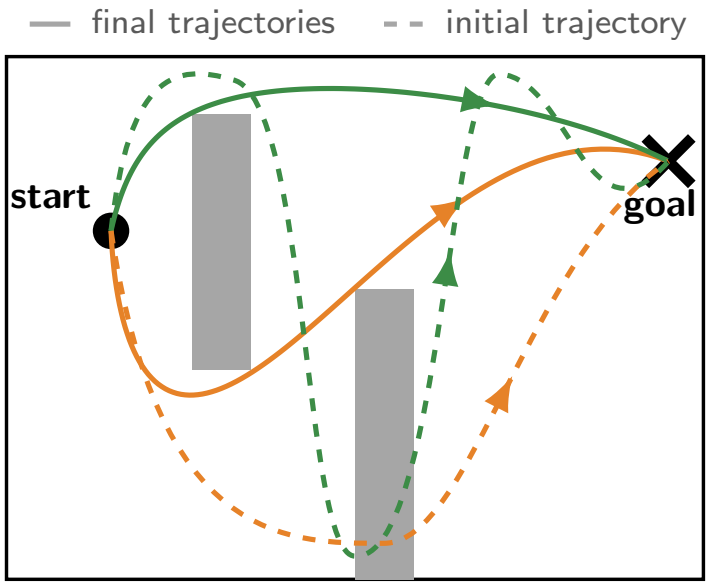


Figure 1.6: Illustration of MC-GBO showing multiple independently and randomly generated initial guess trajectories (dashed) and their corresponding optimized final trajectories (solid) in an environment with obstacles. Each refined trajectory is a local Gradient-Based Optimization (GBO)-based optimum; global search is achieved through the multiple independent initial guesses.

Algorithm 1.1: `MC_GBO`

Data: A base random seed s_0 used to generate independent initial trajectory guesses.
Result: Parallel optimization of trajectories to find the minimum cost path until some iteration limit, N_{MC}, (optional) desired number of solutions, $N_{sol,des}$, or (optional) maximum time T_{max}.

$c_{best} \leftarrow \infty$;
$\mathbf{x}_{best} \leftarrow \emptyset$;
$N_{sol} \leftarrow 0$;
for $i \leftarrow 0$ **to** $N_{MC} - 1$ **do (optionally in parallel)**
  if $T \geq T_{max}$ **then**
    exit loop;
  $s_i \leftarrow s_0 + i$;
  $\mathbf{x}_{init} \leftarrow$ `random_initial_guess`(s_i);
  $(converged, \mathbf{x}_{opt}, c) \leftarrow$ `gbo_solve`$(\mathbf{x}_{init})$;
  if *converged* **then**
    N_{sol}++;
    if $c < c_{best}$ **then**
      $c_{best} \leftarrow c$;
      $\mathbf{x}_{best} \leftarrow \mathbf{x}_{opt}$;
    if $N_{sol} = N_{sol,des}$ **then**
      exit loop;
return $(\mathbf{x}_{best}, c_{best})$;

1.6.2 Rapidly-exploring Random Tree Star with Gradient-Based Optimization (RRT*-GBO)

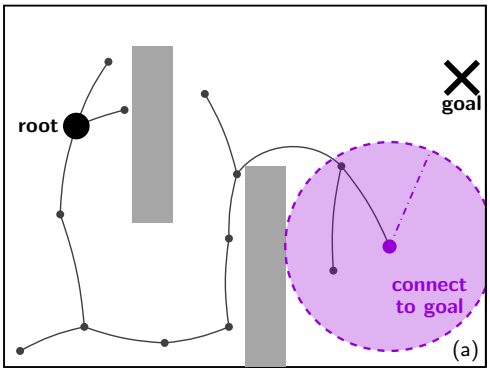
RRT*-GBO is the central focus of this work regarding its performance relative to other modern trajectory optimization methods [16]. Stoneman and Lampariello [16] build upon standard Rapidly-exploring Random Tree Star (RRT*)—first introduced by Karaman and Frazzoli [6] as an extension of the original Rapidly-exploring Random Tree (RRT) algorithm by La Valle [9]—by incorporating gradient-based optimization for the edge connections between tree nodes. Descriptions of each RRT variant are best reviewed in the original publications [6, 9, 16], the following discussion provides a high-level overview as it pertains to this work. The core strength of RRT*-GBO lies in combining the global exploration of sampling-based planners with local Gradient-Based Optimization (GBO), replacing traditional straight-line or SLERP connections. A primary limitation of standard RRT and RRT* in kinodynamic planning is that straight-line connections fail to respect system dynamics, resulting in kinodynamically infeasible trajectories that implicitly demand instantaneous control adjustments [16]. Additionally, straight-line connections are suboptimal across many cost landscapes [16]. The GBO edge connection addresses these fundamental shortcomings by directly solving a local BVP for each candidate edge, thereby strictly obeying the dynamics, boundary conditions, and path constraints [16]. While solving numerous local BVPs increases computational complexity [16], this drawback is significantly mitigated by generating sparser trees via node pruning, substantially reducing the required number of optimization solves [16]. In fact, RRT*-GBO often converges to high-quality solutions rapidly. As an extension of RRT*, RRT*-GBO inherits the asymptotic optimality with probability one over an infinite time horizon property [6]; this property guarantees that, given sufficient time and samples, the algorithm will converge to the optimal solution.

RRT*-GBO may be additionally extended with informed sampling strategies such as ellipsoidal heuristic bounding, which can accelerate convergence once an initial solution is found, this work employs standard uniformed formulation. Although an informed variant was fully implemented within the software framework developed for this thesis, it was deliberately excluded from the final results configuration in order to analyze and benchmark the foundational RRT*-GBO algorithm exactly as documented in the original publication [16].

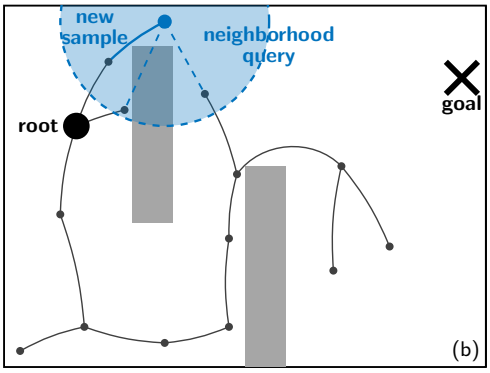
Algorithms Algorithms 1.2 to 1.5 summarize the main components of the RRT*-GBO approach [16]. Fig. 1.7 illustrates the tree construction and trajectory refinement process of the RRT*-GBO algorithm, highlighting particularly its ability to escape local minima and explore between homotopy classes. Certain parameters of RRT*-GBO require problem and model specific implementations; as we introduced two models in this work, the following subsections details their respective configurations and adaptations.

Spacecraft Model Implementation

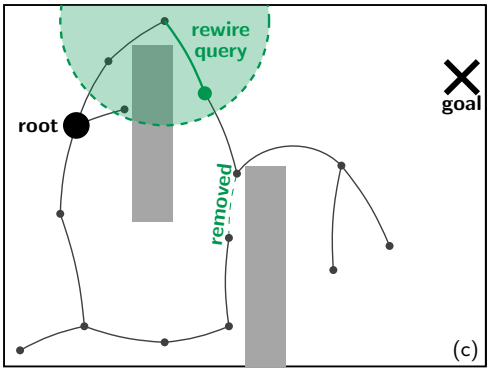
Two main functions of the RRT*-GBO algorithm require specific implementations for the spacecraft model: sampling via `random_generator()` and metrics for r_{prune}, r_{ball}, and $r_{connect}$. `random_generator()` generates random new quaternion nodes via uniform sampling on SO(3) using (1.77) and the method described in that section, with the consideration that $q \sim -q$.



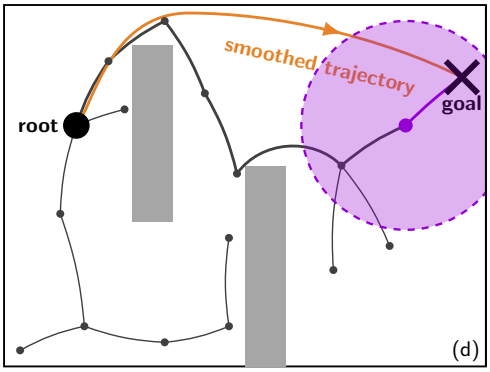
(a) Initial tree and goal connection attempt; the goal lies outside the connection radius.



(b) Neighborhood query for a new sample.



(c) Rewiring to a lower cost path and removal of the previous edge.



(d) Goal connection success and trajectory smoothing.

Figure 1.7: Illustration of the main stages of the RRT*-GBO tree construction and trajectory refinement.

Secondly, the metrics for r_{prune}, r_{ball}, and $r_{connect}$ are defined for the spacecraft model as a function of the principal eigenaxis angle between two unit quaternions, $\theta(q_{near}, q_{rand}) = 2\cos^{-1}(|q_{near}^T q_{rand}|)$. However, evaluating the inverse cosine for every nearest-neighbor query and proximity check incurs a high computational burden. To optimize performance, the algorithm evaluates distances using a more computationally efficient proxy metric:

$$d_{\text{proxy}}(q_{near}, q_{rand}) = 1 - |q_{near}^{\top} q_{rand}|^2 = \sin^2\left(\frac{\theta}{2}\right) \tag{1.137}$$

This proxy avoids the expensive transcendental calculation while remaining strictly monotonic over the interval $\theta \in [0, \pi]$. Since it exactly preserves the ordering of nearest-neighbor and radius tests, it guarantees the exact same algorithmic decisions as the raw angular metric. Furthermore, squaring the dot product natively resolves the dual-covering ambiguity of quaternions, ensuring invariance to q and $-q$. To maintain geometric intuition during tuning, the user-defined angular configuration radii (r_{prune}, r_{ball}, and $r_{connect}$) are simply converted into their equivalent proxy radii prior to execution.

Finally, a strategy is required when a sampled node q_{rand} falls outside the neighborhood of its nearest tree node q_{near}. If the distance exceeds the maximum radius r_{ball}, the sample is truncated to the edge of the ball along the shortest geodesic on SO(3). To execute this, the true angular separation θ is recovered from the proxy metric:

$$\theta = 2\cos^{-1}\left(\sqrt{1 - d_{\text{proxy}}(q_{\text{near}}, q_{\text{rand}})}\right) \tag{1.138}$$

If $\theta > r_{\text{ball}}$, an interpolation fraction $\alpha = \frac{r_{\text{ball}}}{\theta}$ is computed, and the sample is replaced using Spherical Linear Interpolation (SLERP):

$$q_{\text{new}} = \text{SLERP}(q_{\text{near}}, q_{\text{rand}}, \alpha) \tag{1.139}$$

This guarantees that q_{new} lies exactly r_{ball} radians from q_{near} along the shortest rotational path. All samples are checked to ensure they lie within the valid configuration space—i.e. outside of EZs—before being accepted as valid nodes in the tree. If the truncation feature is disabled, samples generated outside the ball are simply rejected, whereby a new sample is drawn.

Local, as well as goal, rewiring occurs based on path cost from root to node/goal alone, without consideration for the number of edges, duration, or other factors.

A final note not well captured in the pseudocode algorithms is the need to deal with the duration of the edges in the tree. In this work, a fixed duration was assigned for all edges in the tree; this commonly results in a raw edge trajectory far longer than fixed-duration specified in the BVP. The algorithm relies on the ability to scale the dynamics of the trajectory to fulfill the fixed-duration requirement only during the smoothing phase. The edge duration question remains an excellent open consideration for future research.

Robotic Manipulator Model Implementation

Likewise, the robotic manipulator model requires specific implementations for sampling and distance metrics, as well as a third more fundamental consideration.
`random_generator()` generates random new joint configuration nodes via uniform sampling within the joint limits of the manipulator, $\mathbf{q}_{\text{rand}} \sim \mathcal{U}(\mathbf{q}_{\min}, \mathbf{q}_{\max}) \in \mathbb{R}^7$. The metric used for r_{prune} and r_{ball} is the joint-space two-norm, defined as:

$$d(\mathbf{q}_{near}, \mathbf{q}_{rand}) = \|\mathbf{q}_{near} - \mathbf{q}_{rand}\|_2, \tag{1.140}$$

whereas the metric used for $r_{connect}$ is the task-space Euclidean distance and orientation difference, defined as:

$$d(\mathbf{x}_{ee}, \mathbf{x}_{ball}) = \|\mathbf{p}_{ee} - \mathbf{p}_{ball}\|_2, d(\mathbf{x}_{ee}, \mathbf{x}_{ball}) = \cos^{-1}(|a_{ee}^T a_{ball}|), \tag{1.141}$$

where $\mathbf{p}_i$ and $\mathbf{a}_i$—defined in § 1.3.2—represent the position and approach axis of the end-effector and ball corresponding to the 6-DoF state $\mathbf{x}_i$. A proxy equivalent can also be used for the approach axis comparison. A `connect_to_goal` attempt must pass both $r_connect$ metrics independently. $r_{connect}$ uses task-space metrics for both position and orientation, rather than joint-space distances, because the inverse dynamics solution to a specified end-effector pose generally has multiple valid joint configurations. In order to not bias the planner toward any particular inverse kinematics solution, task-space metrics are used.

Truncation behavior is handled similarly to the spacecraft model, but executes via linear interpolation in the 7-dimensional joint space rather than SLERP. If the joint-space distance d_q exceeds $r_{\text{ball,joint}}$, the sample is either rejected or truncated along the straight line segment toward $\mathbf{q}_{\text{near}}$. The truncated sample is calculated using the fraction $\alpha = \frac{r_{\text{ball,joint}}}{d_q}$:

$$\mathbf{q}_{\text{new}} = \text{clip}\Big(\mathbf{q}_{\text{near}} + \alpha(\mathbf{q}_{\text{rand}} - \mathbf{q}_{\text{near}}),\ \mathbf{q}_{\text{min}},\ \mathbf{q}_{\text{max}}\Big) \tag{1.142}$$

The clipping function ensures the newly interpolated configuration strictly adheres to the manipulator's physical joint bounds.

The more significant consideration for the robotic manipulator model is coming to terms with the goal being an arc corresponding to the ballistic path of the ball, rather than a fixed final pose. As such, the distance metrics for $r_{connect}$ evaluate the proximity of the end-effector to the entire arc of the ball's trajectory, and calculate the shortest distance to the arc and the corresponding ball time along its trajectory at that point. The corresponding point on the ball trajectory provides the initial guess for the terminal position of the end-effector and catch time t_f. The GBO solve then treats t_f as a free decision variable and adjusts it, together with the terminal joint configuration and trajectory control vertices, until the terminal catching constraints are satisfied within the prescribed time bounds. For non-goal edges, the BVP is modified slightly: rather than a free-time, free-position problem, the final position is fixed to the configuration of the newly added node and each edge time is a fixed span–as described for the spacecraft model. Thus, when goal rewiring occurs, often an entirely new path to a new point along the ball's trajectory is found, rather than a traditional rewiring of the existing successful path.

Algorithm 1.2: `RRT*_GBO`

Data: A tree $\mathcal{T}$ which stores states as nodes, x, and trajectories as edges, e, containing a start state.

Result: Incremental building of the tree until some iteration limit, N, (optional) desired number of solutions, $N_{sol,des}$, or (optional) maximum time T_{max}.

```
for i ← 0 to N do
    if T ≥ T_max then
        exit loop;
    connect_to_goal(x_last, x_goal);
    x_new ← random_generator();
    if x_new ∉ R_obstacle then
        x_near ← neighbor_query(x_new, x_near, r_prune, tree);
        if e_new converged then
            add_to_tree(e_new, x_new);
            rewire_query(x_near, x_new);
```

Algorithm 1.3: `connect_to_goal`(x_{last}, x_{goal})

```
e_goal ← gbo_extend(x_last → x_goal);
if e_goal converged then
    if metric(e_goal) ≤ r_connect then
        add_to_tree(e_goal, x_goal);
        N_sol++;
        if N_sol = N_sol,des then
            exit loop;
```

Algorithm 1.4: `neighborhood_query`$(x_{new}, r_{prune}, tree)$

```
for j ← 0 to length_of(tree) do
    e_j ← gbo_extend(tree.x[j] → x_new);
    if metric(e_j) ≥ r_prune then
        if metric(e_j) ≤ r_ball then
            x_near[j] = x[j] ;                    // store x[j] ∈ r_ball
            if metric(e_j) < metric(e_new) then
                e_new = e_j;
add_to_tree(e_new);
```

Algorithm 1.5: `rewire_query`$(x_{new}, \boldsymbol{x}_{near})$

```
for j ← 0 to length_of(x_near) do
    e_j ← gbo_extend(x_new → x_near[j]);
    if metric(e_j) < metric(e_near,incoming[j]) then
        remove_from_tree(e_near,incoming[j]);
        add_to_tree(e_j);
```

1.6.3 Probabilistic Homotopy Optimization (P-HO)

Probabilistic Homotopy Optimization (P-HO) is a stochastic continuation-based algorithm developed for trajectory optimization problems introduced by Pardis, Chignoli, and Kim [10]. The algorithm operates by exploring the the joint state-homotopy parameter search space defined by $\mathcal{X} \times \mathcal{H}_d$ where $\mathcal{X}$ represents the set of possible decision variables and $\mathcal{H}_d \in [0,1]^d$ represents the set of d-dimensional homotopy parameters. While classical homotopy methods typically follow a deterministic path from the initial to the goal problem, P-HO introduces stochastic sampling in the selection of homotopy parameters with $\mathcal{H}_d = \mathbf{0}$ being the baseline unconstrained or minimally constrained problem and $\mathcal{H}_d = \mathbf{1}$ representing the fully constrained goal problem. P-HO grows an optimization tree $\mathcal{T}$ in the joint state-homotopy parameter space, with each tree node representing a converged candidate solution $\mathbf{x} \in \mathcal{X}$ and its associated homotopy parameter $\boldsymbol{\lambda} \in \mathcal{H}_d$. A thorough description of the P-HO algorithm can be found in [10].

Initialization of the algorithm is discussed in [10], but no universal method for generating initial trajectories at $\boldsymbol{\lambda} = \mathbf{0}$ is described. Instead, the initial guess $\mathbf{x}_0$ is a user defined prerogative. However, the core requirement is that the homotopy parameter mapping $\Gamma(\boldsymbol{\lambda})$ must be designed such that at $\boldsymbol{\lambda} = \mathbf{0}$, the problem $\mathcal{P}_{\Gamma(\mathbf{0})}$ possesses a *sufficiently large* basin of attraction—i.e. even a trivial initial guess should converge to a feasible solution [10]. Neither is the number of initial guesses specified, again leaving it to implementation discretion [10]. In this work, we adopted a variable number of initial guesses, which becomes a hyperparameter that can be tuned based on the specific problem instance and computational resources available. Each initial guess solve compares the resulting trajectory against the existing solutions in the optimization tree to determine if it represents a novel solution; repeated solution are discarded and attempted again up to five times. Likewise, the state vector is initialized either randomly or randomly with a single informed guess based on a heuristic–such as a geodesic interpolation between the start and goal states.

Model Specific Implementation The algorithm defines $\boldsymbol{\lambda}$ as potentially being d-dimensional, but in this work it was treated as a scalar for simplicity $d = 1$, with it scaling all homotopy constraints simultaneously. Although other approaches are possible, our implementation applies homotopy continuation to path constraints while boundary conditions remain strictly enforced for all values of the homotopy parameter λ.

- **Spacecraft Model:** λ smoothly introduces task constraints along $[0,1]$:

  1. Exclusion zones: the exclusion zone radius from relaxed to constrained one of two ways, either removing EZs entirely or merely minimizing them greatly, as shown respectively as:
  $$r_{\mathrm{EZ}}(\lambda) = \lambda r_{\mathrm{EZ}} \tag{1.143}$$
  $$r_{\mathrm{EZ}}(\lambda) = [(1-s) + s\lambda] r_{\mathrm{EZ}}, \tag{1.144}$$
  with s representing the shrink factor, for this work's analysis set as $s = 0.9$, resulting in an initial EZ radius 10% of the final.
  2. Angular velocity limits: the angular velocity limits are set at ten-times the nominal value initially and gradually reduced to the actual limits as λ increases:
  $$\omega_{\max}(\lambda) = [1 + 9(1-\lambda)]\omega_{\max}. \tag{1.145}$$

3. Control torque limits: the torque limits are set at ten-times the nominal value initially and gradually reduced to the actual limits as λ increases:

$$\tau_{\max}(\lambda) = [1 + 9(1 - \lambda)]\tau_{\max}. \tag{1.146}$$

- **Robotic Manipulator:** The parameter vector $\boldsymbol{\lambda} = [\lambda_{\mathbf{g}}, \lambda_{\boldsymbol{\tau}}, \lambda_{\text{ee}}]^\top$ regulates dynamic difficulty:

1. Joint position limits: the position limits are set at $\pm 2\pi$ initial and gradually reduced to the actual limits of each individual joint as λ increases:

$$q_{\min}(\lambda) = (1 - \lambda)(-2\pi) + \lambda q_{\min}, \tag{1.147}$$

$$q_{\max}(\lambda) = (1 - \lambda)(2\pi) + \lambda q_{\max}. \tag{1.148}$$

2. Joint velocity limits: the velocity limits are set at ten-times the nominal value initially and gradually reduced to the actual limits of each individual joint as λ increases:

$$\omega_{\max}(\lambda) = [1 + 9(1 - \lambda)]\omega_{\max}. \tag{1.149}$$

3. Control torque limits: the torque limits are set at ten-times the nominal value initially and gradually reduced to the actual limits as λ increases:

$$\tau_{\max}(\lambda) = [1 + 9(1 - \lambda)]\tau_{\max}. \tag{1.150}$$

4. End-effector orientation and position constraints: these constraints are not relaxed in our formulation, instead remaining strictly enforced throughout the planning process.

Initial guess generation for the of the relaxed problem is done via:

- **Spacecraft Model:** The initial guess for the spacecraft model uses a single informed trajectory with all remaining initial guesses generated randomly.

1. Random initialization: the control vertices parameterizing the B-spline trajectory are randomly initialized as described in § 1.3.1.
2. Informed initialization: one initial guess is generated using a trajectory SLERP-interpolation between the initial and goal orientations.

- **Robotic Manipulator:** The initial guess for the robotic manipulator uses pure random initialization without an informed trajectory due to the inverse dynamics generating multiple solutions for a given end-effector trajectory.

1. Random initialization: the control vertices parameterizing the B-spline trajectory are randomly initialized within the control vertex bounds as described in § 1.3.2.

P-HO Workflow The P-HO framework alternates between a *sampling phase* and a *solving phase* based on the solve-to-sample ratio $\rho = \frac{|\mathcal{S}_\mathcal{A}|}{|\mathcal{T}|\times|\mathcal{S}_\lambda|}$, where $\mathcal{S}_\lambda$ is the set of sampled homotopy parameters, $\mathcal{T}$ is the set of nodes in the optimization tree, and $\mathcal{S}_\mathcal{A}$ tracks previously attempted solver pairs $(N_{i,j}, \boldsymbol{\lambda}_l)$:

1. **Initialization:** The optimization tree is initialized at the root node $N_{0,0} = (\mathbf{x}_0^*, \mathbf{0}_d)$. The initial guess $\mathbf{x}_0$ easily converges under $\mathcal{P}_{\Gamma(\mathbf{0}_d)}$ because constraints are fully relaxed. The parameter candidate set is seeded as $\mathcal{S}_\lambda = \{\mathbf{0}_d, \mathbf{1}_d\}$.

2. **Solve Phase ($\rho \leq \rho_\mathcal{A}$):** A candidate node $N_{i,j} = (\mathbf{x}_i^*, \boldsymbol{\lambda}_j)$ and a target parameter vector $\boldsymbol{\lambda}_l \in \mathcal{S}_\lambda$ are selected via `sampleAttempt` (Algorithm 1.7). With probability P_g, the target parameter is directly chosen as the goal vector $\mathbf{1}_d$. The local gradient solver (IPOPT) attempts to solve $\mathcal{P}_{\Gamma(\boldsymbol{\lambda}_l)}$ using $\mathbf{x}_i^*$ as a warm start:

$$\mathbf{x}_{\text{new}}^* = \texttt{solve}\left(\mathcal{P}_{\Gamma(\boldsymbol{\lambda}_l)}, \mathbf{x}_i^*\right). \tag{1.151}$$

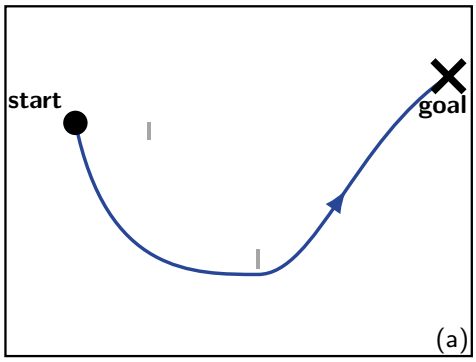
   If IPOPT successfully converges to a feasible local minimum $\mathbf{x}_{\text{new}}^* \in \mathcal{C}$ that represents a novel trajectory, $N_{\text{new}} = (\mathbf{x}_{\text{new}}^*, \boldsymbol{\lambda}_l)$ is added to $\mathcal{T}$.

3. **Sample Phase ($\rho > \rho_\mathcal{A}$):** A new parameter vector $\boldsymbol{\lambda}_{\text{new}} \sim \mathcal{U}([0,1]^d)$ is uniformly sampled and appended to $\mathcal{S}_\lambda$, opening new directional pathways through the parameter space.

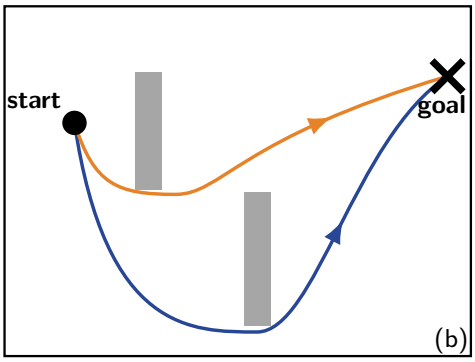
The advantage of the P-HO framework lies in its ability to explore the solution space and jump between different homotopy paths, thereby increasing the likelihood of finding a globally optimal trajectory. Specifically, P-HO demonstrates a robustness to common topological pitfalls of homotopy methods [10]:

- **Bifurcations and Folds:** When the solution manifold splits into multiple distinct homotopy branches or folds back on itself in parameter space, P-HO maintains distinct tree branches rooted at identical parameter values $\boldsymbol{\lambda}$. This enables tracking of disconnected local minima without getting trapped [10].

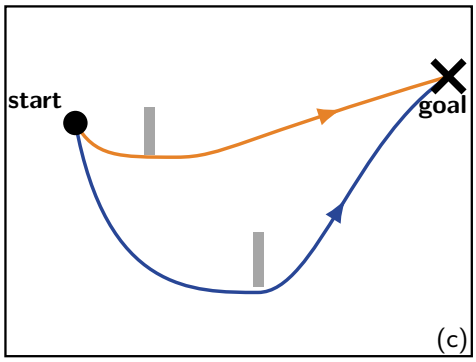
- **Disconnected and Abbreviated Paths:** Due to the sampling strategy, P-HO can effectively navigate between disconnected and abbreviated homotopy paths, using solutions from previously explored branches as warm starts to bridge gaps in the solution manifold [10].



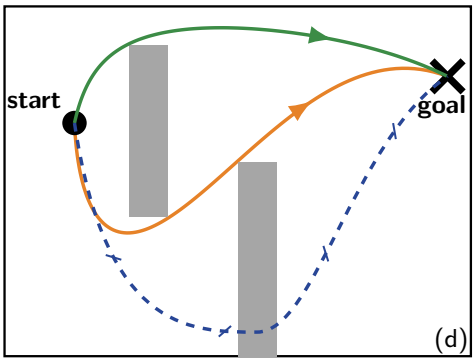
Initialization of the relaxed NLP, $\lambda = 0$.



First homotopy sample, $\lambda = 0.8$. The single seed from the $\lambda = 0$ solve is used as a warm start and uncovers solutions of two homotopy classes.



Second homotopy sample, $\lambda = 0.35$. The algorithm continues to explore the solution space using previous solutions as warm starts.



Goal homotopy sample, $\lambda = 1$. The algorithm attempts the fully constrained solve. One homotopy class vanishes.

Figure 1.8: Illustration of P-HO demonstrating stochastic sampling and the resulting ability to navigate between different homotopy classes in the solution space. The colored trajectories show candidate solutions obtained as the homotopy parameter progresses from the relaxed problem to the fully constrained goal problem.

Algorithm 1.6: `P_HO`

Data: Initial guess $\mathbf{x}_0$, iteration limit N, solve/sample threshold $\rho_{\mathcal{A}}$, and an optimization tree $\mathcal{T}$.
Result: A set of optimal trajectories corresponding to the goal problem parameterization $\mathbf{1}_d$, until an iteration, (optional) solution-count, $N_{sol,des}$, or (optional) time limit, T_{max}, is reached.

$\mathcal{T} \leftarrow \{(\texttt{solve}(\mathcal{P}_{\Gamma(\mathbf{0}_d)}, \mathbf{x}_0), \mathbf{0}_d)\}$;
$\mathcal{S}_\lambda \leftarrow \{\mathbf{1}_d, \mathbf{0}_d\}$;
$\mathcal{S}_{\mathcal{A}} \leftarrow \emptyset$;
$N_{sol} \leftarrow 0$;
for $k \leftarrow 1$ **to** N **do**
  if $T \geq T_{max}$ **then**
    exit loop;
  if $\frac{|\mathcal{S}_{\mathcal{A}}|}{|\mathcal{T}|\times|\mathcal{S}_\lambda|} \leq \rho_{\mathcal{A}}$ **then**
    $(\mathbf{N}_{i,j}, \boldsymbol{\lambda}_l) \leftarrow \texttt{sampleAttempt}(\mathcal{T}, \mathcal{S}_\lambda, \mathcal{S}_{\mathcal{A}})$;
    $\mathcal{S}_{\mathcal{A}} \leftarrow \mathcal{S}_{\mathcal{A}} \cup (\mathbf{N}_{i,j}, \boldsymbol{\lambda}_l)$;
    $\mathbf{x}^*_{new} \leftarrow \texttt{solve}(\mathcal{P}_{\Gamma(\boldsymbol{\lambda}_l)}, \mathbf{x}^*_i)$;
    if $\mathbf{x}^*_{new} \in \mathcal{C}$ ***and*** $\mathbf{x}^*_{new} \neq \mathbf{x}^*_j \; \forall \mathbf{N}_{j,\cdot} \in \mathcal{T}$ **then**
      $\mathcal{T} \leftarrow \mathcal{T} \cup (\mathbf{x}^*_{new}, \boldsymbol{\lambda}_l)$;
      if $\boldsymbol{\lambda}_l = \mathbf{1}_d$ **then**
        N_{sol}++;
        if $N_{sol} = N_{sol,des}$ **then**
          exit loop;
  else
    $\mathcal{S}_\lambda \leftarrow \mathcal{S}_\lambda \cup \texttt{sample}(\mathcal{H}_d)$;
return $\{\mathbf{x}^*_i : \mathbf{N}_{i,j} \in \mathcal{T}, \boldsymbol{\lambda}_j = \mathbf{1}_d\}$;

Algorithm 1.7: `sampleAttempt`

if *with probability* P_g **then**
  $\mathbf{N} \leftarrow \texttt{sample}(\{\mathbf{N}_{i,j} \in \mathcal{T} : (\mathbf{N}_{i,j}, \mathbf{1}_d) \notin \mathcal{S}_{\mathcal{A}}\})$;
  return $(\mathbf{N}, \mathbf{1}_d)$;
else
  return $\texttt{sample}(\{(\mathbf{N} \in \mathcal{T}, \boldsymbol{\lambda} \in \mathcal{S}_\lambda) : (\mathbf{N}, \boldsymbol{\lambda}) \notin \mathcal{S}_{\mathcal{A}}\})$;

1.6.4 Inverse Dynamics Particle Swarm Optimization (IPSO)

IPSO is a metaheuristic, derivative-free global trajectory optimization framework based on swarm intelligence and differential flatness, originally introduced by Spiller et al. [14, 15] specifically for spacecraft slew trajectory optimization. Fig. 1.9 illustrates the particle update mechanism in IPSO, highlighting the contributions of the inertia, individual-best, and social-best components to the overall velocity update. In this work, IPSO synthesizes and adapts the algorithms presented in [14, 15] into a hybrid multistart framework specifically for the spacecraft model alone. Unlike gradient-based methods that rely on derivates to perform updates, IPSO leverages a population of candidate solutions (particles) that explore the search space collectively, guided by both individual and neighborhood experiences. The implementation combines Spiller's original near-optimal IPSO strategies with follow-on GBO refinement to settle results into local optima.

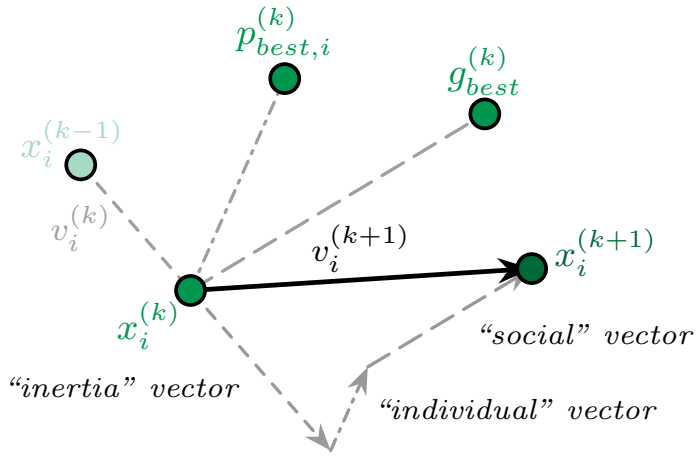


Figure 1.9: Illustration of the IPSO particle update, showing the inertia, individual-best, and social-best vector components contributing to the velocity update.

The inverse-dynamics formulation is central to the IPSO approach. Following Spiller, the particles parameterize flat-output trajectories and their derivatives, while the required control torques are obtained from the equations of motion [14, 15]. Consequently, each particle represents a complete candidate maneuver, and its fitness can be evaluated from the resulting state, control, and constraint quantities without treating the control history as a separate set of particle variables or repeatedly forward-integrating the dynamics [14, 15]. This reduces the computational cost and numerical accumulation of errors during the large number of fitness evaluations required by a swarm-based method. Additionally, boundary constraints are inherently satisfied via the analytical evaluation of the initial and final control vertices, without NLP solver evaluation of the equality constraints.

Attitude Parameterization and Search Space

Unlike all other methods presented herein, IPSO does not directly operate on quaternion-based representations of attitude. To circumvent the unit-norm equality constraint $\|\mathbf{q}\|_2 = 1$ required by quaternion-based B-splines—which poses severe challenges for unconstrained particle velocity updates—IPSO parameterizes attitude trajectories using MRP $\boldsymbol{\sigma} \in \mathbb{R}^3$ as flat outputs [14, 15]. As detailed in § 1.1.1, MRP provide a minimal, 3-parameter description of SO(3) derived via stereographic projection.

The search hyper-cube $[\mathbf{x}_{\min}, \mathbf{x}_{\max}]$ for particle positions is defined either explicitly via a specified angular limit θ_{search} or derived dynamically from the endpoint geodesic rotation angle $\theta_{\text{geodesic}} = 2\cos^{-1}(|\mathbf{q}_0^\top \mathbf{q}_f|)$ scaled by a factor s_{bound}:

$$p_{\max} = \tan\left(\frac{\min(2\pi - \varepsilon,\, s_{\text{bound}} \cdot \theta_{\text{geodesic}})}{4}\right), \tag{1.152}$$

yielding upper and lower bounds $\mathbf{x}_{\max} = p_{\max}$ and $\mathbf{x}_{\min} = -p_{\max}$. A maximum search box is set strictly less than 2π to ensure that the particles do not encounter singularities associated with the MRP parameterization. The swarm population is initialized by seed-

ing the first particle with a straight-line geodesic path in MRP space, while the remaining $N_{\text{particles}} - 1$ particles are uniformly sampled across the search box $\mathcal{U}(\mathbf{x}_{\min}, \mathbf{x}_{\max})$.

Several specifics of algorithm functions are enumerated as:

- `evaluate_fitness()`: Evaluates the fitness of a particle based on the objective function and constraint violations.

$$f_j = \frac{J_{\text{objective}}(P_j) + J_{\text{torque}}(P_j) + J_{\text{angular velocity}}(P_j) + J_{\text{exclusion}}(P_j)}{C_{\text{scale}}} + w_{\text{violation}} N_{\text{viol}}(P_j, \delta), \tag{1.153}$$

  with J_{torque}, $J_{\text{angular velocity}}$, and $J_{\text{exclusion}}$ representing the respective penalty terms for torque, angular velocity, and exclusion constraint violations, while $w_{\text{violation}}$ is the weight assigned to the violation count $N_{\text{viol}}(P_j, \delta)$. The penalty terms scale with the respective magnitudes of the constraint violations, whereas the violation count term assigns a fixed penalty for each violation simply discouraging any violation at all. Violations are penalized on a per-element basis. Additionally, torque and angular-rate limits are always evaluated strictly; δ relaxes only the exclusion-zone constraint during the adaptive tightening phase, adding a degree of homotopy to the optimization process.

- `update_velocity()`: Updates the individual particle velocities based on personal best, local best, and global best particle positions as shown in Fig. 1.9.

$$v_{j,d}^{k+1} = r\Big[w^k v_{j,d}^k + u_{p,j,d}^k c_p \left(p_{j,d}^k - x_{j,d}^k\right) + u_{l,j,d}^k c_l^k \left(l_{j,d}^k - x_{j,d}^k\right) + u_{g,j,d}^k c_g^k \left(g_d^k - x_{j,d}^k\right)\Big], \tag{1.154}$$

  where sub/superscripts j, d and k denote the particle, dimension, and iteration indices, respectively—i.e. $p_{j,d}^k$ is the personal best position of particle j in dimension d at iteration k—and $u_{p,j,d}^k$, $u_{l,j,d}^k$, $u_{g,j,d}^k \sim \mathcal{U}(0,1)$. Weights w, c_p, c_l, and c_g are the inertia, personal, local, and global weight coefficients, respectively; v, p, l, and g represent the velocity, personal best, local best, and global best position terms; x denotes the current position of the particle; and r is an additional multiplicative scale factor depending on the problem [14].

- `clamp()`: Clamping limits the particle velocities to predefined bounds to prevent excessively large and unstable updates:

$$v_{j,d}^{k+1} = \text{clip}\left(v_{j,d}^{k+1}, -v_{\max,d}, v_{\max,d}\right), \tag{1.155}$$

- `compute_scheduled_weights()`: To dynamically shift the swarm from global exploration to localized exploitation, the inertia w, local c_l, and global c_g weights are varied via linear interpolation:

$$\psi(\rho) = \psi_{\text{initial}} + (\psi_{\text{final}} - \psi_{\text{initial}})\,\rho, \quad \text{with} \quad \rho = \min\left(\frac{\text{progress}}{N_{\text{schedule}}}, 1\right), \tag{1.156}$$

  where N_{schedule} sets the schedule horizon. The scheduling for each coefficient are structured as follows:

  - Inertia Weight (w): Evaluated at every inner iteration using the progress metric $\rho_w = \frac{o + i/N_{\text{inner}}}{N_{\text{schedule}}}$, where o and i denote the outer and inner iteration counters, respectively. The inertia linearly decays from $w : w_{\text{initial}} \longrightarrow w_{\text{final}}$, gradually dampening the influence of past particle momentum.

  - Local-Neighborhood Weight (c_l): Scheduled solely over outer iterations via $\rho_l = \frac{o}{N_{\text{schedule}}}$, decaying from $c_l : c_{l,\text{initial}} \longrightarrow c_{l,\text{final}}$. This provides strong early exploration within local particle neighborhoods before fading entirely after N_{schedule} outer iterations.
  - Global-Best Weight (c_g): Follows the same outer-iteration progress metric as ρ_l, increasing from $c_g : c_{g,\text{initial}} \longrightarrow c_{g,\text{final}}$. This shifts the particle search dynamics from neighborhood interaction to attraction toward the global best position.
  - Cognitive Weight (c_p): The personal-best coefficient remains static throughout optimization at $c_p = c_{p,\text{initial}}$.

- `check_convergence()`: Evaluates whether the optimization process has met the predefined convergence criteria defined as: Following each outer cycle o, the solver records the current global-best fitness score $J_g^{(o)}$. Once N_C outer-cycle evaluations are available, the mean relative improvement $\Delta\bar{J}$ over the sliding history window N_C is calculated as:

$$\Delta\bar{J} = \frac{1}{N_C} \sum_{k=o-N_C}^{o-1} \frac{J_g^{(k)} - J_g^{(k+1)}}{\max\left(|J_g^{(k)}|,\, \varepsilon_{\text{div}}\right)}, \tag{1.157}$$

  where $\varepsilon_{\text{div}} > 0$ prevents zero division. Convergence is declared when $\Delta\bar{J} < \varepsilon_{\text{tol}}$ (with $\varepsilon_{\text{tol}} = 10^{-6}$), subject to the strict requirement that the global-best trajectory is fully feasible ($N_{\text{viol}} = 0$ at zero relaxation tolerance $\delta = 0$). This formulation aligns the convergence criteria defined in Spiller's 2018 paper [15], replacing the earlier metric from Spiller's 2016 work [14].

The resulting solutions generated by the IPSO algorithm are reported as *near-optimal* solutions; in order to directly compare them with our other algorithms, we implement the IPSO solutions as warm starts for a standard GBO run using our standard CasADi paired with IPOPT framework.

Algorithm 1.9: `IPSO_GBO`

Data: Warmstart trajectories from the IPSO runs for GBO refinement.
Result: Best optimized trajectory $\mathbf{x}_{best}$ and its corresponding minimum cost c_{best}.

$\mathcal{N} \leftarrow \emptyset$;
for $r \leftarrow 0$ **to** $N_{runs} - 1$ **do in parallel**
    $s_r \leftarrow s_0 + r$;
    $(P_{opt}, c_f) \leftarrow$ `IPSO_solve`(P_0, s_r);
    $\mathcal{N} \leftarrow \mathcal{N} \cup \{(P_{opt}, c_f)\}$;
$\mathcal{N}_{feasible} \leftarrow$ `sort_and_filter_feasible`$(\mathcal{N})$;
for $k \leftarrow 1$ **to** N_{ws} **do**
    $\mathbf{x}_{warm} \leftarrow \mathcal{N}_{feasible}[k].P_{opt}$;
    $(converged, \mathbf{x}_{opt}, c) \leftarrow$ `gbo_solve`$(\mathbf{x}_{warm})$;
    if *converged* ***and*** $c < \mathcal{N}_{feasible}[k].c_f$ **then**
        $\mathcal{N}_{feasible}[k] \leftarrow (\mathbf{x}_{opt}, c)$; ; `// Overwrite with refined GBO solution`
$(\mathbf{x}_{best}, c_{best}) \leftarrow$ `select_minimum_cost`$(\mathcal{N}_{feasible})$;
return $(\mathbf{x}_{best}, c_{best})$;

Algorithm 1.8: `IPSO_solve`

Data: Initialization of constants, swarm and tolerances.
Result: Optimized global best trajectory P^{global} and its final fitness f^{global}, until a maximum outer iteration N_{outer} or time T_{max} limit is reached.

```
1  initialize_swarm(P_0, s_r);
2  δ ← δ_0;
3  for o ← 0 to N_outer − 1 do
4      if T ≥ T_max then
5          exit loop;
6      for i ← 0 to N_inner − 1 do
7          for j ← 1 to N_particles do
8              f_j ← evaluate_fitness(P_j, δ);
9              if f_j < f_j^best then
10                 P_j^best ← P_j;
11                 f_j^best ← f_j;
12         (P^local, P^global, f^global) ← update_neighborhood_bests(P^best, f^best);
13         (w, c_local, c_global) ← compute_scheduled_weights(o, i);
14         for j ← 1 to N_particles do
15             r_p, r_l, r_g ~ U(0, 1);
16             V_j ← update_velocity();
17             P_j, V_j ← clamp();
18     δ_new ← update_tolerance(P^global, δ);
19     if δ_new < δ then
20         reset_swarm_memories() ;                          // Reset bests
21         δ ← δ_new;
22     if check_convergence(P^global, H, K, ε_conv) then
23         exit loop;
24 return (P^global, f^global);
```

1.6.5 Adaptive Constraint Refinement

A known limitation of discrete collocation is the potential for path constraint violations between evaluation points. While the optimization enforces constraints at discrete, uniform via points, we formulated an adaptive local refinement process to guarantee continuous-time feasibility. To preclude inter-sample constraint violations, the solver analytically audits the continuous trajectory post-optimization. This method was implemented for the spacecraft model, but could equally be applied to other dynamical systems where continuous-time constraint satisfaction is critical.

After an initial B-spline optimization sequence, the continuous-time angular velocity and control torque profiles, $\boldsymbol{\omega}(t)$ and $\boldsymbol{\tau}(t)$, are reconstructed as piecewise polynomials. Let the B-spline knot times be defined as $\boldsymbol{\kappa} = \{t_0, t_1, \ldots, t_{n+p+1}\}$ with $t_0 = 0$ and $t_{n+p+1} = t_f$, where t_f is the final time of the maneuver. Each knot span is evaluated over a local time variable $s = t - t_i$, where t_i is the starting time of the current knot span and s has a range of $[0, \Delta T = t_{i+1} - t_i]$. On each knot span, the quaternion-based trajectory is represented by polynomials defined in § 1.3.1. Likewise, the EOMs for angular velocity and torque are defined in the same section. Because the quaternions are polynomial on each knot span, so too are the angular velocity and angular acceleration, which then provides the torque.

To identify constraint violations that escaped the initial via point grid, the exact timestamps of interior extrema are identified by extracting the roots of their time derivatives, $\frac{d\boldsymbol{\omega}(s)}{ds} = 0$ and $\frac{d\boldsymbol{\tau}(s)}{ds} = 0$. In addition to the span-wise extrema, the segment endpoints are also considered. Only valid roots within the current knot span are retained for evaluation and must satisfy the conditions:

$$|\mathrm{Im}(s)| = 10^{-2} \quad \text{and} \quad 0 \leq \mathrm{Re}(s) \leq \Delta T. \tag{1.158}$$

The local roots are the converted back to the global time frame by adding the starting time of the current knot span, t_i, to each valid root s.

The method evaluates the these analytical extrema against the trajectory bounds, detecting any path constraint violations. Upon identifying a violation, the exact timestamp as well as a dense cluster of neighboring times, scaled to a proportion 20% of the local grid span are inserted as new via points in the via point grid— i.e. the time grid is locally densified.

The optimizer then re-solves the problem utilizing the previously solved B-spline coefficients as a warm start. This cycle repeats until continuous-time dynamic constraints are fully satisfied or a refinement iteration limit (typically five iterations) is reached. Even if the refinement limit is reached, violating points may still be analytically identified, allowing for informed decision-making regarding trajectory acceptance. Consequently, strict feasibility may be enforced for the continuous-time trajectory without necessitating an excessively dense global discretization, ensuring analytical rigor while maintaining computational efficiency.

1.7 Analytical Verification

To verify the numerical optimal control solver's implementation, we establish first an analytical solution using Pontryagin's Minimum Principle (PMP). Specifically, we de-

rive the optimal angular velocity profile for a single-axis slew minimizing control effort (squared torque), which is known to yield a strictly quadratic velocity trajectory.

The cost functional is defined as the integral of the control effort:

$$J = \int_0^{t_f} \tau^2(t)\, dt, \tag{1.159}$$

subject to the simplified rotational dynamics and kinematics of a rigid body with a spherical inertia matrix–i.e. the gyroscopic cross coupling terms typical of rotation EOMs cancel:

$$\dot{\omega} = \frac{\tau}{I}, \quad \dot{\theta} = \omega, \tag{1.160}$$

and the final boundary constraint for the total slew rotation:

$$\int_0^{t_f} \omega(t)\, dt = \theta_f - \theta_i = \Delta\theta. \tag{1.161}$$

Defining the state vector as $\mathbf{x} = \begin{bmatrix}\theta & \omega\end{bmatrix}^\top$, the state equations become $\dot{\mathbf{x}} = \begin{bmatrix}\omega & \tau/I\end{bmatrix}^\top$. Utilizing the running cost $L = \tau^2(t)$, the Hamiltonian H is constructed as:

$$H = L + \boldsymbol{\lambda}^\top \dot{\mathbf{x}} = \tau^2 + \lambda_1\omega + \lambda_2\frac{\tau}{I}. \tag{1.162}$$

Applying PMP yields the costate differential equations ($\dot{\boldsymbol{\lambda}} = -\frac{\partial H}{\partial \mathbf{x}}$) and the optimality condition ($0 = \frac{\partial H}{\partial \tau}$). Solving the costate equations gives:

$$\dot{\lambda}_1 = -\frac{\partial H}{\partial \theta} = 0 \implies \lambda_1 = c_1, \tag{1.163}$$

$$\dot{\lambda}_2 = -\frac{\partial H}{\partial \omega} = -\lambda_1 = -c_1 \implies \lambda_2 = \int -c_1\, dt = -c_1 t + c_2. \tag{1.164}$$

Evaluating the optimality condition allows us to isolate the optimal control torque $\tau(t)$:

$$\frac{\partial H}{\partial \tau} = 2\tau + \frac{\lambda_2}{I} = 0 \implies \tau(t) = -\frac{1}{2I}\lambda_2 = -\frac{1}{2I}(-c_1 t + c_2). \tag{1.165}$$

Rewritten in terms of the new constants c_3 and c_4, this reveals that the optimal control torque takes the form of a linear polynomial, $\tau(t) = c_3 t + c_4$.

Substituting this torque profile back into the system dynamics ($\dot{\omega} = \tau/I$) and integrating yields the optimal angular velocity trajectory:

$$\omega(t) = \int \frac{c_3 t + c_4}{I}\, dt \implies \omega(t) = \frac{c_3}{2I}t^2 + \frac{c_4}{I}t + c_5. \tag{1.166}$$

Again rewriting the constants c_3, c_4, and c_5 as a, b, and c, the analytical optimal angular velocity profile for a control-effort minimization slew is quadratic:

$$\omega(t) = at^2 + bt + c, \tag{1.167}$$

where the constants a, b, and c are determined by the boundary conditions ω_i, ω_f, and the required slew angle Δq.

This closed-form quadratic result serves as a ground truth baseline to validate the solutions generated by the numerical B-spline nonlinear programming (NLP) solver.

Chapter 2

Experimental Setup

Write a brief introduction summarizing the purpose of this chapter: to define the hardware, software, evaluation metrics, and exact physical and algorithmic parameters required to reproduce the global trajectory optimization benchmarks.

2.1 Computational Environment & Software Stack

The experiments were conducted on four workstations running Ubuntu 24.04.5 LTS on an $x86_{64}$ architecture. The first computer is equipped with an Intel® Xeon® CPU E5-1630 v4 @ 3.70 GHz, providing 4 physical cores and 8 logical processors and 16 GB of RAM. The second computer is equipped with an Intel® Core™ Ultra 7 265 CPU, providing 10 physical cores and 20 logical processors and 32 GB of RAM. The third computer is equipped with an Intel® Xeon® CPU E5-2687W v3 @ 3.10 GHz, providing 10 physical cores and 20 logical processors, and 16 GB of RAM. The fourth computer is equipped with an Intel® Core™ i7-8700K CPU @ 3.70 GHz, providing 6 physical cores and 12 logical processors and 32 GB of RAM. As these workstations are materially different in terms of CPU and RAM, the performance results will include notes indicating which workstation was used for each experiment. Hereafter, the workstations will be referred to as Workstation 1, Workstation 2, Workstation 3, and Workstation 4, corresponding to the order in which they were described above.

The code framework is based on a Python orchestration layer that interacts with a C++ core for computational efficiency. CasADi constructs the symbolic B-spline dynamics, objective, constraints, and automatic differentiation, while IPOPT solves the resulting nonlinear programs with HSL MA27 as the linear solver. NumPy provides vectorized numerical operations, including trajectory sampling. SciPy provides selected interpolation, B-spline, spatial, and rotation utilities used by parts of the framework.

IPOPT convergence criteria are unique to each solver as shown in Table 2.1. All other unspecified stopping criteria use the IPOPT defaults:

Table 2.1: Solver-specific stopping criteria used in the experiments.

Algorithm	**Solver stage**	**IPOPT max_iter**	**IPOPT tol**	**Other stopping criteria**
MC, both systems	Single-stage solve	500	10^{-10}	`constr_viol_tol=1e-4`
P-HO, both systems	Intermediate solve	500	10^{-2}	`acceptable_tol=5e-2`; `acceptable_iter=3`; `constr_viol_tol=1e-2`
P-HO, both systems	Final solve	500	10^{-10}	`constr_viol_tol=1e-4`
RRT*-GBO, both systems	Intermediate edge solve	500	10^{-2}	`acceptable_tol=5e-2`; `acceptable_iter=3`; `constr_viol_tol=1e-2`
RRT*-GBO, both systems	Final solve	500	10^{-10}	`constr_viol_tol=1e-4`
IPSO, spacecraft	IPSO stage	N/A	N/A	`convergence_window=50`; `convergence_tolerance`=1.0×10^{-6}; Eq. (1.157)
IPSO, spacecraft	Warmstarted GBO stage	500	10^{-10}	`constr_viol_tol=1e-4`

2.2 Evaluation Metrics

Define Computational Effort: Wall-clock CPU time, including notes on parallelization/multi-threading for MC-GBO and IPSO.

Define Solution Quality: The final objective cost J achieved (effort for spacecraft, effort + acceleration for the manipulator).

The evaluation is organized into four metric groups. Reliability and computation time are reported for all attempts, whereas solution quality is evaluated only for successful solutions. Search-behavior metrics are reported for repeated runs and for the algorithms for which the corresponding internal statistics are available. The individual results sections specify the applicable metrics and reporting subsets for each experiment.

Reliability Reliability is characterized by the success rate and failure rate over all attempted runs. IPOPT failures are classified as infeasible problem instances, maximum-iteration termination, timeouts, or other unexpected exceptions. A run is successful only when the solver returns converges with the resulting trajectory passing the applicable kinodynamic constraint checks.

Computation Time Computation time includes per test case statistics on wall-clock time, sequential-equivalent time for parallelized algorithms, time to first solution, and time to best solution. Solve-time distributions are summarized using the median, mean, standard deviation, and relevant quantiles. Quantile solve durations are reported where the number of trials supports a meaningful distributional comparison.

Solution Quality For successful solutions, the final objective cost is reported using the mean, standard deviation, median, and relevant quantiles, together with the best, worst, and interquartile-range costs when appropriate. Results also include the relative

cost gap to the best solution found across all algorithms. The objective definition is problem-specific: spacecraft results use the control effort objective, while manipulator results include the specified control effort plus acceleration terms.

Search Behavior and Diversity Search behavior is analyzed to characterize local-minimum discovery and solution diversity. Metrics include the raw number of solutions, unique basin frequencies. Algorithm-specific statistics are reported when available: for P-HO, final $\lambda = 1$ solutions, tree size; for RRT*-GBO, sampled nodes, accepted edges; and for IPSO, particle evaluations, completed outer and inner iterations, and the number of solutions sent to GBO refinement.

The complete metric inventory is defined here for consistency, but not every metric is relevant to every experiment. The results section therefore reports the metrics applicable to the algorithm and test case under study, while retaining the common definitions above for cross-algorithm comparisons.

2.3 Problem 1: Spacecraft Attitude Slew

2.3.1 Physical Parameters

The spacecraft inertia matrix denoted by $\mathbf{J}_{\text{sc}}$ is set as a spherical inertia, $\mathbf{J}_{\text{sc}} = 10000 * \mathbf{I}$ kg $\cdot$ m^2. The baseline control torque limits are set to $\boldsymbol{\tau}_{\max} = 200$ N $\cdot$ m, and the angular velocity limits are set to $\boldsymbol{\omega}_{\max} = 0.2$ rad/s (11.46 °/s). If different torque or angular velocity limits are used for specific test cases, they are explicitly stated in the scenario description. Both limits are defined in the spacecraft body frame.

2.3.2 B-Spline Discretization

For all test scenarios, the B-splines were parameterized via:

- Fixed time horizon: $t_f = 60$ s
- B-spline degree: cubic $p = 3$
- Number of free control vertices: $n_{free} = 10$
- Number of boundary conditions: $n_{bc} = 6^{\dagger}$ equating to an initial and final angular orientation, zero angular velocity, and zero angular acceleration, † except for two test cases that include additionally zero angular jerk $n_{bc} = 8$.
- Total number of control vertices: $n_{ctrl} = n_{free} + n_{bc}$
- Number of via points: $n_{via} = 61$
- Knot vector: size$(\mathbf{t}) = n_{ctrl} + p + 1$

2.3.3 Test Scenarios

show geodesic slew as proof of optimizer correctness.

Tables 2.2 and 2.3 summarize the spacecraft slew test case scenarios and the corresponding exclusion-zone definitions, respectively. The test case scenarios range from an unconstrained large-angle slew—as a baseline verification of the optimization framework—to increasingly constrained realistic maneuvers with exclusion zones and actuator limits, and finally complex configuration scenarios testing the limits of the global solver's ability to find feasible and optimal trajectories.

In addition to the listed orientation vectors of the EZs, the corresponding sensor vector is defined as ${}^{b}\hat{\mathbf{s}} = [0, 0, 1]^T$, which is transformed to the inertial frame via the current attitude of the spacecraft for comparison with the exclusion-zone directions.

Table 2.2: Summary of the spacecraft slew test case scenarios.

Test case	$\mathbf{q}_0$	$\mathbf{q}_f$	**# EZs**	**Constraints and boundary conditions**
`geodesic slew`	$[1,\ 0,\ 0,\ 0]^T$	$[0,\ 0.563122,\ 0.563122,\ -0.604803]^T$	0	EZ off; $\tau_{\max}$ off; $\omega_{\max}$ off.
`sun`	$[0,\ 0,\ 0,\ 1]^T$	$[0.7044,\ -0.0616,\ 0.0616,\ 0.7044]^T$	1	EZ on; $\tau_{\max}$ on; $\omega_{\max}$ on.
`sun and moon`	$[-0.2897,\ 0.5727,\ 0.3462,\ 0.6843]^T$	$[-0.2664,\ 0.0290,\ -0.3852,\ 0.8831]^T$	2	EZ on; $\tau_{\max}$ on; $\omega_{\max}$ on.
`sun, moon, and earth 1`	$[0.5577,\ 0.3861,\ 0.0545,\ 0.7328]^T$	$[-0.2520,\ 0.2684,\ -0.9272,\ -0.0689]^T$	3	EZ on; $\tau_{\max}$ on; $\omega_{\max}$ on.
`sun, moon, and earth 2`	$[0.5577,\ 0.3861,\ 0.0545,\ 0.7328]^T$	$[-0.2520,\ 0.2684,\ -0.9272,\ -0.0689]^T$	3	EZ on; $\tau_{\max}$ on; $\omega_{\max}$ on.
`narrow corridor`	$[0.4670,\ 0.5309,\ 0.5225,\ 0.4764]^T$	$[-0.2951,\ 0.2018,\ -0.5403,\ 0.7618]^T$	3	EZ on; $\tau_{\max}$ on; $\omega_{\max}$ on; jerk BC on.
`narrow corridor` $+\omega_{\max}$	$[0.4670,\ 0.5309,\ 0.5225,\ 0.4764]^T$	$[-0.2951,\ 0.2018,\ -0.5403,\ 0.7618]^T$	3	EZ on; $\tau_{\max}$ on; $\omega_{\max} = 0.08$ rad/s; jerk BC on.
`bug trap`	$[-0.2340,\ 0.4977,\ 0.3554,\ 0.7558]^T$	$[0.3318,\ 0.7358,\ 0.5824,\ -0.0968]^T$	5	EZ on; $\tau_{\max}$ on; $\omega_{\max}$ on.
`reverse bug trap`	$[0.3318,\ 0.7358,\ 0.5824,\ -0.0968]^T$	$[-0.2340,\ 0.4977,\ 0.3554,\ 0.7558]^T$	5	EZ on; $\tau_{\max}$ on; $\omega_{\max}$ on.
`double bug trap`	$[-0.7420,\ 0.4052,\ 0.4687,\ 0.2559]^T$	$[0.3823,\ 0.3889,\ 0.8366,\ -0.0526]^T$	10	EZ on; $\tau_{\max}$ on; $\omega_{\max}$ on.
`maze`	$[-0.2512,\ 0.5018,\ 0.3705,\ 0.7402]^T$	$[-0.5295,\ 0.6263,\ 0.5720,\ 0.0135]^T$	8	EZ on; $\tau_{\max}$ on; $\omega_{\max}$ on.

Table 2.3: Exclusion-zone definitions for the spacecraft slew test case scenarios.

Test case	k	**Inertial exclusion vector** ${}^{i}\hat{\mathbf{e}}_k$	θ_k
`geodesic slew`	0	-	-
`sun`	1	$[-0.1,\ -0.5,\ 0.7]^T$	25°

Test case	k	**Inertial exclusion vector** ${}^{i}\hat{\mathbf{e}}_k$	θ_k
`sun and moon`	1	$[0.8605,\ 0.1795,\ 0.4767]^T$	30°
	2	$[0.3743,\ 0.7954,\ 0.4767]^T$	15°
`sun, moon and earth 1`	1	$[0.2796,\ -0.8605,\ 0.4258]^T$	25°
	2	$[0.7754,\ -0.4615,\ 0.4310]^T$	15°
	3	$[-0.5176,\ 0.7392,\ -0.4310]^T$	135°
`sun, moon and earth 2`	1	$[0.8074,\ -0.4085,\ 0.4258]^T$	15°
	2	$[0.2796,\ -0.8605,\ 0.4258]^T$	25°
	3	$[-0.5176,\ 0.7392,\ -0.4310]^T$	135°
`narrow corridor`	1	$[0.7954,\ 0.6061,\ 0.0000]^T$	25°
	2	$[0.7530,\ -0.4140,\ 0.5114]^T$	40°
	3	$[0.6790,\ 0.3937,\ 0.6196]^T$	10°
`narrow corridor` $+\omega_{\max}$	1	$[0.7954,\ 0.6061,\ 0.0000]^T$	25°
	2	$[0.7530,\ -0.4140,\ 0.5114]^T$	40°
	3	$[0.6790,\ 0.3937,\ 0.6196]^T$	10°
`bug trap`	1	$[0.8577,\ 0.5141,\ 0.0000]^T$	25°
	2	$[0.7370,\ 0.2236,\ 0.6378]^T$	25°
	3	$[0.1737,\ 0.6806,\ 0.7118]^T$	25°
	4	$[0.2243,\ 0.8786,\ 0.4215]^T$	15°
	5	$[0.5775,\ 0.8162,\ 0.0201]^T$	15°
`reverse bug trap`	1	$[0.8577,\ 0.5141,\ 0.0000]^T$	25°
	2	$[0.7370,\ 0.2236,\ 0.6378]^T$	25°
	3	$[0.1737,\ 0.6806,\ 0.7118]^T$	25°
	4	$[0.2243,\ 0.8786,\ 0.4215]^T$	15°
	5	$[0.5775,\ 0.8162,\ 0.0201]^T$	15°
`double bug trap`	1	$[0.8577,\ 0.5141,\ 0.0000]^T$	25°
	2	$[0.7370,\ 0.2236,\ 0.6378]^T$	25°
	3	$[0.1737,\ 0.6806,\ 0.7118]^T$	25°
	4	$[0.2243,\ 0.8786,\ 0.4215]^T$	15°
	5	$[0.5775,\ 0.8162,\ 0.0201]^T$	15°
	6	$[-0.5156,\ 0.8517,\ 0.0936]^T$	25°
	7	$[-0.2944,\ 0.5012,\ -0.8137]^T$	25°

Test case	k	Inertial exclusion vector ${}^{i}\hat{\mathbf{e}}_k$	θ_k
	8	$[-0.8422,\ 0.3938,\ -0.3683]^T$	25°
	9	$[-0.1110,\ 0.7969,\ -0.5938]^T$	15°
	10	$[-0.1682,\ 0.9819,\ -0.0870]^T$	15°
`maze`	1	$[0.8577,\ 0.5141,\ 0.0000]^T$	25°
	2	$[0.7370,\ 0.2236,\ 0.6378]^T$	25°
	3	$[0.1737,\ 0.6806,\ 0.7118]^T$	25°
	4	$[0.2243,\ 0.8786,\ 0.4215]^T$	15°
	5	$[0.5775,\ 0.8162,\ 0.0201]^T$	15°
	6	$[-0.0827,\ 0.9722,\ -0.2191]^T$	25°
	7	$[0.4316,\ 0.4720,\ -0.7687]^T$	25°
	8	$[-0.6277,\ 0.6802,\ 0.3785]^T$	25°

2.3.4 Algorithm-Specific Hyperparameters

List what workstations were used

Tables Tables 2.4 to 2.7 list the comprehensive set of algorithm-specific hyperparameters used in this work for the spacecraft NLP. Refer back to § 1.6 and the papers by Stoneman and Lampariello [16], Pardis et al. [10], and Spiller et al. [14, 15] for further details on the respective algorithms.

Table 2.4: Monte Carlo Gradient-Based Optimization (MC-GBO) hyperparameters.

Parameter	Value	Description
N_{MC}	100	Total number of independent Monte Carlo random seeds evaluated.
N_{parallel}	4	Number of optimizations executed concurrently.
$T_{\max}$	180.0 s	Maximum allowable wall-clock computation time.

Table 2.5: Probabilistic Homotopy Optimization (P-HO) hyperparameters.

Parameter	Value	Description
N_{iters}	100	Maximum number of homotopy iterations.
$T_{\max}$	180.0 s	Maximum allowable wall-clock computation time.
$\rho_{\mathcal{A}}$	1.0	Solve-to-sample ratio threshold regulating phase transitions.

oprule extbfParameter	**Value**	**Description**
P_g	0.3	Probability of explicitly targeting the goal parameter $\boldsymbol{\lambda} = \mathbf{1}_d$.
N_{init}	20	Number of initial solutions generated at the relaxed state $\boldsymbol{\lambda} = \mathbf{0}_d$.
s	0.9	Shrink factor for the exclusion zone constraints.
Geodesic Seed	True	Initializes one baseline solve with the shortest-path geodesic slew.

Table 2.7: Algorithm-specific hyperparameters for Inverse Dynamics Particle Swarm Optimization (IPSO).

Parameter	**Value**	**Description**
Execution & Swarm Dynamics		
$N_{\text{particles}}$	30	Number of particles in the swarm population.
N_{runs}	20	Number of independent IPSO multistart runs.
$N_{\text{parallel,IPSO}}$	4	Number of concurrent IPSO processes executed.
$N_{\text{threads,IPSO}}$	2	threads allocated per IPSO process.
N_{outer}	1000	Maximum number of outer loop iterations.
N_{inner}	20	Number of internal optimization loops per outer cycle.
$r_{\text{neighborhood}}$	3	Number of adjacent particles defining the local neighborhood.
v_{fraction}	0.1	Search space fraction defining the maximum particle velocity clamping.
Geodesic Seed	True	Initializes one particle with the shortest-path geodesic trajectory.
MRP Search-Space Limits		
θ_{search}	5.0 rad	Fixed angular boundary applied componentwise to the MRP search space.
s_{bound}	1.5	Margin multiplier applied to the geodesic magnitude (used if θ_{search} is null).
GBO Refinement		
GBO Refinement	True	Enables local gradient-based refinement on the final IPSO trajectories.

Parameter	Value	Description
Feasible Only	True	Restricts GBO refinement exclusively to strictly feasible IPSO outputs.
$N_{\text{warmstarts}}$	20	Maximum number of top IPSO candidates passed to the GBO solver.
$N_{\text{parallel,GBO}}$	4	Number of concurrent GBO refinement processes executed.
Weight Scheduling ($N_{schedule} = 300$ outer cycles)		
r	0.9	Static scalar multiplier for the combined inertia and cognitive velocity update.
$w_0 \rightarrow w_f$	$1.2 \rightarrow 0.6$	Inertia weight linear decay schedule.
c_p	1.5	Static cognitive weight for personal-best attraction.
$c_{l0} \rightarrow c_{lf}$	$2.0 \rightarrow 0.0$	Local neighborhood weight linear decay schedule.
$c_{g0} \rightarrow c_{gf}$	$0.0 \rightarrow 2.0$	Global swarm weight linear growth schedule.
Constraint Handling & Convergence		
δ_0	0.05	Initial additive keep-out tolerance for exclusion zones.
γ	0.05	Adaptive decay factor for exclusion zone tolerance tightening.
N_δ	150	Cycle threshold triggering adaptive keep-out tolerance decay.
N_C	50	Sliding history window size for evaluating cost convergence.
ε_{tol}	10^{-6}	Relative cost improvement threshold required for convergence.
Penalties (Static Multipliers)		
$w_{\text{path}}, w_{\text{torque}}, w_\omega$	10.0	Respective penalty weights for EZ, torque, and velocity bounds.
$w_{\text{violation}}$	10.0	Fixed penalty applied per active constraint violation count.

Table 2.6: Algorithm-specific hyperparameters for Rapidly-exploring Random Tree Star with GBO (RRT*-GBO).

Parameter	**Value**	**Description**
N_{iters}	5000	Maximum number of tree expansion iterations.
$T_{\max}$	180.0 s	Maximum allowable wall-clock computation time.
$N_{\text{sol, max}}$	3	Maximum number of feasible solutions found before termination (only improved solutions increment).
r_{prune}	0.1 rad	Radius threshold for pruning redundant nodes.
r_{ball}	0.33 rad	Radius for neighborhood querying and rewiring consideration.
r_{connect}	0.5π rad	Goal connection threshold radius.
W_{prune}	20	Pruning window size for tracking recent node addition attempts to dynamically adjust the pruning radius.
$\varepsilon_{\text{cost}}$	10^{-4}	Minimum relative cost improvement tolerance for rewiring acceptance.
t_f	60.0 s	Final time for the edge BVP solve.
Edge n_{via}	11	Number of via points used for discretizing local edge BVP solves.
Edge n_{fcv}	3	Number of free interior control vertices per edge BVP solve.
Truncation	True	Truncates out-of-bounds samples to the edge of r_{ball} instead of rejecting.

2.4 Problem 2: 7-DOF Robotic Manipulator Ball Catching

2.4.1 Physical Parameters

A core component of the robotic manipulator model is its kinematic chain, which is defined using the Modified Denavit–Hartenberg (MDH) parameters as described in Section 1.1.2. By chaining frame multiplication using the MDH parameters and the fixed transformations connecting the inertial frame to the robot base and the last link to the end-effector, the homogeneous transformation from the inertial frame to the end-effector frame can be obtained. Table 2.8 lists the MDH parameters for all links of the robotic manipulator, seven of which depend on the joint positions q_1 to q_7.

Table 2.8: Modified DH parameters for the robotic manipulator kinematic chain.

i	α_{i-1}	a_{i-1} [mm]	θ_{i-1}	d_{i-1} [mm]
1	$0°$	0	q_1	310
2	$+90°$	0	q_2	0
3	$-90°$	0	$q_3 + 90°$	400
4	$+90°$	0	q_4	0
5	$-90°$	0	$q_5 - 90°$	390
6	$+90°$	0	$q_6 + 90°$	0
7	$+90°$	0	$q_7 - 90°$	0
8	$-90°$	0	$-180°$	118

The fixed transformations connecting the inertial frame to the robot base, frame 0, and frame 8 to the end-effector frame are:

$$\mathbf{T}_{i,0} = \begin{bmatrix} -0.498791 & -0.866722 & 0 & 760.2 \\ 0.866722 & -0.498791 & 0 & -921.7 \\ 0 & 0 & 1 & 942.0 \\ 0 & 0 & 0 & 1 \end{bmatrix}, \tag{2.1}$$

$$\mathbf{T}_{8,\mathrm{ee}} = \begin{bmatrix} -0.5411 & -0.2439 & 0.8048 & 35.0 \\ -0.1669 & -0.9065 & -0.3875 & -30.0 \\ 0.8237 & -0.3440 & 0.4499 & 181.0 \\ 0 & 0 & 0 & 1 \end{bmatrix}, \tag{2.2}$$

with the translation components are given in millimeters. The full conversion from the inertial frame to the end-effector frame is obtained by chaining the transformations as shown in (1.89).

The initial joint configuration, and end-effector pose are given by:

$$\mathbf{q}_0 = \begin{bmatrix} 0.8098 & 0.3560 & 1.0873 & -1.7558 & 0.1728 & 0.1065 & 0.2409 \end{bmatrix}^\top \text{rad}, \tag{2.3}$$

$$\mathbf{T}_{i,ee} = \begin{bmatrix} 0.636098 & 0.271960 & 0.721818 & 1.389180 \\ -0.267261 & 0.955338 & -0.124191 & -1.361460 \\ -0.723240 & -0.113721 & 0.681046 & 1.401479 \\ 0 & 0 & 0 & 1 \end{bmatrix}. \tag{2.4}$$

The model dynamics were evaluated using a Maple-generated inverse dynamics model written in C. The model returns the mass matrix, centripetal-Coriolis vector, and gravity vector—used in (1.86)—for the manipulator given the current joint positions and velocities. The link physical parameters such as mass, center of mass, and the inertia matrix are stored within this generated dynamics model, rather than supplied explicitly. The optimization uses this multi-rigid-body model directly and the link-specific physical parameters were not available in the project source. As such, these parameters are not explicitly listed in this work. However, the model does accept parameters for an added payload and joint motor inertias—which is used to calculate the cost function. The payload parameters were $m_{load} = 2.725$ kg for the mass, ${}^{ee}\mathbf{p}_{load,com} = [0, 0, 0.1]^\top$m for the center of mass position in the end-effector frame, and $\mathbf{J}_{load} = \mathbf{0}$ for the inertia matrix. The joint motor inertias were specified as $\mathbf{J}_{\mathrm{m}} = \mathrm{diag}\,(4.121601, 4.121601, 1.384900, 1.384900, 0.541000, 0.600832, 0.600832)$ for joints 1 through 7.

Table Table 2.9 lists the joint position, velocity, and torque limits for the robotic manipulator on a per joint basis.

Table 2.9: Robotic manipulator joint position, velocity, and torque limits.

Joint	$q_{\min}$ [rad]	$q_{\max}$ [rad]	$\dot{q}_{\min}$ [rad/s]	$\dot{q}_{\max}$ [rad/s]	$\tau_{\min}$ [N m]	$\tau_{\max}$ [N m]
1	-2.9671	2.9671	-1.7453	1.7453	-126	126
2	-2.0944	2.0944	-1.7453	1.7453	-76	76
3	-2.9671	2.9671	-1.7453	1.7453	-36	36
4	-2.0944	2.0944	-1.7453	1.7453	-36	36
5	-2.9671	2.9671	-2.6180	2.6180	-11	11
6	-0.7854	1.3963	-1.7453	1.7453	-16	16
7	-0.7854	2.3562	-1.7453	1.7453	-16	16

Gravity acts respective to the inertial frame as ${}^{i}\mathbf{g} = [0, 0, -9.81]^\top$ m/s^2.

2.4.2 Target (Ball) Parameters

The ball's initial position, velocity, and trajectory start time are specified as follows:

$${}^{i}\mathbf{p}_{b,0} = [4.974756, -3.563887, 1.619703]^\top \text{ m}, \tag{2.5}$$

$${}^{i}\mathbf{v}_{b,0} = [v_x, v_y, v_z]^\top \text{ m/s}, \tag{2.6}$$

$$t_0 = 0.058991 \text{ s}. \tag{2.7}$$

The velocity components v_x, v_y, and v_z are the main parameters defining each ball catching test caseThe resulting trajectory of the ball is determined by the ballistic equation:

$$ {}^{i}\mathbf{p}_b(t_b) = {}^{i}\mathbf{p}_{b,0} + {}^{i}\mathbf{v}_{b,0}t_b + \frac{1}{2}{}^{i}\mathbf{g}t_b^2, \quad t_b = t - t_0 \tag{2.8} $$

2.4.3 B-Spline Discretization

For all test scenarios, the clamped, uniform B-splines were parameterized via:

- Fixed time horizon: $t_f = [0.06, 1.8]$ s
- B-spline degree: cubic $p = 3$
- Number of free control vertices: $n_{free} = 6$
- Number of B-spline-embedded endpoint conditions: $n_{bc} = 5$. These are the initial joint position and the zero-velocity and zero-acceleration conditions at the initial and final boundaries that determine the corresponding endpoint control vertices. The terminal catching task is imposed separately through five scalar pose equalities (three position and two orientation); these constraints do not reduce the number of free B-spline control vertices, as described in § 1.5.
- Total number of control vertices: $n_{ctrl} = n_{free} + n_{bc}$
- Number of via points: $n_{via} = 20$
- Knot vector: $\text{size}(\mathbf{t}) = n_{ctrl} + p + 1$

2.4.4 Test Scenarios

The ball-velocity test cases replicate the experiments of Lampariello et al. [8]. The velocity combinations are taken from a lookup table containing 1,825 discrete velocity triplets and solutions. The table does not contain all possible combinations of the velocity component ranges; instead, it contains 22 distinct v_x values, 12 v_y values, and 24 v_z values combined into the 1,825 tested triplets.

Four algorithms are evaluated in the experiments: MC-GBO, RRT*-GBO, P-HO, and a baseline comparison method: local optimization with a trivial initial guess. The trivial guess consists of setting all free control vertices to the root— that is, setting every free control vertex to the initial joint configuration $\mathbf{q}_0$. The initial guess for the final time is selected as the time at which the ball passes closest, by measure of Euclidean distance, to the end-effector position corresponding to $\mathbf{q}_0$. This guess is naturally infeasible with respect to the NLP and is purely used as the initial point for the local optimization.

Two evaluation sets are produced. The first evaluates the available lookup-table cases broadly across the 1,825 recorded velocity triplets, running a selection of 1000 random velocity sets. Each test case is allotted a 30-minute optimization budget and ends when the first feasible solution is found or the time budget is exhausted. The second is a focused global-landscape study intended to explore each selected case more thoroughly. For this study, 100 lookup-table rows are selected evenly among the list of 1825 velocity triplets, so that the selected cases cover the range of velocity pairings throughout the

table rather than clustering in specific regions. Each selected velocity case is assigned a two-hour optimization budget with no cap on the number of solutions. The list of selected velocity cases for both evaluation sets is provided in § B.

Table 2.10 summarizes the range and grid spacing of the lookup-table entries.

Table 2.10: Ranges and grid steps of the discrete ball-velocity lookup table.

Component	**Minimum [m/s]**	**Maximum [m/s]**	**Grid step [m/s]**
v_x	−6.265625	−2.656250	0.171875
v_y	2.000000	4.750000	0.250000
v_z	2.625000	5.500000	0.125000

2.4.5 Algorithm-Specific Hyperparameters

Tables Tables 2.12 to 2.14 list the algorithm-specific hyperparameters used for the robotic manipulator NLP. The values in those tables correspond to the 100-LUT global-landscape study. The 1000-LUT study uses the same settings unless an override is listed in Table 2.11. The IPSO method is not evaluated for this problem.

Table 2.11: Hyperparameters that differ between the 1000-LUT and 100-LUT global robotic-manipulator studies.

Algorithm	**Parameter**	**1000-LUT study**	**100-LUT global study**
MC	N_{MC}	24	200
MC	$T_{\max}$	1800 s	7200 s
MC	$N_{\text{sol,max}}$	1	0 (unlimited)
P-HO	N_{iters}	50	500
P-HO	$T_{\max}$	1800 s	7200 s
P-HO	$N_{\text{sol,max}}$	1	0 (unlimited)
RRT*-GBO	N_{runs}	5000	1500
RRT*-GBO	$T_{\max}$	1800 s	7200 s
RRT*-GBO	$N_{\text{sol,max}}$	1	0 (unlimited)
Local GBO	N_{runs}	1	1
Local GBO	$T_{\max}$	400 s	400 s

Table 2.12: Monte Carlo Gradient-Based Optimization (MC-GBO) hyperparameters for the robotic manipulator.

Parameter	**Value**	**Description**
N_{MC}	200	Total number of independent Monte Carlo random seeds evaluated.
N_{parallel}	†	Number of optimizations executed concurrently.
$T_{\max}$	7200.0 s	Maximum allowable wall-clock computation time.
$N_{\text{sol, max}}$	0	Maximum number of feasible solutions; 0 denotes unlimited solutions.

† The number of parallel workers depends on workstation resources. Workstations 1–4 used 8, 6, 4, and 8 workers, respectively.

Table 2.13: Probabilistic Homotopy Optimization (P-HO) hyperparameters for the robotic manipulator.

Parameter	**Value**	**Description**
N_{iters}	500	Maximum number of homotopy iterations.
$T_{\max}$	7200.0 s	Maximum allowable wall-clock computation time.
$N_{\text{sol, max}}$	0	Maximum number of feasible solutions; 0 denotes unlimited solutions.
$\rho_{\mathcal{A}}$	0.7	Solve-to-sample ratio threshold regulating phase transitions.
P_g	0.3	Probability of explicitly targeting the goal parameter $\boldsymbol{\lambda} = \mathbf{1}_d$.
N_{init}	5	Number of initial solutions used to seed the PHO optimization.

Table 2.14: Algorithm-specific hyperparameters for Rapidly-exploring Random Tree Star with GBO (RRT*-GBO) for the robotic manipulator.

Parameter	**Value**	**Description**
N_{runs}	1500	Maximum number of tree expansion runs.
$T_{\max}$	7200.0 s	Maximum allowable wall-clock computation time.
$N_{\text{sol, max}}$	0	Maximum number of feasible solutions; 0 denotes unlimited solutions.
$r_{\text{prune,joint}}$	0.1 rad	Joint-space distance below which samples are considered redundant.
$r_{\text{ball,joint}}$	0.7 rad	Joint-space radius for local steering and neighborhood queries.
Truncation	True	Projects out-of-ball samples to the joint-space boundary instead of rejecting them.
$r_{\text{connect,position}}$	0.5 m	Position radius for the goal connection.
$r_{\text{connect,approach axis}}$	0.8 rad	Approach-axis angle for the goal connection.
W_{prune}	20	Pruning window size for tracking recent node addition attempts to dynamically adjust the pruning radius.
$\varepsilon_{\text{cost}}$	10^{-4}	Minimum relative cost improvement tolerance for rewiring acceptance.
t_f	1.741009 s	Final time for the edge BVP solve.
Edge n_{via}	11	Number of via points used for discretizing local edge BVP solves.
Edge n_{fcv}	1	Number of free interior control vertices per local edge BVP solve.
Goal-edge n_{via}	11	Number of via points used for discretizing goal-edge BVP solves.
Goal-edge n_{fcv}	3	Number of free interior control vertices per goal-edge BVP solve.

Chapter 3

Results

This chapter presents the experimental results for the two benchmark models introduced in § 1.5.1 and 1.5.2. The results are organized by physical model to analyze individually the algorithms performances on each model. Within each model, the algorithms are compared using the evaluation metrics defined in § 2.2.

3.1 Evaluation Protocol

State the number of independent runs, random seeds, test cases, and the workstation used for each result reported in this chapter.

State how failed runs, timeouts, infeasible solutions, and missing measurements are handled in the tables and plots.

Confirm the statistical summaries reported throughout this chapter: success rate, computation time, objective cost, constraint violation, and solution diversity.

The algorithms' performances over the three evaluation runs—the spacecraft slew test cases, 1000-lookup table (LUT) robotic manipulator test cases, and 100-LUT global landscape robotic manipulator test cases—are evaluated according to reliability, computation time, and solution quality. Relevant statistical summaries including success rate, computation time, objective cost, and solution diversity are reported. At the end of each evaluation set's reported data, a discussion highlighting the key findings is performed.

3.2 Spacecraft Attitude Slew Results

3.2.1 Spacecraft Results Discussion

Summarize the main spacecraft findings and connect them to the algorithm mechanisms discussed in § 1.6.

Identify which methods are most reliable, fastest, and most effective at finding low-cost or diverse solutions.

3.3 Robotic Manipulator Ball-Catching Results

The results of the robotic manipulator ball-catching experiments consist of two evaluation sets: the 1,000-LUT test cases and the 100-LUT global landscape test cases. The 1000-LUT test cases are used to evaluate the algorithms' performance on a large set ball velocities. This set's primary purpose is to compare against the results obtained during Lampariello et al.'s 2011 study [**Lampariello2011**]. In this study, a Monte Carlo-based global search was performed to establish reference solutions for the test cases. As each algorithm's individual test case terminates upon finding a first solution, this set primarily evaluates the algorithms' ability to quickly find feasible solutions and the quality of that initial solution. In contrast, the 100-LUT evaluation set focuses on a global landscape study of the solution space. This evaluation set is designed to assess the algorithms' ability to explore the solution space comprehensively and identify high-quality solutions beyond the initial feasible ones.

3.3.1 1000LUT-Runs Evaluation Set

Table 3.1: Per-test-case best-cost groups, treating costs within 0.5% of the minimum as equivalent.

Best-cost winner group	Number of rows
MC	1
P-HO	23
RRT*-GBO	8
Local GBO	0
MC + P-HO	6
MC + RRT*-GBO	6
MC + Local GBO	4
P-HO + RRT*-GBO	2
P-HO + Local GBO	4
RRT*-GBO + Local GBO	1
MC + P-HO + RRT*-GBO	12
MC + P-HO + Local GBO	56
MC + RRT*-GBO + Local GBO	12
P-HO + RRT*-GBO + Local GBO	17
MC + P-HO + RRT*-GBO + Local GBO	848
Total	1000

Make the text on this success plots larger, or make only 2 plots per row

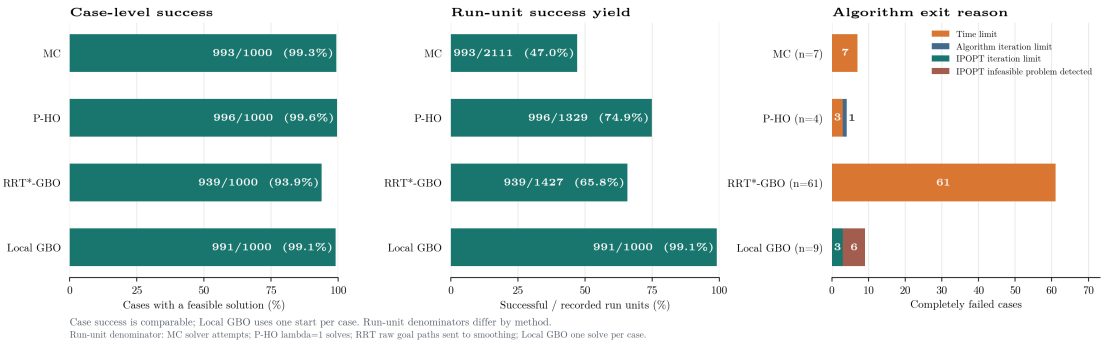


Figure 3.1: Reliability of the four algorithms over the 1,000 manipulator test cases. Case-level success, run-unit success yield, and completely failed cases are shown together with the corresponding termination reasons.

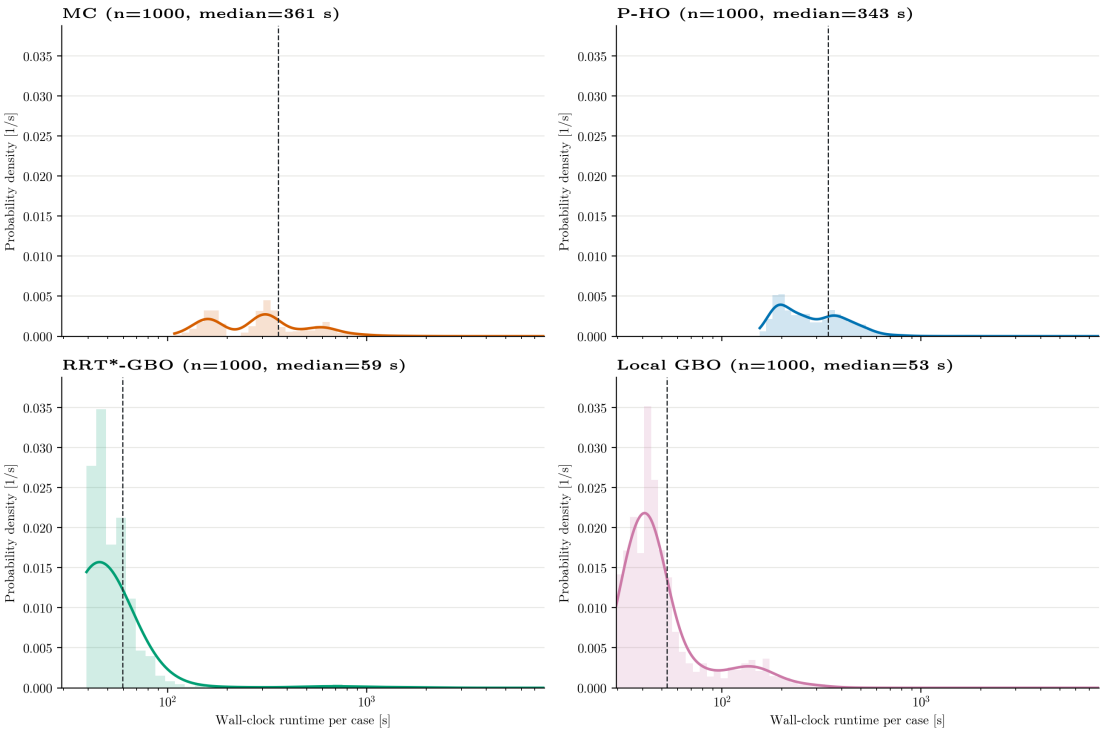


Figure 3.2: Distribution of wall-clock runtime per manipulator test case for the four algorithms. The dashed lines indicate the median runtime reported in the plot titles.

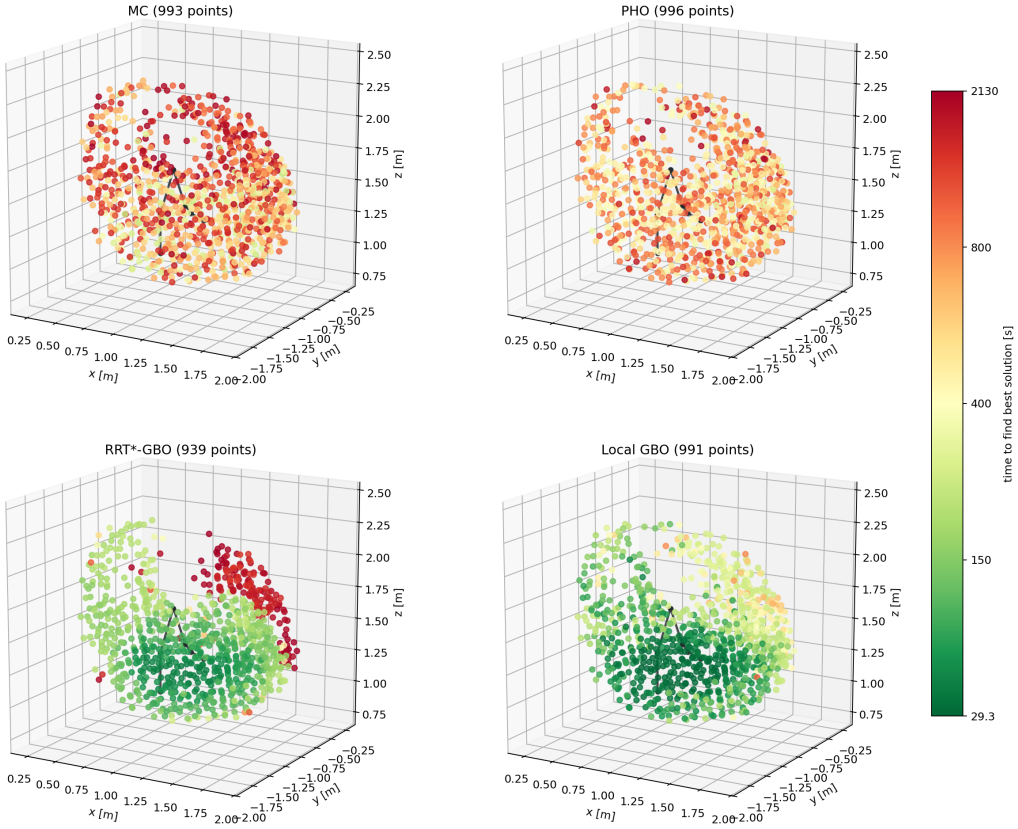


Figure 3.3: Spatial distribution of successful end-effector catch points for the four algorithms across the 1,000 manipulator test cases. The point colors encode the time required to find the best solution for each case.

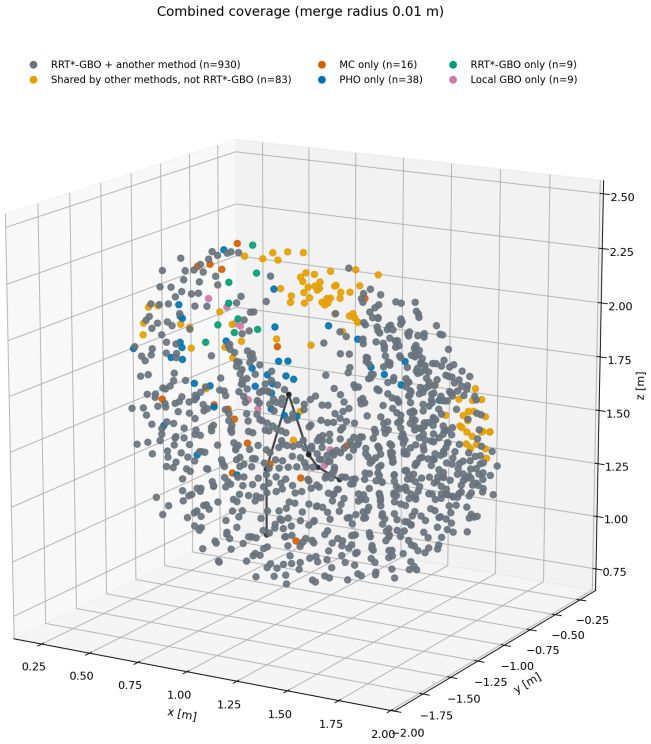


Figure 3.4: Combined coverage of successful end-effector catch points. Points are grouped according to which algorithms produce a solution within the merge radius of 0.01 m.

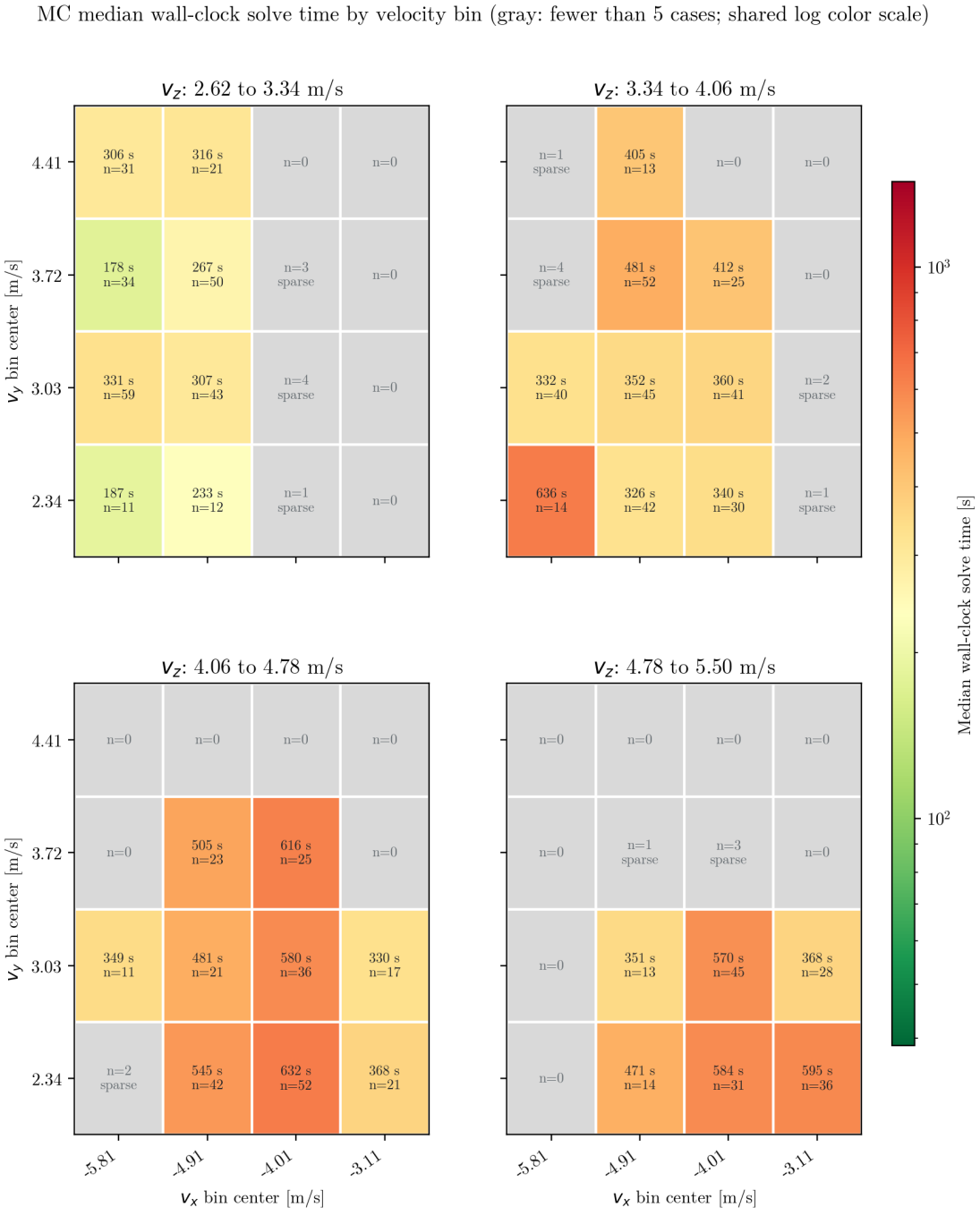


Figure 3.5: Runtime heatmap for MC over the 1,000 manipulator test cases, showing the relationship between computation time and initial velocity.

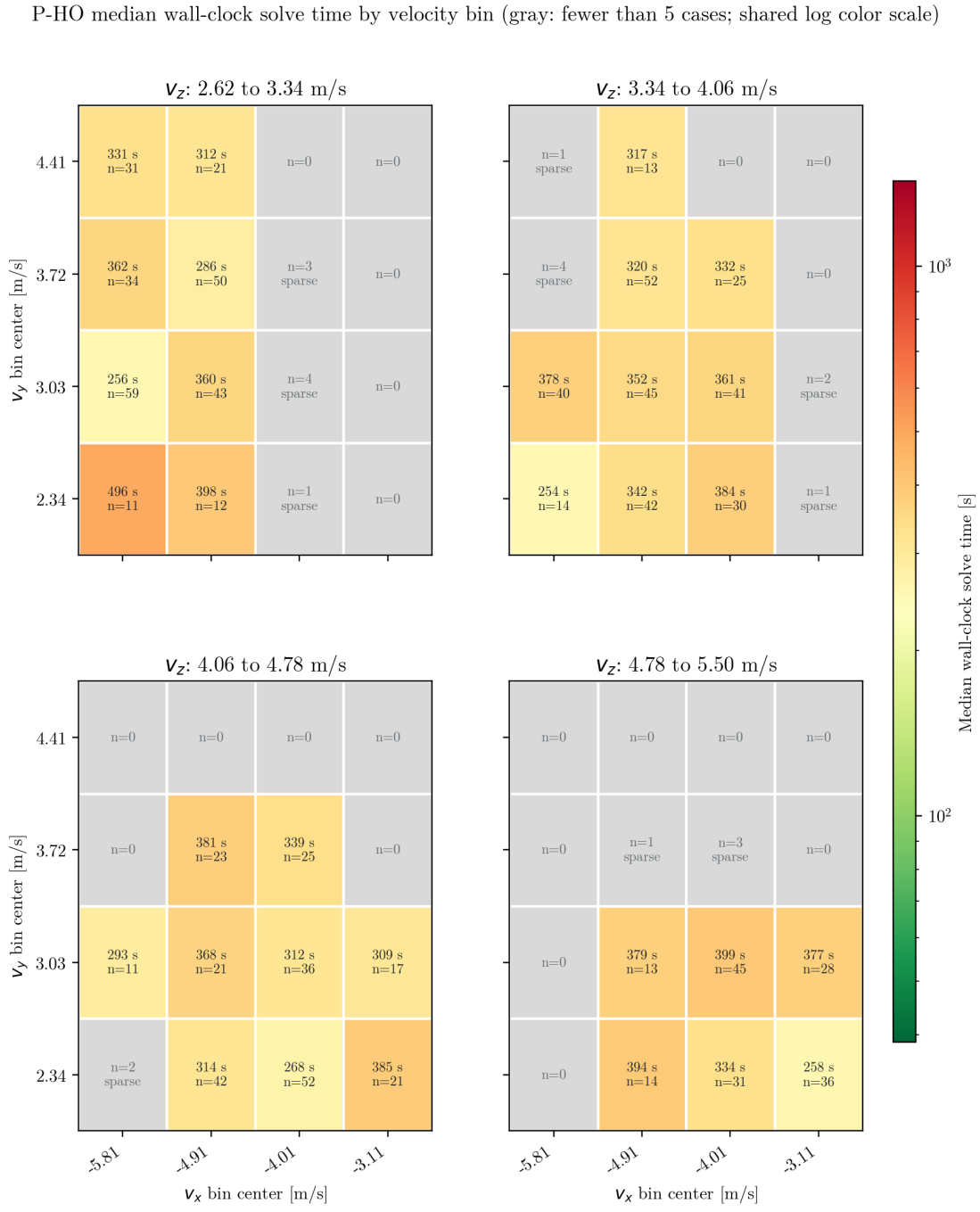


Figure 3.6: Runtime heatmap for P-HO over the 1,000 manipulator test cases, showing the relationship between computation time and initial velocity.

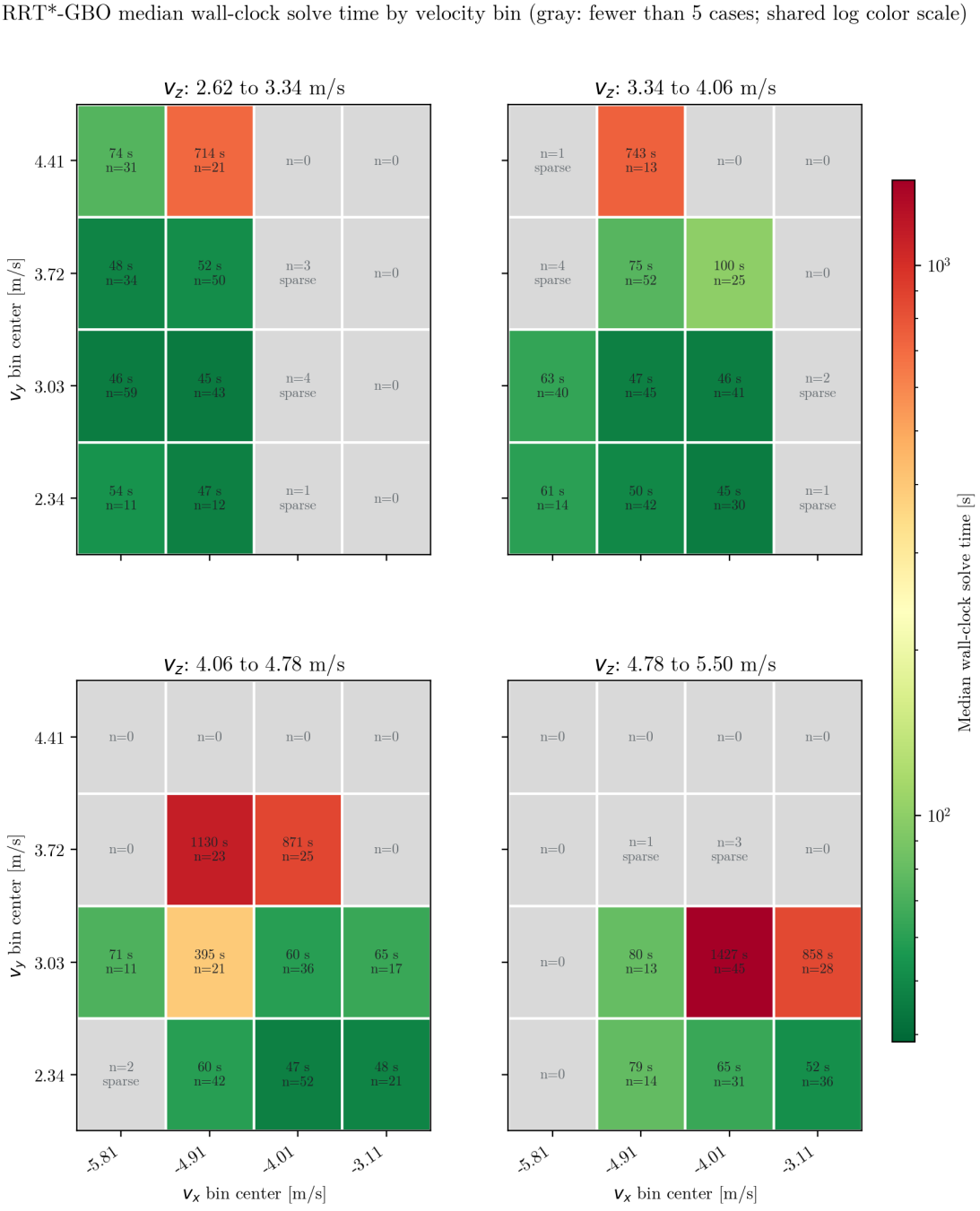


Figure 3.7: Runtime heatmap for RRT*-GBO over the 1,000 manipulator test cases, showing the relationship between computation time and initial velocity.

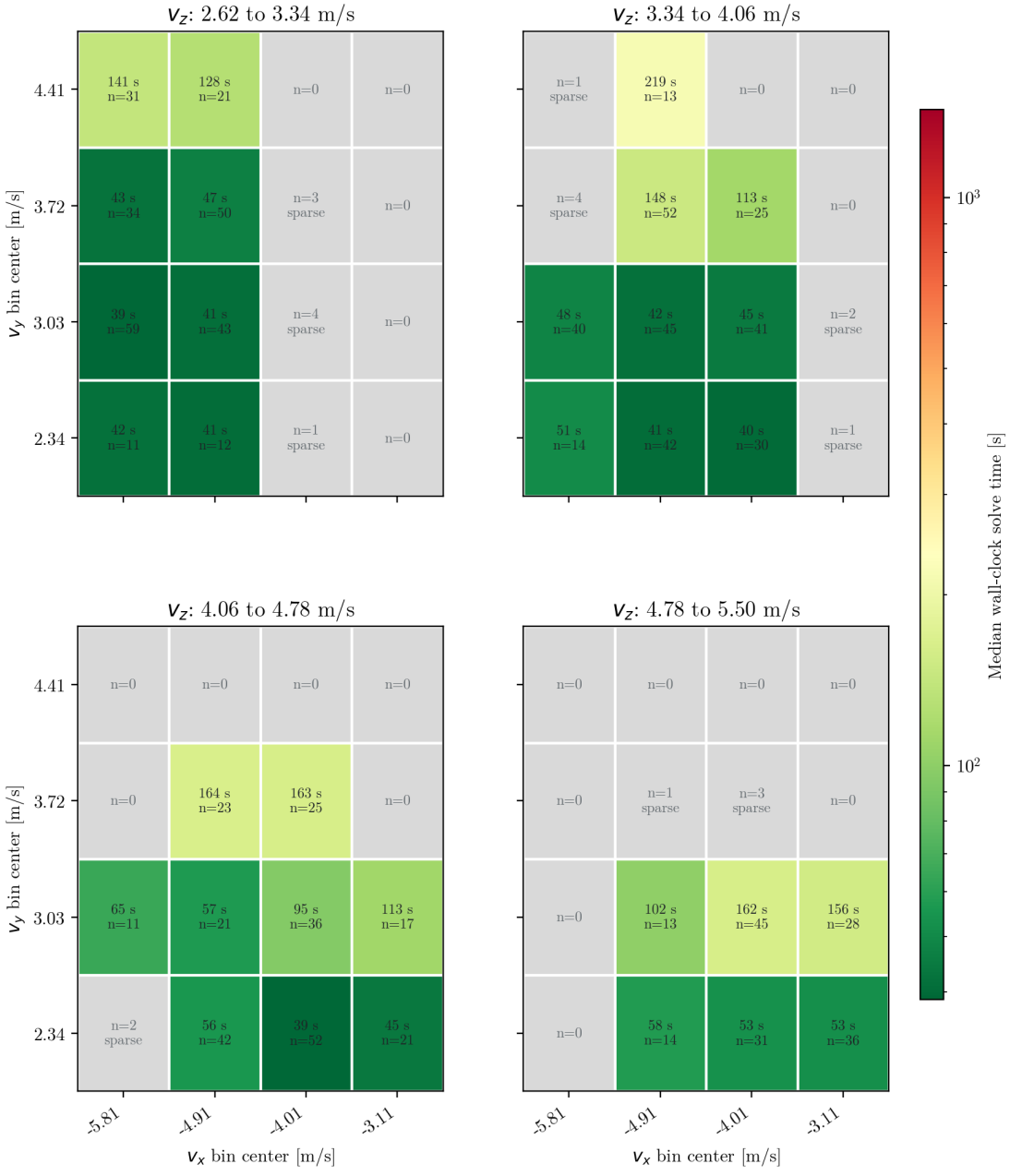


Figure 3.8: Runtime heatmap for Local GBO over the 1,000 manipulator test cases, showing the relationship between computation time and initial velocity.

Table 3.2: Per-test-case wall-clock runtime summary in seconds.

Algorithm	n	Median (s)	Q_1–Q_3 (s)	P_{10}–P_{90} (s)
MC	1000	360.9	295.6–624.9	170.7–834.1
P-HO	1000	342.7	231.3–434.3	192.2–536.3
RRT*-GBO	1000	59.5	47.3–113.2	42.9–1019.5
Local GBO	1000	53.0	42.3–132.9	35.3–190.4

Table 3.3: Per-test-case costs compared with the LUT reference; differences within 0.5% are equivalent.

Algorithm	Better	Equivalent	Worse	No feasible cost	Total rows
MC	7	943	43	7	1000
P-HO	13	956	27	4	1000
RRT*-GBO	9	899	31	61	1000
Local GBO	5	944	42	9	1000

3.3.2 100-Lut Runs Global Landscape Evaluation Set

Table 3.4: Best costs versus the LUT reference among the selected 100 LUT rows; differences within 0.5% are equivalent.

Algorithm	Better	Equivalent	Worse
MC	1	55	0
P-HO	1	55	0
RRT*-GBO	1	55	0
Local GBO	1	54	1

probably remove the limit dashed lines on all except the normalized plot. cant see.

3.3.3 Robotic Manipulator Results Discussion

Summarize the main manipulator findings and relate them to the additional dynamic and terminal constraints.

Identify which methods best balance reliability, computation time, objective quality, and catch-trajectory diversity.

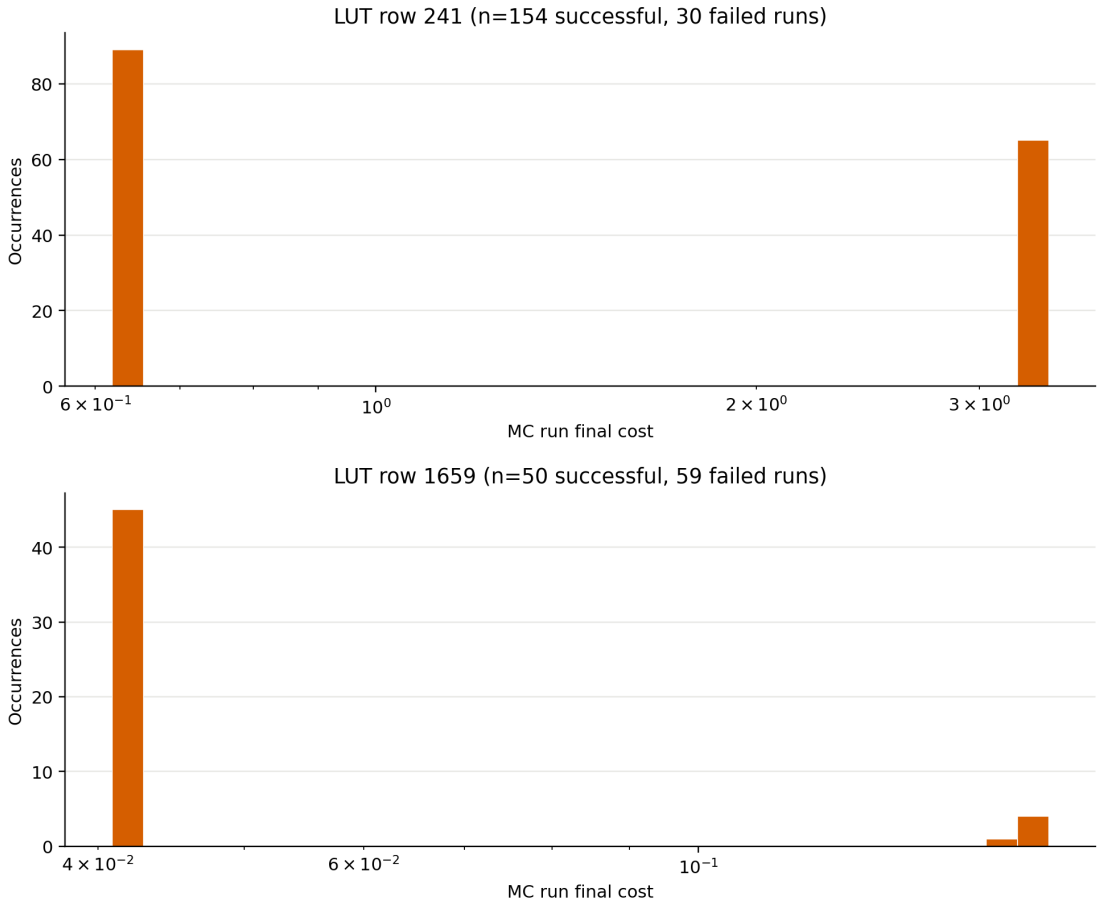


Figure 3.9: Distributions of the distinct final costs obtained by MC for LUT rows 241 and 1659. The plots show the frequency of the two observed local-minimum groups among successful runs.

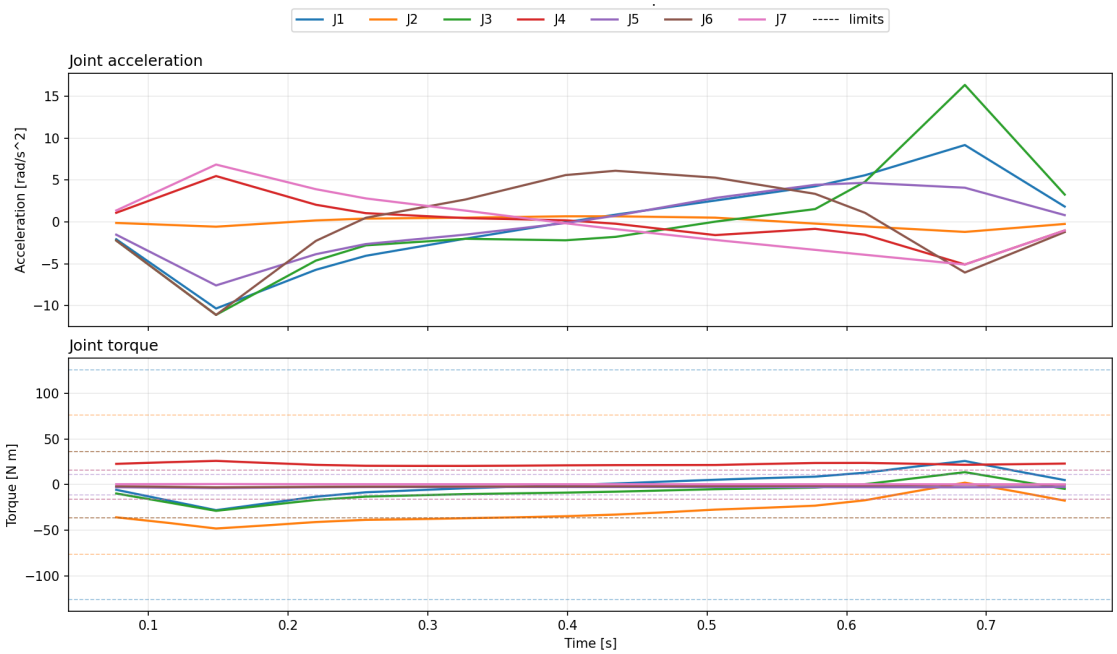


Figure 3.10: Joint accelerations and torques for a representative MC solution of LUT row 241. Dashed lines indicate the corresponding actuator limits.

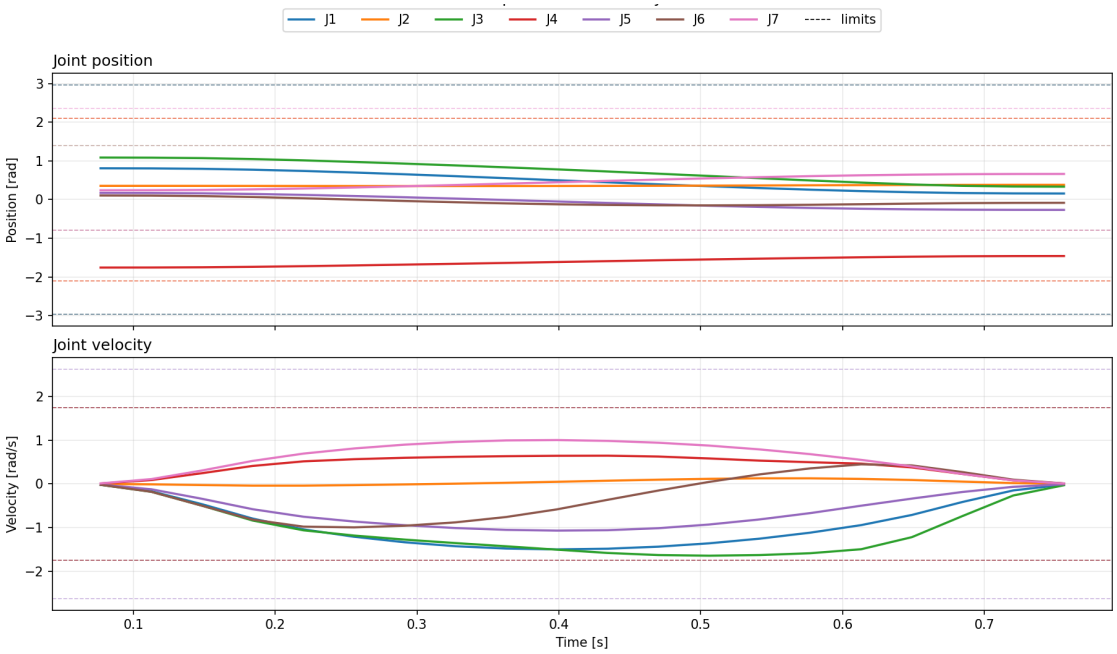


Figure 3.11: Joint positions and velocities for a representative MC solution of LUT row 241. Dashed lines indicate the corresponding state limits.

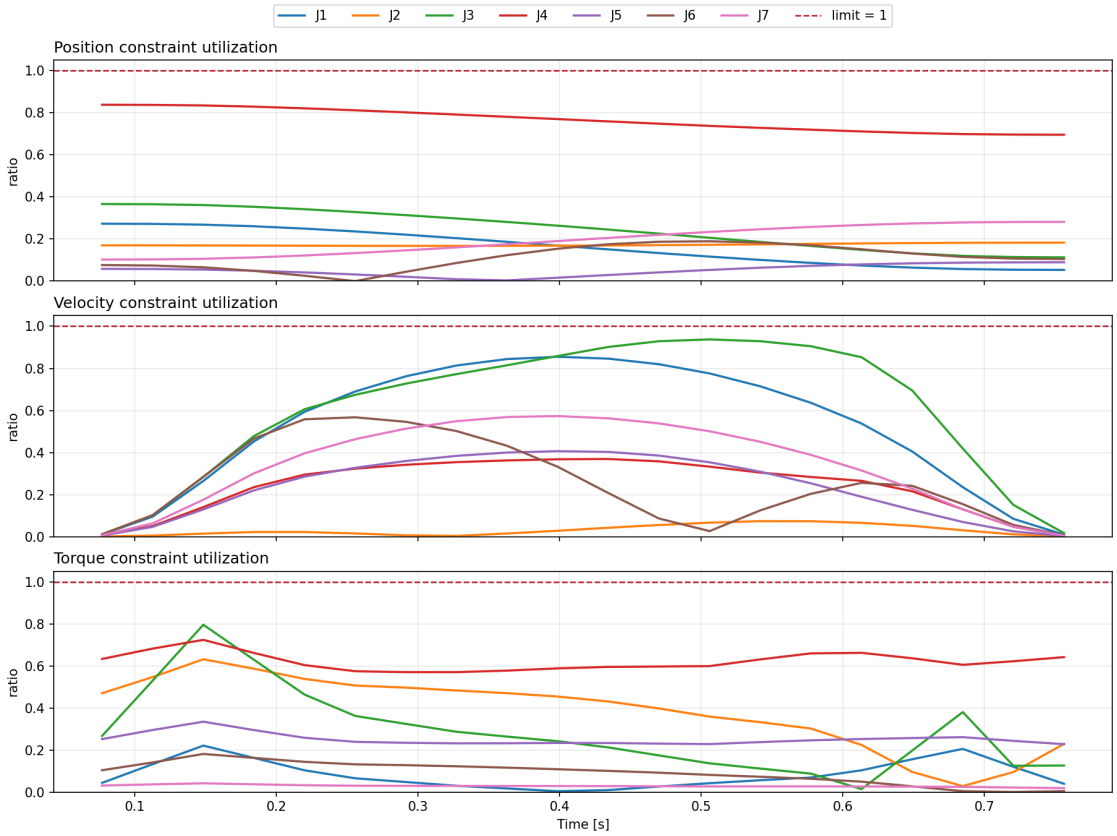


Figure 3.12: Normalized position, velocity, and torque constraint utilization for a representative MC solution of LUT row 241. The red dashed line marks the unit utilization limit.

3.4 Cross-Model Algorithm Comparison

Provide a compact comparison of algorithm rankings across reliability, solution quality, computation time, and solution diversity for both models.

Explain which conclusions generalize across models and which are specific to spacecraft attitude or manipulator ball catching.

Discuss limitations caused by different workstation configurations, sample counts, time limits, or test-case difficulty.

3.5 Chapter Summary

Summarize the most important quantitative findings without introducing new analysis. Use this section to transition to the conclusions in **??**.

Table 3.5: Distinct successful local minima for the selected LUT cases with finite results from all four methods. Costs within 0.5% of a group minimum are merged; $\times n$ gives the number of solutions in each group.

LUT row	MC	P-HO	RRT*-GBO	Local GBO
1	2.824357 (×110)	2.824357 (×28)	2.824357	2.824357
56	0.243846 (×182)	0.243846 (×32)	0.243846	0.243846
93	2.017878 (×3)	2.017878 (×23)	2.017878	2.017878
185	0.196607 (×158)	0.196607 (×45)	0.196607	0.196607
204	0.497014 (×74)	0.497009 (×67)	0.497009	0.497009
241	0.618631 (×89), 3.404011 (×65)	0.618631 (×33)	0.618631	0.618631
333	0.826499 (×167)	0.826499 (×35)	0.826499	0.826499
388	0.395060 (×133)	0.395058 (×44)	0.395058	0.395058
554	0.008523 (×140)	0.008523 (×17)	0.008523	0.008523
646	2.322030 (×154)	2.322030 (×32)	2.322030	2.322030
941	3.384427 (×72)	3.384426 (×20)	3.384426	3.384426
1014	0.031773 (×88), 3.040450	0.031773 (×13)	0.031773	0.031773
1217	0.359450 (×45)	0.359449 (×14)	0.359449	0.359449
1383	0.454051 (×38), 0.462116, 0.517761, 0.533384	0.454048 (×25)	0.454048	0.454048
1401	1.065105 (×8)	1.065100 (×12)	1.065100	1.065100
1585	0.380868 (×21)	0.380867 (×10)	0.380867	0.380867
1659	0.040879 (×42), 0.041151 (×2), 0.041467, 0.162265, 0.167889 (×2), 0.169325, 0.170625	0.040879 (×6)	0.040879	0.040879
1678	0.811470 (×38)	0.811470 (×26)	0.811470	0.811470
1751	0.038230 (×72)	0.038230 (×33)	0.038230	0.038230
1788	1.418019 (×116)	1.418018 (×36)	1.418018	1.418018

Table 3.6: Representative unique terminal configurations for LUT row 241.

$\theta(t_f)$ [deg]							J
9	22	19	-83	-15	-5	38	0.619
66	-34	8	-78	-72	-3	-5	3.404

Table 3.7: Representative unique terminal configurations for LUT row 1659.

$\theta(t_f)$ [deg]							J
47	15	71	-99	82	49	-45	0.041
56	6	78	-90	47	3	-9	0.162

Appendix A

Notation

A.1 Special operators and symbols

Identity matrix

$$\boldsymbol{I}_{n\times n} = \begin{bmatrix} 1 & 0 & \cdots & 0 \\ 0 & 1 & \cdots & 0 \\ \vdots & \vdots & \ddots & \vdots \\ 0 & 0 & \cdots & 1 \end{bmatrix} \tag{A.1}$$

Skew-symmetric matrix

$$[\mathbf{v}]_\times = \begin{bmatrix} 0 & -v_z & v_y \\ v_z & 0 & -v_x \\ -v_y & v_x & 0 \end{bmatrix} \tag{A.2}$$

Inner product

$$\langle \boldsymbol{u}, \boldsymbol{v} \rangle = \boldsymbol{u}^\mathsf{T} \boldsymbol{v} \tag{A.3}$$

Cross product

$$\mathbf{a} \times \mathbf{b} = [\mathbf{a}]_\times \mathbf{b} = \begin{bmatrix} a_2 b_3 - a_3 b_2 \\ a_3 b_1 - a_1 b_3 \\ a_1 b_2 - a_2 b_1 \end{bmatrix} \tag{A.4}$$

Mathematical spaces and sets

$\mathbb{R}^n$ n-dimensional real space

SO(3) special orthogonal group of three-dimensional rotations

SE(3) special Euclidean group of three-dimensional rigid-body motions

$\mathbb{H}$ Hamilton's quaternion algebra

S^3 unit three-sphere representing unit quaternions

$\mathcal{Q}$ manipulator joint configuration space

$\mathcal{H}_d$ d-dimensional homotopy-parameter space

$\mathcal{U}$ uniform probability distribution

Operators and common matrices

$\boldsymbol{I}$ identity matrix

$\mathbf{0}$ zero vector or zero matrix

$[\boldsymbol{v}]_\times$ skew-symmetric matrix associated with vector $\boldsymbol{v}$

Frames, rotations, and transformations

$\boldsymbol{R}$ rotation matrix

$\boldsymbol{R}_{AB}$ rotation matrix that maps coordinates from frame B to frame A

$\boldsymbol{p}_{AB}$ position of the origin of frame B expressed in frame A

$\boldsymbol{T}$ homogeneous transformation matrix

$\boldsymbol{T}_{AB}$ homogeneous transformation from frame B to frame A

Quaternions and attitude representations

$\boldsymbol{q}$ configuration/state vector; in the spacecraft model, a unit quaternion, and in the manipulator model, a joint-position vector

In the spacecraft model, $\boldsymbol{q} = [q_x, q_y, q_z, q_w]^\mathsf{T}$ satisfies $q_x^2 + q_y^2 + q_z^2 + q_w^2 = 1$

q_s scalar part of a quaternion

$\boldsymbol{q}_v$ vector part of a quaternion

$\overline{\boldsymbol{q}}$ quaternion conjugate

$\boldsymbol{C}_q(\boldsymbol{q})$ quaternion differential matrix

$\boldsymbol{\sigma}$ modified Rodrigues parameter vector

Trajectories and B-splines

$\boldsymbol{q}(t)$ time-dependent configuration/state trajectory; in the spacecraft model, a unit quaternion trajectory, and in the manipulator model, a joint-position trajectory

$\boldsymbol{x}(t)$ generic state or trajectory vector; in IPSO, a particle-position vector

$\boldsymbol{c}_i$ B-spline control vertex

$\boldsymbol{v}_i$ derivative control vertex of a B-spline

$\boldsymbol{\kappa}$ B-spline knot vector

$\boldsymbol{\varphi}(s)$ B-spline basis vector

$\boldsymbol{\Phi}$ B-spline basis matrix evaluated at all sampling or via points

$\boldsymbol{P}$ B-spline control-vertex matrix

$\boldsymbol{Q}$ matrix of trajectory values evaluated at all sampling or via points

$\boldsymbol{B}_0$, $\boldsymbol{B}_f$ initial and final boundary basis matrices mapping exterior control vertices to boundary derivatives

$\boldsymbol{Y}_0$, $\boldsymbol{Y}_f$ initial and final boundary-condition matrices

$B_{\max}$, $B_{\min}$ upper and lower bounds for a control vertex

Manipulator dynamics and kinematics

$\boldsymbol{M}(\boldsymbol{q})$ manipulator mass matrix

$\boldsymbol{J}_{\mathrm{m}}$ diagonal matrix of joint motor inertias

$\boldsymbol{J}_{\mathrm{load}}$ payload inertia matrix

$\boldsymbol{J}_{g,e}$ geometric end-effector Jacobian

$\boldsymbol{h}(\boldsymbol{q}, \dot{\boldsymbol{q}})$ centripetal-Coriolis vector

$\boldsymbol{g}(\boldsymbol{q})$ gravity vector

$\boldsymbol{\tau}$ control torque vector

IPSO and nonlinear optimization

$\boldsymbol{w}$ nonlinear-programming decision vector

$\boldsymbol{x}_{\min}$, $\boldsymbol{x}_{\max}$ lower and upper IPSO search-space bounds

$p_{\max}$ maximum scalar MRP search bound

f_j fitness of particle j

$v_{j,d}^k$ velocity of particle j in dimension d at iteration k

$p_{j,d}^k$, $l_{j,d}^k$, g_d^k personal-best, local-best, and global-best particle positions

w, c_p, c_l, c_g inertia, personal, local, and global weight coefficients

$\boldsymbol{\omega}$ angular velocity vector

H Hamiltonian in the optimal-control formulation

Craig modified Denavit–Hartenberg parameters

a_{i-1} link length

α_{i-1} link twist

d_{i-1} link offset

θ_{i-1} joint angle

In the Craig modified Denavit–Hartenberg convention, the transform from frame i to frame $i-1$ is

$$^{i-1}\boldsymbol{T}_i = \mathrm{Rot}_x(\alpha_{i-1})\,\mathrm{Trans}_x(a_{i-1})\,\mathrm{Rot}_z(\theta_i)\,\mathrm{Trans}_z(d_i). \tag{A.5}$$

A.2 Abbreviations and acronyms

3-D three dimensional space

4-D four dimensional space

SE(3) special Euclidean group in three dimensions

SO(3) special orthogonal group in three dimensions

BVP boundary value problem

CasADi Computer Algebra System for Automatic Differentiation and Optimization

DCM direction cosine matrix

DH Denavit-Hartenberg

DOF degree of freedom

EOM equations of motion

EZ exclusion zone

GBO Gradient-Based Optimization

IPOPT Interior Point OPTimizer

IPSO Inverse-dynamics Particle Swarm Optimization

KKT Karush-Kuhn-Tucker

LUT lookup table

MC-GBO Monte Carlo with Gradient-Based Optimization

MDH Modified Denavit-Hartenberg

MRBE Mechatronics, Robotics, and Biomechanical Engineering

MRP Modified Rodrigues Parameters

NLP nonlinear program

NLP nonlinear programming

OCP optimal control problem

P-HO Probabilistic Homotopy Optimization

PSO Particle Swarm Optimization

RRT Rapidly-exploring Random Tree

RRT* Rapidly-exploring Random Tree Star

RRT*-GBO Rapidly-exploring Random Tree Star with Gradient-Based Optimization

SLERP Spherical Linear Interpolation

SOTA State-of-the-Art

TUM Technische Universität München (Technical University of Munich)

Appendix B

100-Run Ball Velocity Triplets

The focused global-landscape study uses the following lookup-table rows and velocity sets.

Table B.1: Ball-velocity lookup-table rows for the one-hundred-run global-landscape evaluation.

Row	v_x	v_y	v_z	**Row**	v_x	v_y	v_z	**Row**	v_x	v_y	v_z
1	-6.26562	2.75000	2.62500	627	-5.06250	4.00000	3.75000	1235	-4.20312	2.00000	4.12500
19	-6.26562	3.00000	3.75000	646	-5.06250	4.50000	2.87500	1254	-4.20312	2.25000	4.37500
38	-6.26562	4.50000	2.87500	664	-4.89062	2.25000	3.62500	1272	-4.20312	2.50000	4.62500
56	-6.09375	3.00000	2.75000	683	-4.89062	2.50000	4.00000	1291	-4.20312	3.00000	3.50000
75	-6.09375	3.25000	3.50000	701	-4.89062	2.75000	3.62500	1309	-4.20312	3.25000	3.37500
93	-5.92188	2.50000	3.25000	720	-4.89062	3.00000	3.37500	1328	-4.20312	3.50000	3.50000
112	-5.92188	3.00000	2.75000	738	-4.89062	3.50000	2.62500	1346	-4.20312	3.75000	4.00000
130	-5.92188	3.25000	3.25000	756	-4.89062	3.75000	2.62500	1364	-4.03125	2.25000	3.87500
148	-5.92188	4.50000	2.62500	775	-4.89062	4.00000	3.00000	1383	-4.03125	2.50000	4.62500
167	-5.75000	2.75000	3.00000	793	-4.89062	4.25000	3.50000	1401	-4.03125	2.75000	5.37500
185	-5.75000	3.00000	3.37500	812	-4.71875	2.00000	4.75000	1420	-4.03125	3.00000	5.50000
204	-5.75000	3.50000	3.00000	830	-4.71875	2.50000	3.00000	1438	-4.03125	3.50000	3.50000
222	-5.75000	4.50000	2.62500	849	-4.71875	2.75000	2.87500	1457	-3.85938	2.00000	4.00000
241	-5.57812	2.50000	4.00000	867	-4.71875	3.00000	3.25000	1475	-3.85938	2.25000	4.50000
259	-5.57812	3.00000	2.62500	885	-4.71875	3.25000	4.00000	1493	-3.85938	2.50000	5.12500
277	-5.57812	3.25000	3.25000	904	-4.71875	3.50000	4.25000	1512	-3.85938	3.00000	3.62500
296	-5.57812	4.00000	3.62500	922	-4.71875	3.75000	4.12500	1530	-3.85938	3.25000	3.75000
314	-5.40625	2.25000	3.87500	941	-4.71875	4.25000	3.00000	1549	-3.85938	3.50000	4.37500
333	-5.40625	2.75000	2.87500	959	-4.54688	2.00000	5.00000	1567	-3.68750	2.25000	4.37500
351	-5.40625	3.00000	3.12500	977	-4.54688	2.25000	5.37500	1585	-3.68750	2.50000	5.00000
369	-5.40625	3.50000	2.75000	996	-4.54688	2.50000	5.50000	1604	-3.68750	3.00000	4.00000
388	-5.40625	4.00000	3.12500	1014	-4.54688	3.00000	3.37500	1622	-3.68750	3.25000	4.37500
406	-5.40625	4.75000	2.62500	1033	-4.54688	3.25000	3.25000	1641	-3.51562	2.00000	5.00000
425	-5.23438	2.50000	3.75000	1051	-4.54688	3.50000	3.25000	1659	-3.51562	2.50000	4.87500
443	-5.23438	2.75000	3.87500	1070	-4.54688	3.75000	3.50000	1678	-3.51562	3.00000	4.37500
462	-5.23438	3.25000	3.00000	1088	-4.54688	4.00000	4.00000	1696	-3.51562	3.25000	5.12500
480	-5.23438	3.75000	3.25000	1106	-4.37500	2.00000	5.12500	1714	-3.34375	2.50000	4.37500
498	-5.23438	4.25000	3.00000	1125	-4.37500	2.25000	5.50000	1733	-3.34375	2.75000	5.37500
517	-5.06250	2.00000	4.00000	1143	-4.37500	2.75000	3.25000	1751	-3.17188	2.00000	5.25000
535	-5.06250	2.50000	3.25000	1162	-4.37500	3.00000	3.62500	1770	-3.17188	2.75000	4.50000
554	-5.06250	2.75000	3.37500	1180	-4.37500	3.25000	3.37500	1788	-3.00000	2.00000	5.12500
572	-5.06250	3.00000	3.12500	1199	-4.37500	3.50000	3.62500	1807	-3.00000	2.75000	5.00000
591	-5.06250	3.50000	3.00000	1217	-4.37500	3.75000	3.87500	1825	-2.65625	2.25000	5.50000
609	-5.06250	3.75000	3.25000								

Appendix C

1000-Run Ball Velocity Triplets

The following table lists the 1,000 lookup-table rows used for the single-solution study, representing a large and diverse selection of ball velocity sets.

Table C.1: Ball-velocity lookup-table rows for the one-thousand-run single-solution evaluation.

Row	v_x	v_y	v_z	**Row**	v_x	v_y	v_z	**Row**	v_x	v_y	v_z
1	-6.26562	2.75000	2.62500	627	-5.06250	4.00000	3.75000	1221	-4.37500	3.75000	4.37500
4	-6.26562	2.75000	3.00000	630	-5.06250	4.00000	4.12500	1223	-4.37500	3.75000	4.62500
5	-6.26562	2.75000	3.12500	632	-5.06250	4.25000	2.62500	1224	-4.37500	4.00000	3.25000
6	-6.26562	2.75000	3.25000	633	-5.06250	4.25000	2.75000	1225	-4.37500	4.00000	3.37500
8	-6.26562	2.75000	3.50000	635	-5.06250	4.25000	3.00000	1226	-4.37500	4.00000	3.50000
11	-6.26562	3.00000	2.75000	636	-5.06250	4.25000	3.12500	1229	-4.37500	4.00000	3.87500
17	-6.26562	3.00000	3.50000	637	-5.06250	4.25000	3.25000	1232	-4.20312	2.00000	3.75000
22	-6.26562	3.25000	2.62500	639	-5.06250	4.25000	3.50000	1233	-4.20312	2.00000	3.87500
26	-6.26562	3.25000	3.12500	641	-5.06250	4.25000	3.75000	1234	-4.20312	2.00000	4.00000
28	-6.26562	3.25000	3.37500	642	-5.06250	4.25000	3.87500	1236	-4.20312	2.00000	4.25000
32	-6.26562	3.50000	2.87500	644	-5.06250	4.50000	2.62500	1237	-4.20312	2.00000	4.37500
34	-6.26562	4.00000	2.62500	645	-5.06250	4.50000	2.75000	1238	-4.20312	2.00000	4.50000
35	-6.26562	4.25000	2.62500	646	-5.06250	4.50000	2.87500	1240	-4.20312	2.00000	4.75000
36	-6.26562	4.50000	2.62500	658	-4.89062	2.00000	4.25000	1242	-4.20312	2.00000	5.00000
40	-6.26562	4.75000	2.62500	660	-4.89062	2.00000	4.50000	1244	-4.20312	2.00000	5.25000
45	-6.09375	2.75000	2.75000	662	-4.89062	2.25000	3.37500	1246	-4.20312	2.00000	5.50000
48	-6.09375	2.75000	3.12500	665	-4.89062	2.25000	3.75000	1248	-4.20312	2.25000	3.62500
49	-6.09375	2.75000	3.25000	666	-4.89062	2.25000	3.87500	1250	-4.20312	2.25000	3.87500
50	-6.09375	2.75000	3.37500	667	-4.89062	2.25000	4.00000	1251	-4.20312	2.25000	4.00000
51	-6.09375	2.75000	3.50000	668	-4.89062	2.25000	4.12500	1252	-4.20312	2.25000	4.12500
55	-6.09375	3.00000	2.62500	670	-4.89062	2.25000	4.37500	1253	-4.20312	2.25000	4.25000
56	-6.09375	3.00000	2.75000	671	-4.89062	2.25000	4.50000	1255	-4.20312	2.25000	4.50000
57	-6.09375	3.00000	2.87500	673	-4.89062	2.25000	4.75000	1256	-4.20312	2.25000	4.62500
58	-6.09375	3.00000	3.00000	675	-4.89062	2.50000	3.00000	1257	-4.20312	2.25000	4.75000
59	-6.09375	3.00000	3.12500	677	-4.89062	2.50000	3.25000	1258	-4.20312	2.25000	4.87500
60	-6.09375	3.00000	3.25000	679	-4.89062	2.50000	3.50000	1261	-4.20312	2.25000	5.50000
65	-6.09375	3.00000	3.87500	680	-4.89062	2.50000	3.62500	1265	-4.20312	2.50000	3.75000

Row	v_x	v_y	v_z	Row	v_x	v_y	v_z	Row	v_x	v_y	v_z
66	-6.09375	3.00000	4.00000	681	-4.89062	2.50000	3.75000	1267	-4.20312	2.50000	4.00000
67	-6.09375	3.00000	4.12500	685	-4.89062	2.50000	4.25000	1268	-4.20312	2.50000	4.12500
69	-6.09375	3.25000	2.75000	686	-4.89062	2.50000	4.37500	1269	-4.20312	2.50000	4.25000
70	-6.09375	3.25000	2.87500	687	-4.89062	2.50000	4.50000	1271	-4.20312	2.50000	4.50000
73	-6.09375	3.25000	3.25000	691	-4.89062	2.50000	5.00000	1276	-4.20312	2.75000	3.37500
74	-6.09375	3.25000	3.37500	692	-4.89062	2.50000	5.12500	1277	-4.20312	2.75000	3.50000
75	-6.09375	3.25000	3.50000	694	-4.89062	2.75000	2.75000	1279	-4.20312	2.75000	3.75000
76	-6.09375	3.50000	2.62500	698	-4.89062	2.75000	3.25000	1282	-4.20312	2.75000	4.12500
79	-6.09375	3.75000	2.62500	699	-4.89062	2.75000	3.37500	1284	-4.20312	2.75000	4.37500
83	-6.09375	4.25000	2.62500	703	-4.89062	2.75000	3.87500	1286	-4.20312	2.75000	5.12500
85	-6.09375	4.50000	2.62500	705	-4.89062	2.75000	4.12500	1287	-4.20312	2.75000	5.50000
86	-6.09375	4.50000	2.75000	707	-4.89062	2.75000	4.37500	1294	-4.20312	3.00000	3.87500
87	-6.09375	4.50000	2.87500	709	-4.89062	2.75000	4.62500	1301	-4.20312	3.00000	4.87500
88	-6.09375	4.50000	3.00000	710	-4.89062	2.75000	4.75000	1303	-4.20312	3.00000	5.12500
89	-6.09375	4.75000	2.62500	712	-4.89062	2.75000	5.12500	1304	-4.20312	3.00000	5.25000
90	-5.92188	2.50000	2.87500	713	-4.89062	2.75000	5.25000	1305	-4.20312	3.00000	5.37500
91	-5.92188	2.50000	3.00000	714	-4.89062	3.00000	2.62500	1311	-4.20312	3.25000	3.62500
93	-5.92188	2.50000	3.25000	715	-4.89062	3.00000	2.75000	1312	-4.20312	3.25000	3.75000
94	-5.92188	2.50000	3.37500	716	-4.89062	3.00000	2.87500	1316	-4.20312	3.25000	4.25000
95	-5.92188	2.50000	3.50000	719	-4.89062	3.00000	3.25000	1321	-4.20312	3.25000	4.87500
97	-5.92188	2.50000	3.75000	720	-4.89062	3.00000	3.37500	1323	-4.20312	3.25000	5.12500
98	-5.92188	2.75000	2.62500	721	-4.89062	3.00000	3.50000	1324	-4.20312	3.25000	5.25000
100	-5.92188	2.75000	2.87500	722	-4.89062	3.00000	3.62500	1327	-4.20312	3.50000	3.37500
101	-5.92188	2.75000	3.00000	723	-4.89062	3.00000	3.75000	1329	-4.20312	3.50000	3.62500
103	-5.92188	2.75000	3.25000	725	-4.89062	3.00000	5.12500	1333	-4.20312	3.50000	4.12500
104	-5.92188	2.75000	3.37500	726	-4.89062	3.25000	2.62500	1334	-4.20312	3.50000	4.25000
105	-5.92188	2.75000	3.50000	727	-4.89062	3.25000	2.75000	1335	-4.20312	3.50000	4.37500
106	-5.92188	2.75000	3.62500	729	-4.89062	3.25000	3.00000	1336	-4.20312	3.50000	4.50000
109	-5.92188	2.75000	4.00000	731	-4.89062	3.25000	3.25000	1337	-4.20312	3.50000	4.62500
110	-5.92188	2.75000	4.12500	733	-4.89062	3.25000	3.50000	1338	-4.20312	3.50000	4.75000
113	-5.92188	3.00000	2.87500	737	-4.89062	3.25000	4.62500	1339	-4.20312	3.50000	4.87500
114	-5.92188	3.00000	3.00000	739	-4.89062	3.50000	2.75000	1341	-4.20312	3.75000	3.37500
115	-5.92188	3.00000	3.12500	740	-4.89062	3.50000	2.87500	1343	-4.20312	3.75000	3.62500
116	-5.92188	3.00000	3.25000	741	-4.89062	3.50000	3.00000	1344	-4.20312	3.75000	3.75000
117	-5.92188	3.00000	3.37500	742	-4.89062	3.50000	3.12500	1345	-4.20312	3.75000	3.87500
118	-5.92188	3.00000	3.50000	743	-4.89062	3.50000	3.25000	1346	-4.20312	3.75000	4.00000
121	-5.92188	3.00000	3.87500	745	-4.89062	3.50000	3.50000	1347	-4.20312	3.75000	4.12500
125	-5.92188	3.25000	2.62500	746	-4.89062	3.50000	3.62500	1348	-4.20312	3.75000	4.25000
127	-5.92188	3.25000	2.87500	747	-4.89062	3.50000	3.75000	1350	-4.20312	3.75000	4.50000
129	-5.92188	3.25000	3.12500	748	-4.89062	3.50000	3.87500	1351	-4.03125	2.00000	3.87500
130	-5.92188	3.25000	3.25000	749	-4.89062	3.50000	4.00000	1354	-4.03125	2.00000	4.25000
131	-5.92188	3.25000	3.37500	750	-4.89062	3.50000	4.12500	1355	-4.03125	2.00000	4.37500
135	-5.92188	3.50000	2.87500	751	-4.89062	3.50000	4.25000	1356	-4.03125	2.00000	4.62500
136	-5.92188	3.50000	3.00000	753	-4.89062	3.50000	4.50000	1357	-4.03125	2.00000	4.75000
140	-5.92188	4.00000	2.62500	754	-4.89062	3.50000	4.62500	1358	-4.03125	2.00000	4.87500
141	-5.92188	4.00000	2.75000	756	-4.89062	3.75000	2.62500	1359	-4.03125	2.00000	5.00000
143	-5.92188	4.25000	2.62500	757	-4.89062	3.75000	2.75000	1360	-4.03125	2.00000	5.12500
145	-5.92188	4.25000	2.87500	760	-4.89062	3.75000	3.12500	1363	-4.03125	2.25000	3.75000

Row	v_x	v_y	v_z	Row	v_x	v_y	v_z	Row	v_x	v_y	v_z
147	-5.92188	4.25000	3.25000	761	-4.89062	3.75000	3.25000	1364	-4.03125	2.25000	3.87500
150	-5.92188	4.50000	2.87500	762	-4.89062	3.75000	3.37500	1365	-4.03125	2.25000	4.00000
152	-5.92188	4.75000	2.62500	764	-4.89062	3.75000	3.62500	1368	-4.03125	2.25000	4.37500
154	-5.75000	2.50000	2.87500	767	-4.89062	3.75000	4.12500	1369	-4.03125	2.25000	4.50000
155	-5.75000	2.50000	3.00000	770	-4.89062	3.75000	4.50000	1373	-4.03125	2.25000	5.00000
156	-5.75000	2.50000	3.12500	772	-4.89062	4.00000	2.62500	1374	-4.03125	2.50000	3.50000
159	-5.75000	2.50000	3.50000	774	-4.89062	4.00000	2.87500	1375	-4.03125	2.50000	3.62500
162	-5.75000	2.50000	3.87500	775	-4.89062	4.00000	3.00000	1376	-4.03125	2.50000	3.75000
165	-5.75000	2.75000	2.75000	778	-4.89062	4.00000	3.37500	1377	-4.03125	2.50000	3.87500
166	-5.75000	2.75000	2.87500	779	-4.89062	4.00000	3.50000	1378	-4.03125	2.50000	4.00000
168	-5.75000	2.75000	3.12500	780	-4.89062	4.00000	3.62500	1379	-4.03125	2.50000	4.12500
169	-5.75000	2.75000	3.25000	781	-4.89062	4.00000	3.75000	1382	-4.03125	2.50000	4.50000
171	-5.75000	2.75000	3.50000	782	-4.89062	4.00000	3.87500	1383	-4.03125	2.50000	4.62500
172	-5.75000	2.75000	3.62500	784	-4.89062	4.00000	4.12500	1385	-4.03125	2.50000	5.50000
173	-5.75000	2.75000	3.75000	785	-4.89062	4.00000	4.25000	1386	-4.03125	2.75000	3.37500
176	-5.75000	2.75000	4.12500	786	-4.89062	4.25000	2.62500	1388	-4.03125	2.75000	3.62500
178	-5.75000	2.75000	4.37500	788	-4.89062	4.25000	2.87500	1389	-4.03125	2.75000	3.75000
180	-5.75000	3.00000	2.75000	793	-4.89062	4.25000	3.50000	1390	-4.03125	2.75000	3.87500
181	-5.75000	3.00000	2.87500	794	-4.89062	4.25000	3.62500	1391	-4.03125	2.75000	4.00000
182	-5.75000	3.00000	3.00000	798	-4.89062	4.50000	2.87500	1394	-4.03125	2.75000	4.37500
185	-5.75000	3.00000	3.37500	799	-4.89062	4.50000	3.00000	1397	-4.03125	2.75000	4.87500
186	-5.75000	3.00000	3.50000	800	-4.89062	4.50000	3.12500	1400	-4.03125	2.75000	5.25000
188	-5.75000	3.00000	3.75000	801	-4.89062	4.50000	3.25000	1401	-4.03125	2.75000	5.37500
189	-5.75000	3.00000	3.87500	802	-4.89062	4.50000	3.37500	1402	-4.03125	2.75000	5.50000
190	-5.75000	3.00000	4.00000	804	-4.71875	2.00000	3.75000	1405	-4.03125	3.00000	3.62500
193	-5.75000	3.00000	4.37500	806	-4.71875	2.00000	4.00000	1406	-4.03125	3.00000	3.75000
194	-5.75000	3.00000	4.50000	808	-4.71875	2.00000	4.25000	1407	-4.03125	3.00000	3.87500
198	-5.75000	3.25000	3.00000	809	-4.71875	2.00000	4.37500	1408	-4.03125	3.00000	4.00000
200	-5.75000	3.25000	3.25000	810	-4.71875	2.00000	4.50000	1409	-4.03125	3.00000	4.12500
201	-5.75000	3.50000	2.62500	811	-4.71875	2.00000	4.62500	1412	-4.03125	3.00000	4.50000
202	-5.75000	3.50000	2.75000	816	-4.71875	2.25000	3.62500	1414	-4.03125	3.00000	4.75000
203	-5.75000	3.50000	2.87500	817	-4.71875	2.25000	3.75000	1415	-4.03125	3.00000	4.87500
204	-5.75000	3.50000	3.00000	818	-4.71875	2.25000	3.87500	1416	-4.03125	3.00000	5.00000
207	-5.75000	3.75000	2.87500	819	-4.71875	2.25000	4.00000	1417	-4.03125	3.00000	5.12500
208	-5.75000	3.75000	3.00000	820	-4.71875	2.25000	4.12500	1418	-4.03125	3.00000	5.25000
210	-5.75000	4.00000	2.75000	821	-4.71875	2.25000	4.25000	1419	-4.03125	3.00000	5.37500
216	-5.75000	4.25000	2.75000	822	-4.71875	2.25000	4.37500	1421	-4.03125	3.25000	3.37500
217	-5.75000	4.25000	2.87500	823	-4.71875	2.25000	4.50000	1423	-4.03125	3.25000	3.62500
218	-5.75000	4.25000	3.00000	824	-4.71875	2.25000	4.62500	1424	-4.03125	3.25000	3.75000
219	-5.75000	4.25000	3.12500	828	-4.71875	2.25000	5.12500	1425	-4.03125	3.25000	3.87500
220	-5.75000	4.25000	3.25000	831	-4.71875	2.50000	3.12500	1426	-4.03125	3.25000	4.00000
221	-5.75000	4.25000	3.37500	832	-4.71875	2.50000	3.25000	1427	-4.03125	3.25000	4.12500
222	-5.75000	4.50000	2.62500	833	-4.71875	2.50000	3.37500	1435	-4.03125	3.25000	5.12500
223	-5.75000	4.50000	2.75000	834	-4.71875	2.50000	3.50000	1436	-4.03125	3.25000	5.25000
227	-5.57812	2.25000	3.25000	835	-4.71875	2.50000	3.62500	1437	-4.03125	3.50000	3.37500
230	-5.57812	2.25000	3.62500	837	-4.71875	2.50000	3.87500	1438	-4.03125	3.50000	3.50000
234	-5.57812	2.50000	3.12500	838	-4.71875	2.50000	4.00000	1439	-4.03125	3.50000	3.62500
235	-5.57812	2.50000	3.25000	840	-4.71875	2.50000	4.25000	1440	-4.03125	3.50000	3.75000

Row	v_x	v_y	v_z	Row	v_x	v_y	v_z	Row	v_x	v_y	v_z
238	-5.57812	2.50000	3.62500	842	-4.71875	2.50000	4.50000	1441	-4.03125	3.50000	3.87500
241	-5.57812	2.50000	4.00000	844	-4.71875	2.50000	4.75000	1446	-4.03125	3.50000	4.50000
244	-5.57812	2.75000	2.62500	845	-4.71875	2.50000	4.87500	1448	-4.03125	3.50000	4.75000
245	-5.57812	2.75000	2.75000	846	-4.71875	2.50000	5.12500	1449	-4.03125	3.50000	4.87500
250	-5.57812	2.75000	3.37500	847	-4.71875	2.50000	5.25000	1452	-4.03125	3.75000	3.87500
251	-5.57812	2.75000	3.50000	848	-4.71875	2.50000	5.37500	1458	-3.85938	2.00000	4.12500
252	-5.57812	2.75000	3.62500	849	-4.71875	2.75000	2.87500	1459	-3.85938	2.00000	4.25000
253	-5.57812	2.75000	3.75000	850	-4.71875	2.75000	3.00000	1460	-3.85938	2.00000	4.37500
254	-5.57812	2.75000	3.87500	853	-4.71875	2.75000	3.37500	1462	-3.85938	2.00000	4.62500
259	-5.57812	3.00000	2.62500	854	-4.71875	2.75000	3.50000	1463	-3.85938	2.00000	4.75000
260	-5.57812	3.00000	2.75000	855	-4.71875	2.75000	3.62500	1464	-3.85938	2.00000	4.87500
262	-5.57812	3.00000	3.00000	857	-4.71875	2.75000	3.87500	1467	-3.85938	2.00000	5.25000
265	-5.57812	3.00000	3.37500	858	-4.71875	2.75000	4.00000	1468	-3.85938	2.00000	5.37500
266	-5.57812	3.00000	3.50000	861	-4.71875	2.75000	5.25000	1471	-3.85938	2.25000	4.00000
267	-5.57812	3.00000	3.62500	863	-4.71875	3.00000	2.75000	1473	-3.85938	2.25000	4.25000
271	-5.57812	3.00000	4.50000	866	-4.71875	3.00000	3.12500	1474	-3.85938	2.25000	4.37500
273	-5.57812	3.25000	2.62500	868	-4.71875	3.00000	3.37500	1476	-3.85938	2.25000	4.62500
276	-5.57812	3.25000	3.12500	870	-4.71875	3.00000	3.62500	1478	-3.85938	2.25000	4.87500
281	-5.57812	3.50000	3.00000	871	-4.71875	3.00000	3.75000	1481	-3.85938	2.50000	3.62500
282	-5.57812	3.50000	3.12500	872	-4.71875	3.00000	3.87500	1483	-3.85938	2.50000	3.87500
284	-5.57812	3.75000	2.75000	874	-4.71875	3.00000	4.75000	1484	-3.85938	2.50000	4.00000
285	-5.57812	3.75000	2.87500	875	-4.71875	3.00000	5.25000	1485	-3.85938	2.50000	4.12500
289	-5.57812	4.00000	2.75000	876	-4.71875	3.25000	2.75000	1488	-3.85938	2.50000	4.50000
290	-5.57812	4.00000	2.87500	877	-4.71875	3.25000	2.87500	1489	-3.85938	2.50000	4.62500
291	-5.57812	4.00000	3.00000	878	-4.71875	3.25000	3.00000	1491	-3.85938	2.50000	4.87500
300	-5.57812	4.25000	3.00000	879	-4.71875	3.25000	3.12500	1492	-3.85938	2.50000	5.00000
302	-5.57812	4.25000	3.25000	881	-4.71875	3.25000	3.37500	1494	-3.85938	2.50000	5.25000
304	-5.57812	4.50000	2.62500	884	-4.71875	3.25000	3.87500	1496	-3.85938	2.75000	3.50000
305	-5.57812	4.50000	2.75000	886	-4.71875	3.25000	4.37500	1498	-3.85938	2.75000	3.75000
306	-5.57812	4.50000	2.87500	887	-4.71875	3.25000	4.50000	1500	-3.85938	2.75000	4.00000
312	-5.40625	2.25000	3.62500	888	-4.71875	3.25000	4.62500	1501	-3.85938	2.75000	4.12500
315	-5.40625	2.25000	4.00000	889	-4.71875	3.25000	4.75000	1502	-3.85938	2.75000	4.25000
317	-5.40625	2.50000	2.87500	893	-4.71875	3.50000	2.87500	1505	-3.85938	2.75000	4.87500
318	-5.40625	2.50000	3.00000	895	-4.71875	3.50000	3.12500	1508	-3.85938	2.75000	5.25000
321	-5.40625	2.50000	3.37500	896	-4.71875	3.50000	3.25000	1510	-3.85938	2.75000	5.50000
324	-5.40625	2.50000	3.75000	898	-4.71875	3.50000	3.50000	1511	-3.85938	3.00000	3.50000
325	-5.40625	2.50000	3.87500	899	-4.71875	3.50000	3.62500	1514	-3.85938	3.00000	3.87500
326	-5.40625	2.50000	4.00000	901	-4.71875	3.50000	3.87500	1516	-3.85938	3.00000	4.12500
327	-5.40625	2.50000	4.12500	902	-4.71875	3.50000	4.00000	1517	-3.85938	3.00000	4.25000
330	-5.40625	2.50000	4.50000	903	-4.71875	3.50000	4.12500	1518	-3.85938	3.00000	4.37500
331	-5.40625	2.75000	2.62500	904	-4.71875	3.50000	4.25000	1519	-3.85938	3.00000	4.50000
332	-5.40625	2.75000	2.75000	910	-4.71875	3.75000	2.62500	1520	-3.85938	3.00000	4.62500
333	-5.40625	2.75000	2.87500	911	-4.71875	3.75000	2.75000	1522	-3.85938	3.00000	4.87500
335	-5.40625	2.75000	3.12500	912	-4.71875	3.75000	2.87500	1523	-3.85938	3.00000	5.00000
337	-5.40625	2.75000	3.37500	913	-4.71875	3.75000	3.00000	1525	-3.85938	3.00000	5.25000
339	-5.40625	2.75000	3.62500	914	-4.71875	3.75000	3.12500	1526	-3.85938	3.00000	5.37500
340	-5.40625	2.75000	3.75000	915	-4.71875	3.75000	3.25000	1530	-3.85938	3.25000	3.75000
341	-5.40625	2.75000	3.87500	917	-4.71875	3.75000	3.50000	1533	-3.85938	3.25000	4.12500

Row	v_x	v_y	v_z	Row	v_x	v_y	v_z	Row	v_x	v_y	v_z
342	-5.40625	2.75000	4.00000	920	-4.71875	3.75000	3.87500	1534	-3.85938	3.25000	4.25000
343	-5.40625	2.75000	4.12500	925	-4.71875	3.75000	4.50000	1535	-3.85938	3.25000	4.37500
344	-5.40625	2.75000	4.50000	926	-4.71875	3.75000	4.62500	1536	-3.85938	3.25000	4.50000
345	-5.40625	2.75000	4.62500	928	-4.71875	4.00000	2.87500	1538	-3.85938	3.25000	4.75000
347	-5.40625	3.00000	2.62500	930	-4.71875	4.00000	3.12500	1539	-3.85938	3.25000	4.87500
350	-5.40625	3.00000	3.00000	932	-4.71875	4.00000	3.37500	1540	-3.85938	3.25000	5.00000
352	-5.40625	3.00000	3.25000	934	-4.71875	4.00000	3.62500	1541	-3.85938	3.25000	5.12500
354	-5.40625	3.00000	3.50000	935	-4.71875	4.00000	3.75000	1544	-3.85938	3.50000	3.75000
358	-5.40625	3.00000	4.00000	936	-4.71875	4.00000	3.87500	1546	-3.85938	3.50000	4.00000
359	-5.40625	3.00000	4.75000	939	-4.71875	4.00000	4.25000	1548	-3.85938	3.50000	4.25000
360	-5.40625	3.25000	2.62500	940	-4.71875	4.25000	2.87500	1549	-3.85938	3.50000	4.37500
361	-5.40625	3.25000	2.75000	941	-4.71875	4.25000	3.00000	1550	-3.85938	3.50000	4.50000
362	-5.40625	3.25000	2.87500	944	-4.71875	4.25000	3.37500	1551	-3.85938	3.50000	4.62500
363	-5.40625	3.25000	3.00000	946	-4.71875	4.25000	3.62500	1552	-3.85938	3.50000	4.75000
365	-5.40625	3.25000	3.25000	949	-4.54688	2.00000	3.62500	1553	-3.85938	3.50000	4.87500
368	-5.40625	3.50000	2.62500	950	-4.54688	2.00000	3.75000	1554	-3.68750	2.00000	4.12500
370	-5.40625	3.50000	2.87500	952	-4.54688	2.00000	4.00000	1555	-3.68750	2.00000	4.25000
372	-5.40625	3.50000	3.12500	953	-4.54688	2.00000	4.12500	1556	-3.68750	2.00000	4.37500
375	-5.40625	3.75000	2.75000	954	-4.54688	2.00000	4.25000	1557	-3.68750	2.00000	4.50000
376	-5.40625	3.75000	2.87500	956	-4.54688	2.00000	4.50000	1558	-3.68750	2.00000	4.62500
377	-5.40625	3.75000	3.00000	957	-4.54688	2.00000	4.62500	1561	-3.68750	2.00000	5.00000
378	-5.40625	3.75000	3.12500	958	-4.54688	2.00000	4.75000	1563	-3.68750	2.00000	5.25000
382	-5.40625	3.75000	3.75000	961	-4.54688	2.25000	3.37500	1564	-3.68750	2.25000	4.00000
383	-5.40625	3.75000	3.87500	962	-4.54688	2.25000	3.50000	1565	-3.68750	2.25000	4.12500
384	-5.40625	4.00000	2.62500	965	-4.54688	2.25000	3.87500	1568	-3.68750	2.25000	4.50000
385	-5.40625	4.00000	2.75000	966	-4.54688	2.25000	4.00000	1569	-3.68750	2.25000	4.62500
386	-5.40625	4.00000	2.87500	967	-4.54688	2.25000	4.12500	1570	-3.68750	2.25000	4.75000
387	-5.40625	4.00000	3.00000	969	-4.54688	2.25000	4.37500	1571	-3.68750	2.25000	4.87500
388	-5.40625	4.00000	3.12500	974	-4.54688	2.25000	5.00000	1577	-3.68750	2.50000	4.00000
392	-5.40625	4.00000	3.62500	976	-4.54688	2.25000	5.25000	1578	-3.68750	2.50000	4.12500
393	-5.40625	4.00000	3.75000	977	-4.54688	2.25000	5.37500	1579	-3.68750	2.50000	4.25000
397	-5.40625	4.25000	3.00000	981	-4.54688	2.50000	3.37500	1584	-3.68750	2.50000	4.87500
399	-5.40625	4.25000	3.25000	984	-4.54688	2.50000	3.75000	1585	-3.68750	2.50000	5.00000
402	-5.40625	4.50000	2.75000	986	-4.54688	2.50000	4.00000	1586	-3.68750	2.50000	5.12500
403	-5.40625	4.50000	2.87500	988	-4.54688	2.50000	4.25000	1588	-3.68750	2.75000	3.75000
405	-5.40625	4.50000	3.12500	989	-4.54688	2.50000	4.37500	1591	-3.68750	2.75000	4.12500
407	-5.23438	2.25000	3.12500	993	-4.54688	2.50000	4.87500	1592	-3.68750	2.75000	4.25000
408	-5.23438	2.25000	3.25000	994	-4.54688	2.50000	5.25000	1594	-3.68750	2.75000	4.50000
409	-5.23438	2.25000	3.37500	995	-4.54688	2.50000	5.37500	1596	-3.68750	2.75000	4.75000
410	-5.23438	2.25000	3.50000	998	-4.54688	2.75000	3.12500	1598	-3.68750	2.75000	5.00000
411	-5.23438	2.25000	3.62500	999	-4.54688	2.75000	3.25000	1601	-3.68750	2.75000	5.50000
412	-5.23438	2.25000	3.75000	1001	-4.54688	2.75000	3.50000	1602	-3.68750	3.00000	3.75000
414	-5.23438	2.25000	4.00000	1003	-4.54688	2.75000	3.75000	1603	-3.68750	3.00000	3.87500
416	-5.23438	2.25000	4.25000	1009	-4.54688	2.75000	5.50000	1604	-3.68750	3.00000	4.00000
417	-5.23438	2.25000	4.37500	1010	-4.54688	3.00000	2.87500	1605	-3.68750	3.00000	4.12500
418	-5.23438	2.50000	2.87500	1012	-4.54688	3.00000	3.12500	1612	-3.68750	3.00000	5.00000
419	-5.23438	2.50000	3.00000	1014	-4.54688	3.00000	3.37500	1613	-3.68750	3.00000	5.12500
427	-5.23438	2.50000	4.00000	1015	-4.54688	3.00000	3.50000	1616	-3.68750	3.00000	5.50000

Row	v_x	v_y	v_z	**Row**	v_x	v_y	v_z	**Row**	v_x	v_y	v_z
430	-5.23438	2.50000	4.37500	1017	-4.54688	3.00000	3.75000	1620	-3.68750	3.25000	4.12500
431	-5.23438	2.50000	4.50000	1019	-4.54688	3.00000	4.00000	1621	-3.68750	3.25000	4.25000
433	-5.23438	2.75000	2.62500	1020	-4.54688	3.00000	4.12500	1622	-3.68750	3.25000	4.37500
436	-5.23438	2.75000	3.00000	1023	-4.54688	3.00000	4.50000	1624	-3.68750	3.25000	4.62500
439	-5.23438	2.75000	3.37500	1026	-4.54688	3.00000	4.87500	1626	-3.68750	3.25000	4.87500
440	-5.23438	2.75000	3.50000	1028	-4.54688	3.00000	5.25000	1630	-3.68750	3.25000	5.37500
441	-5.23438	2.75000	3.62500	1030	-4.54688	3.25000	2.87500	1631	-3.68750	3.50000	4.12500
443	-5.23438	2.75000	3.87500	1031	-4.54688	3.25000	3.00000	1632	-3.68750	3.50000	4.25000
444	-5.23438	2.75000	4.00000	1032	-4.54688	3.25000	3.12500	1634	-3.68750	3.50000	4.50000
445	-5.23438	2.75000	4.12500	1033	-4.54688	3.25000	3.25000	1639	-3.51562	2.00000	4.75000
449	-5.23438	3.00000	2.62500	1034	-4.54688	3.25000	3.37500	1640	-3.51562	2.00000	4.87500
450	-5.23438	3.00000	2.75000	1035	-4.54688	3.25000	3.50000	1644	-3.51562	2.25000	4.25000
452	-5.23438	3.00000	3.00000	1036	-4.54688	3.25000	3.62500	1646	-3.51562	2.25000	4.50000
454	-5.23438	3.00000	3.25000	1039	-4.54688	3.25000	4.00000	1647	-3.51562	2.25000	4.62500
455	-5.23438	3.00000	3.37500	1040	-4.54688	3.25000	4.12500	1649	-3.51562	2.25000	4.87500
457	-5.23438	3.00000	3.62500	1041	-4.54688	3.25000	4.25000	1651	-3.51562	2.25000	5.12500
458	-5.23438	3.00000	4.87500	1042	-4.54688	3.25000	4.37500	1652	-3.51562	2.50000	4.00000
459	-5.23438	3.25000	2.62500	1043	-4.54688	3.25000	4.50000	1653	-3.51562	2.50000	4.12500
461	-5.23438	3.25000	2.87500	1047	-4.54688	3.25000	5.00000	1654	-3.51562	2.50000	4.25000
464	-5.23438	3.25000	3.25000	1048	-4.54688	3.50000	2.87500	1655	-3.51562	2.50000	4.37500
466	-5.23438	3.25000	4.87500	1049	-4.54688	3.50000	3.00000	1656	-3.51562	2.50000	4.50000
468	-5.23438	3.50000	2.75000	1051	-4.54688	3.50000	3.25000	1659	-3.51562	2.50000	4.87500
469	-5.23438	3.50000	2.87500	1054	-4.54688	3.50000	3.62500	1661	-3.51562	2.50000	5.12500
471	-5.23438	3.50000	3.12500	1055	-4.54688	3.50000	3.75000	1662	-3.51562	2.75000	4.00000
472	-5.23438	3.50000	3.25000	1056	-4.54688	3.50000	3.87500	1663	-3.51562	2.75000	4.12500
473	-5.23438	3.50000	3.37500	1059	-4.54688	3.50000	4.25000	1664	-3.51562	2.75000	4.25000
474	-5.23438	3.50000	3.50000	1060	-4.54688	3.50000	4.37500	1667	-3.51562	2.75000	4.62500
476	-5.23438	3.75000	2.75000	1064	-4.54688	3.50000	4.87500	1668	-3.51562	2.75000	4.75000
478	-5.23438	3.75000	3.00000	1065	-4.54688	3.75000	2.87500	1669	-3.51562	2.75000	4.87500
483	-5.23438	3.75000	3.62500	1066	-4.54688	3.75000	3.00000	1670	-3.51562	2.75000	5.00000
484	-5.23438	3.75000	3.75000	1067	-4.54688	3.75000	3.12500	1672	-3.51562	2.75000	5.25000
485	-5.23438	3.75000	3.87500	1068	-4.54688	3.75000	3.25000	1673	-3.51562	2.75000	5.37500
486	-5.23438	4.00000	2.62500	1069	-4.54688	3.75000	3.37500	1674	-3.51562	2.75000	5.50000
487	-5.23438	4.00000	2.75000	1071	-4.54688	3.75000	3.62500	1675	-3.51562	3.00000	4.00000
488	-5.23438	4.00000	2.87500	1072	-4.54688	3.75000	3.75000	1677	-3.51562	3.00000	4.25000
490	-5.23438	4.00000	3.25000	1074	-4.54688	3.75000	4.00000	1678	-3.51562	3.00000	4.37500
491	-5.23438	4.00000	3.37500	1075	-4.54688	3.75000	4.12500	1681	-3.51562	3.00000	4.75000
492	-5.23438	4.00000	3.50000	1080	-4.54688	4.00000	3.00000	1682	-3.51562	3.00000	4.87500
494	-5.23438	4.00000	3.75000	1081	-4.54688	4.00000	3.12500	1683	-3.51562	3.00000	5.00000
497	-5.23438	4.25000	2.87500	1084	-4.54688	4.00000	3.50000	1684	-3.51562	3.00000	5.12500
499	-5.23438	4.25000	3.12500	1085	-4.54688	4.00000	3.62500	1685	-3.51562	3.00000	5.25000
501	-5.23438	4.25000	3.37500	1087	-4.54688	4.00000	3.87500	1686	-3.51562	3.00000	5.37500
502	-5.23438	4.25000	3.50000	1088	-4.54688	4.00000	4.00000	1687	-3.51562	3.00000	5.50000
506	-5.23438	4.50000	3.00000	1090	-4.54688	4.00000	4.25000	1690	-3.51562	3.25000	4.37500
508	-5.23438	4.50000	3.25000	1093	-4.54688	4.25000	3.50000	1691	-3.51562	3.25000	4.50000
509	-5.23438	4.50000	3.37500	1094	-4.54688	4.25000	3.62500	1693	-3.51562	3.25000	4.75000
511	-5.23438	4.75000	2.62500	1097	-4.37500	2.00000	4.00000	1694	-3.51562	3.25000	4.87500
514	-5.06250	2.00000	3.62500	1100	-4.37500	2.00000	4.37500	1695	-3.51562	3.25000	5.00000

Row	v_x	v_y	v_z	Row	v_x	v_y	v_z	Row	v_x	v_y	v_z
515	-5.06250	2.00000	3.75000	1101	-4.37500	2.00000	4.50000	1699	-3.34375	2.00000	4.62500
518	-5.06250	2.00000	4.12500	1104	-4.37500	2.00000	4.87500	1701	-3.34375	2.00000	4.87500
519	-5.06250	2.25000	3.12500	1105	-4.37500	2.00000	5.00000	1702	-3.34375	2.00000	5.00000
520	-5.06250	2.25000	3.25000	1107	-4.37500	2.00000	5.25000	1704	-3.34375	2.25000	4.25000
523	-5.06250	2.25000	3.62500	1108	-4.37500	2.00000	5.37500	1705	-3.34375	2.25000	4.37500
525	-5.06250	2.25000	3.87500	1109	-4.37500	2.25000	3.37500	1706	-3.34375	2.25000	4.50000
527	-5.06250	2.25000	4.12500	1110	-4.37500	2.25000	3.50000	1708	-3.34375	2.25000	4.75000
528	-5.06250	2.25000	4.25000	1112	-4.37500	2.25000	3.75000	1709	-3.34375	2.25000	4.87500
530	-5.06250	2.25000	4.50000	1117	-4.37500	2.25000	4.37500	1710	-3.34375	2.25000	5.00000
531	-5.06250	2.25000	4.62500	1119	-4.37500	2.25000	4.62500	1713	-3.34375	2.50000	4.25000
533	-5.06250	2.50000	3.00000	1120	-4.37500	2.25000	4.75000	1716	-3.34375	2.50000	4.62500
534	-5.06250	2.50000	3.12500	1121	-4.37500	2.25000	4.87500	1717	-3.34375	2.50000	4.75000
537	-5.06250	2.50000	3.50000	1123	-4.37500	2.25000	5.25000	1720	-3.34375	2.50000	5.12500
538	-5.06250	2.50000	3.62500	1124	-4.37500	2.25000	5.37500	1723	-3.34375	2.50000	5.50000
539	-5.06250	2.50000	3.75000	1126	-4.37500	2.50000	3.25000	1725	-3.34375	2.75000	4.37500
540	-5.06250	2.50000	3.87500	1129	-4.37500	2.50000	3.62500	1728	-3.34375	2.75000	4.75000
542	-5.06250	2.50000	4.12500	1131	-4.37500	2.50000	3.87500	1729	-3.34375	2.75000	4.87500
544	-5.06250	2.50000	4.37500	1132	-4.37500	2.50000	4.00000	1731	-3.34375	2.75000	5.12500
547	-5.06250	2.50000	4.75000	1133	-4.37500	2.50000	4.12500	1732	-3.34375	2.75000	5.25000
548	-5.06250	2.50000	4.87500	1135	-4.37500	2.50000	4.37500	1736	-3.34375	3.00000	4.37500
549	-5.06250	2.75000	2.75000	1136	-4.37500	2.50000	4.50000	1739	-3.34375	3.00000	4.75000
551	-5.06250	2.75000	3.00000	1137	-4.37500	2.50000	4.62500	1740	-3.34375	3.00000	4.87500
552	-5.06250	2.75000	3.12500	1140	-4.37500	2.50000	5.00000	1742	-3.34375	3.00000	5.12500
554	-5.06250	2.75000	3.37500	1143	-4.37500	2.75000	3.25000	1743	-3.34375	3.00000	5.25000
555	-5.06250	2.75000	3.50000	1144	-4.37500	2.75000	3.37500	1747	-3.17188	2.00000	4.75000
556	-5.06250	2.75000	3.62500	1145	-4.37500	2.75000	3.50000	1749	-3.17188	2.00000	5.00000
558	-5.06250	2.75000	3.87500	1147	-4.37500	2.75000	3.75000	1750	-3.17188	2.00000	5.12500
559	-5.06250	2.75000	4.00000	1150	-4.37500	2.75000	4.12500	1751	-3.17188	2.00000	5.25000
562	-5.06250	2.75000	4.37500	1151	-4.37500	2.75000	4.25000	1752	-3.17188	2.25000	4.50000
563	-5.06250	2.75000	4.50000	1152	-4.37500	2.75000	4.37500	1753	-3.17188	2.25000	4.62500
564	-5.06250	2.75000	4.62500	1153	-4.37500	2.75000	5.00000	1756	-3.17188	2.25000	5.00000
565	-5.06250	2.75000	4.75000	1154	-4.37500	2.75000	5.12500	1758	-3.17188	2.25000	5.25000
567	-5.06250	2.75000	5.00000	1155	-4.37500	2.75000	5.37500	1760	-3.17188	2.25000	5.50000
568	-5.06250	3.00000	2.62500	1156	-4.37500	2.75000	5.50000	1763	-3.17188	2.50000	4.75000
569	-5.06250	3.00000	2.75000	1158	-4.37500	3.00000	3.12500	1764	-3.17188	2.50000	4.87500
570	-5.06250	3.00000	2.87500	1160	-4.37500	3.00000	3.37500	1770	-3.17188	2.75000	4.50000
574	-5.06250	3.00000	3.37500	1161	-4.37500	3.00000	3.50000	1772	-3.17188	2.75000	4.75000
575	-5.06250	3.00000	3.50000	1162	-4.37500	3.00000	3.62500	1777	-3.17188	2.75000	5.37500
576	-5.06250	3.00000	3.62500	1163	-4.37500	3.00000	3.75000	1779	-3.17188	3.00000	4.75000
577	-5.06250	3.00000	3.75000	1164	-4.37500	3.00000	3.87500	1781	-3.17188	3.00000	5.00000
578	-5.06250	3.00000	5.00000	1166	-4.37500	3.00000	4.12500	1782	-3.17188	3.00000	5.12500
580	-5.06250	3.25000	2.75000	1167	-4.37500	3.00000	4.25000	1784	-3.17188	3.00000	5.37500
583	-5.06250	3.25000	3.12500	1172	-4.37500	3.00000	4.87500	1785	-3.17188	3.00000	5.50000
586	-5.06250	3.25000	3.50000	1173	-4.37500	3.00000	5.00000	1787	-3.00000	2.00000	5.00000
591	-5.06250	3.50000	3.00000	1176	-4.37500	3.00000	5.37500	1788	-3.00000	2.00000	5.12500
593	-5.06250	3.50000	3.25000	1177	-4.37500	3.25000	3.00000	1790	-3.00000	2.00000	5.37500
594	-5.06250	3.50000	3.37500	1179	-4.37500	3.25000	3.25000	1797	-3.00000	2.25000	5.37500
596	-5.06250	3.50000	3.62500	1180	-4.37500	3.25000	3.37500	1798	-3.00000	2.25000	5.50000

Row	v_x	v_y	v_z	**Row**	v_x	v_y	v_z	**Row**	v_x	v_y	v_z
597	-5.06250	3.50000	3.75000	1181	-4.37500	3.25000	3.50000	1799	-3.00000	2.50000	4.75000
598	-5.06250	3.50000	3.87500	1185	-4.37500	3.25000	4.00000	1801	-3.00000	2.50000	5.00000
599	-5.06250	3.50000	4.12500	1188	-4.37500	3.25000	4.50000	1802	-3.00000	2.50000	5.12500
601	-5.06250	3.50000	4.37500	1189	-4.37500	3.25000	4.62500	1803	-3.00000	2.50000	5.25000
602	-5.06250	3.50000	4.62500	1193	-4.37500	3.25000	5.12500	1804	-3.00000	2.50000	5.37500
603	-5.06250	3.50000	4.75000	1195	-4.37500	3.50000	3.12500	1806	-3.00000	2.75000	4.87500
604	-5.06250	3.75000	2.62500	1197	-4.37500	3.50000	3.37500	1807	-3.00000	2.75000	5.00000
605	-5.06250	3.75000	2.75000	1198	-4.37500	3.50000	3.50000	1808	-3.00000	2.75000	5.12500
606	-5.06250	3.75000	2.87500	1201	-4.37500	3.50000	3.87500	1811	-3.00000	2.75000	5.50000
608	-5.06250	3.75000	3.12500	1203	-4.37500	3.50000	4.12500	1812	-2.82812	2.00000	5.12500
611	-5.06250	3.75000	3.50000	1204	-4.37500	3.50000	4.25000	1813	-2.82812	2.00000	5.25000
613	-5.06250	3.75000	3.75000	1206	-4.37500	3.50000	4.50000	1814	-2.82812	2.00000	5.37500
614	-5.06250	3.75000	3.87500	1208	-4.37500	3.50000	4.75000	1818	-2.82812	2.25000	5.37500
616	-5.06250	3.75000	4.37500	1212	-4.37500	3.75000	3.25000	1819	-2.82812	2.25000	5.50000
618	-5.06250	4.00000	2.62500	1213	-4.37500	3.75000	3.37500	1821	-2.82812	2.50000	5.25000
622	-5.06250	4.00000	3.12500	1214	-4.37500	3.75000	3.50000	1822	-2.82812	2.50000	5.37500
624	-5.06250	4.00000	3.37500	1215	-4.37500	3.75000	3.62500	1823	-2.82812	2.50000	5.50000
625	-5.06250	4.00000	3.50000	1217	-4.37500	3.75000	3.87500	1825	-2.65625	2.25000	5.50000
626	-5.06250	4.00000	3.62500								

List of Algorithms

1.1 MC_GBO 33
1.2 RRT*_GBO 38
1.3 connect_to_goal(x_{last}, x_{goal}) 38
1.4 neighborhood_query($x_{new}, r_{prune}, tree$) 38
1.5 rewire_query($x_{new}, \boldsymbol{x}_{near}$) 38
1.6 P_HO 43
1.7 sampleAttempt 43
1.8 IPSO_solve 47
1.9 IPSO_GBO 48

List of Figures

1.1 Rotation of the body frame $\{\hat{\mathbf{B}}_x, \hat{\mathbf{B}}_y, \hat{\mathbf{B}}_z\}$ by angle α about the z-axis to the inertial frame $\{\hat{\mathbf{I}}_x, \hat{\mathbf{I}}_y, \hat{\mathbf{I}}_z\}$. 2

1.2 Initial frame a, rotated frame b, unit rotation axis $\hat{\mathbf{u}}$, rotation angle θ, and unit quaternion rotation $\mathbf{q}_{ba}$. 6

1.3 Modified Denavit–Hartenberg (MDH) parameters for consecutive link frames. 10

1.4 1-D cubic B-spline trajectory demonstrating the local support property and control vertex influence at an example point $t = 0.55\,\text{s}$. The active knot span $[t_7, t_8] = [0.50, 0.62]\,\text{s}$ dictates that the curve value is determined by control vertices $\mathbf{c}_4$ through $\mathbf{c}_7$, with basis weights $N_{i,3}$ corresponding to active basis functions. Basis weights vary depending on the value of the basis functions for the evaluated point t, respective only to its location within the active knot span. Clamping the knot vector ($\{t_0, \ldots, t_3\}$ and $\{t_{11}, \ldots, t_{14}\}$) anchors the curve endpoints to $\mathbf{c}_0$ and $\mathbf{c}_{10}$, while repeating the initial and final three control vertex values enforces zero derivatives up to second order ($\dot{\mathbf{q}} = \mathbf{0}, \ddot{\mathbf{q}} = \mathbf{0}$) at the boundaries. The grey shaded region indicates local convex hull bounds [5, 10]. 13

1.5 Comparison of uniform and naïve quaternion sampling. The spatial projection, axis–angle distribution, and empirical rotation-angle distribution illustrate the effect of sampling method on the resulting distribution over SO(3). 21

1.6 Illustration of MC-GBO showing multiple independently and randomly generated initial guess trajectories (dashed) and their corresponding optimized final trajectories (solid) in an environment with obstacles. Each refined trajectory is a local GBO-based optimum; global search is achieved through the multiple independent initial guesses. 33

1.7 Illustration of the main stages of the RRT*-GBO tree construction and trajectory refinement. 35

1.8 Illustration of P-HO demonstrating stochastic sampling and the resulting ability to navigate between different homotopy classes in the solution space. The colored trajectories show candidate solutions obtained as the homotopy parameter progresses from the relaxed problem to the fully constrained goal problem. 42

1.9 Illustration of the IPSO particle update, showing the inertia, individual-best, and social-best vector components contributing to the velocity update. 44

3.1 Reliability of the four algorithms over the 1,000 manipulator test cases. Case-level success, run-unit success yield, and completely failed cases are shown together with the corresponding termination reasons. 69

3.2 Distribution of wall-clock runtime per manipulator test case for the four algorithms. The dashed lines indicate the median runtime reported in the plot titles. 69

3.3 Spatial distribution of successful end-effector catch points for the four algorithms across the 1,000 manipulator test cases. The point colors encode the time required to find the best solution for each case. 70

3.4 Combined coverage of successful end-effector catch points. Points are grouped according to which algorithms produce a solution within the merge radius of 0.01 m. 71

3.5 Runtime heatmap for MC over the 1,000 manipulator test cases, showing the relationship between computation time and initial velocity. 72

3.6 Runtime heatmap for P-HO over the 1,000 manipulator test cases, showing the relationship between computation time and initial velocity. 73

3.7 Runtime heatmap for RRT*-GBO over the 1,000 manipulator test cases, showing the relationship between computation time and initial velocity. . 74

3.8 Runtime heatmap for Local GBO over the 1,000 manipulator test cases, showing the relationship between computation time and initial velocity. . 75

3.9 Distributions of the distinct final costs obtained by MC for LUT rows 241 and 1659. The plots show the frequency of the two observed local-minimum groups among successful runs. 77

3.10 Joint accelerations and torques for a representative MC solution of LUT row 241. Dashed lines indicate the corresponding actuator limits. 78

3.11 Joint positions and velocities for a representative MC solution of LUT row 241. Dashed lines indicate the corresponding state limits. 78

3.12 Normalized position, velocity, and torque constraint utilization for a representative MC solution of LUT row 241. The red dashed line marks the unit utilization limit. 79

List of Tables

2.1 Solver-specific stopping criteria used in the experiments. 52

2.2 Summary of the spacecraft slew test case scenarios. 54

2.3 Exclusion-zone definitions for the spacecraft slew test case scenarios. . . . 55

2.4 Monte Carlo Gradient-Based Optimization (MC-GBO) hyperparameters. 56

2.5 Probabilistic Homotopy Optimization (P-HO) hyperparameters. 57

2.6 Algorithm-specific hyperparameters for Rapidly-exploring Random Tree Star with GBO (RRT*-GBO). 57

2.7 Algorithm-specific hyperparameters for Inverse Dynamics Particle Swarm Optimization (IPSO). 58

2.8 Modified DH parameters for the robotic manipulator kinematic chain. . . 60

2.9 Robotic manipulator joint position, velocity, and torque limits. 61

2.10 Ranges and grid steps of the discrete ball-velocity lookup table. 63

2.11 Hyperparameters that differ between the 1000-LUT and 100-LUT global robotic-manipulator studies. 63

2.13 Probabilistic Homotopy Optimization (P-HO) hyperparameters for the robotic manipulator. 63

2.12 Monte Carlo Gradient-Based Optimization (MC-GBO) hyperparameters for the robotic manipulator. 65

2.14 Algorithm-specific hyperparameters for Rapidly-exploring Random Tree Star with GBO (RRT*-GBO) for the robotic manipulator. 66

3.1 Per-test-case best-cost groups, treating costs within 0.5% of the minimum as equivalent. 68

3.2 Per-test-case wall-clock runtime summary in seconds. 76

3.3 Per-test-case costs compared with the LUT reference; differences within 0.5% are equivalent. 76

3.4 Best costs versus the LUT reference among the selected 100 LUT rows; differences within 0.5% are equivalent. 76

3.5 Distinct successful local minima for the selected LUT cases with finite results from all four methods. Costs within 0.5% of a group minimum are merged; $\times n$ gives the number of solutions in each group. 81

3.6 Representative unique terminal configurations for LUT row 241. 81
3.7 Representative unique terminal configurations for LUT row 1659. 82

B.1 Ball-velocity lookup-table rows for the one-hundred-run global-landscape evaluation. 90

C.1 Ball-velocity lookup-table rows for the one-thousand-run single-solution evaluation. 91

References

[1] Andersson, Joel A. E. et al. "CasADi: A Software Framework for Nonlinear Optimization and Optimal Control." In: *Mathematical Programming Computation* 11.1 (2019), pp. 1–36. DOI: 10.1007/s12532-018-0139-4.

[2] Betts, John T. *Practical Methods for Optimal Control and Estimation Using Nonlinear Programming*. 2nd ed. SIAM, 2010.

[3] Boor, Carl de. *A Practical Guide to Splines*. Vol. 27. Applied Mathematical Sciences. New York, NY: Springer-Verlag, 1978.

[4] Craig, John J. *Introduction to Robotics: Mechanics and Control*. 3rd ed. Upper Saddle River, NJ: Pearson/Prentice Hall, 2005, p. 400. ISBN: 0-201-54361-3.

[5] Diebel, James. *Representing Attitude: Euler Angles, Unit Quaternions, and Rotation Vectors*. Tech. rep. Stanford University, Oct. 2006.

[6] Karaman, Sertac and Frazzoli, Emilio. "Sampling-based Algorithms for Optimal Motion Planning." In: *The International Journal of Robotics Research* 30.7 (2011), pp. 846–894. DOI: 10.1177/0278364911406761.

[7] Kuffner, James J. "Effective Sampling and Distance Metrics for 3D Rigid Body Path Planning." In: *Proceedings. 2004 IEEE International Conference on Robotics and Automation*. Vol. 4. 2004, pp. 3993–3998. DOI: 10.1109/ROBOT.2004.1308895.

[8] Lampariello, Roberto et al. "Trajectory Planning for Optimal Robot Catching in Real-Time." In: *2011 IEEE International Conference on Robotics and Automation*. 2011, pp. 3719–3726. DOI: 10.1109/ICRA.2011.5980114.

[9] LaValle, Steven M. *Planning Algorithms*. Cambridge, UK: Cambridge University Press, 2006.

[10] Pardis, Shayan, Chignoli, Matthew, and Kim, Sangbae. "Probabilistic Homotopy Optimization for Dynamic Motion Planning." In: *2024 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*. 2024. DOI: 10.1109/IROS58592.2024.10802528.

[11] Rao, Anil V. "A Survey of Numerical Methods for Optimal Control." In: *Advances in the Astronautical Sciences* 135.1 (2009), pp. 497–528.

[12] Schuster, Malcolm D. "A Survey of Attitude Representations." In: *The Journal of the Astronautical Sciences* 41.4 (1993), pp. 439–517.

[13] Shoemake, Ken. "Uniform Random Rotations." In: *Graphics Gems III*. Ed. by Kirk, David. Boston, MA: Academic Press, 1992. Chap. III.6, pp. 124–132. ISBN: 0-12-409671-9.

[14] Spiller, Dario, Ansalone, Luigi, and Curti, Fabio. "Particle Swarm Optimization for Time-Optimal Spacecraft Reorientation with Keep-Out Cones." In: *Journal of Guidance, Control, and Dynamics* 39.2 (2016), pp. 312–325. DOI: 10.2514/1.G001228.

[15] Spiller, Dario and Curti, Fabio. "Inverse-Dynamics Particle Swarm Optimization for Real Time Optimal Control: Challenges and Opportunities." In: *69th International Astronautical Congress (IAC)*. Paper IAC-18,C1,IP,19,x47444. 2018.

[16] Stoneman, Samantha and Lampariello, Roberto. "Embedding Nonlinear Optimization in RRT* for Optimal Kinodynamic Planning." In: *2014 IEEE 53rd Annual Conference on Decision and Control (CDC)*. 2014, pp. 3737–3744.

[17] Wächter, Andreas and Biegler, Lorenz T. "On the Implementation of an Interior-Point Filter Line-Search Algorithm for Large-Scale Nonlinear Programming." In: *Mathematical Programming* 106.1 (2006), pp. 25–57. DOI: 10.1007/s10107-004-0559-y.